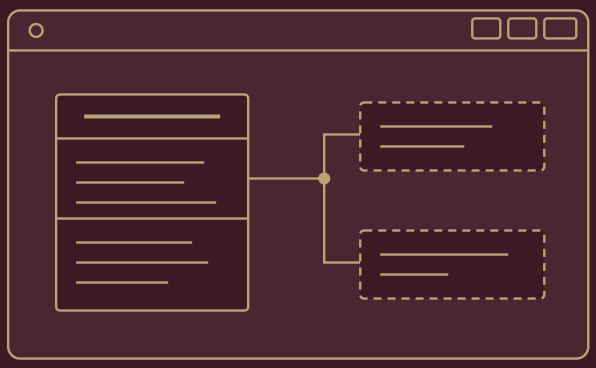
DERS KİTABI

YBS 2. SINIF

NESNE TABANLI PROGRAMLAMA II



Yunus Emre Demir

DERSİN ÖĞRETİM ÜYESİ

Dr. Mesut TOĞAÇAR

ÜCRETSİZDİR · PARA İLE SATILAMAZ

YBS 2. SINIF

NESNE TABANLI PROGRAMLAMA II

Yunus Emre Demir

Ücretsizdir · Para ile satılamaz

Künye

KİTABIN ADI	YBS 2. SINIF NESNE TABANLI PROGRAMLAMA II
YAZAR	Yunus Emre Demir
HAZIRLAYAN	Yunus Emre Demir
DERSİN ÖĞRETİM ÜYESİ	Dr. Mesut TOĞAÇAR
TÜR	Ders Kitabı
ÜCRET	Ücretsizdir
İLETİŞİM	0543 781 38 95 yunusemredemir051@gmail.com
TARİH	27 Ekim 2026
SAYFA SAYISI	110

Bu kitap ücretsizdir; para ile satılamaz.
Öğrencilerin yararlanması amacıyla hazırlanmıştır. Hiçbir ad altında ücret karşılığında satılamaz, kiralanamaz ve ticari amaçla çoğaltılamaz; kaynak gösterilerek ücretsiz olarak paylaşılabilir.

Bu kitap, dersin sunumlarından derlenmiştir. Kitaptaki tüm kod örnekleri ve soru cevapları derlenip çalıştırılarak denetlenmiştir.

© Yunus Emre Demir



İSTİKLÂL MARŞI

Korkma, sönmez bu şafaklarda yüzen al sancak;
Sönmeden yurdumun üstünde tüten en son ocak.
O benim milletimin yıldızıdır, parlayacak;
O benimdir, o benim milletimindir ancak.

Çatma, kurban olayım, çehreni ey nazlı hilâl!
Kahraman ırkıma bir gül! Ne bu şiddet, bu celâl?
Sana olmaz dökülen kanlarımız sonra helâl...
Hakkıdır, Hakk'a tapan, milletimin istiklâl!

Ben ezelden beridir hür yaşadım, hür yaşarım.
Hangi çılgın bana zincir vuracakmış? Şaşarım!
Kükremiş sel gibiyim, bendimi çiğner, aşarım.
Yırtarım dağları, enginlere sığmam, taşarım.

Garbın âfâkını sarmışsa çelik zırhlı duvar,
Benim iman dolu göğsüm gibi serhaddim var.
Ulusun, korkma! Nasıl böyle bir imanı boğar,
"Medeniyet!" dediğin tek dişi kalmış canavar?

Arkadaş! Yurduma alçakları uğratma, sakın.
Siper et gövdeni, dursun bu hayâsızca akın.
Doğacaktır sana va'dettiği günler Hakk'ın...
Kim bilir, belki yarın, belki yarından da yakın.

Bastığın yerleri "toprak!" diyerek geçme, tanı:
Düşün altındaki binlerce kefensiz yatanı.
Sen şehit oğlusun, incitme, yazıktır, atanı:
Verme, dünyaları alsan da, bu cennet vatanı.

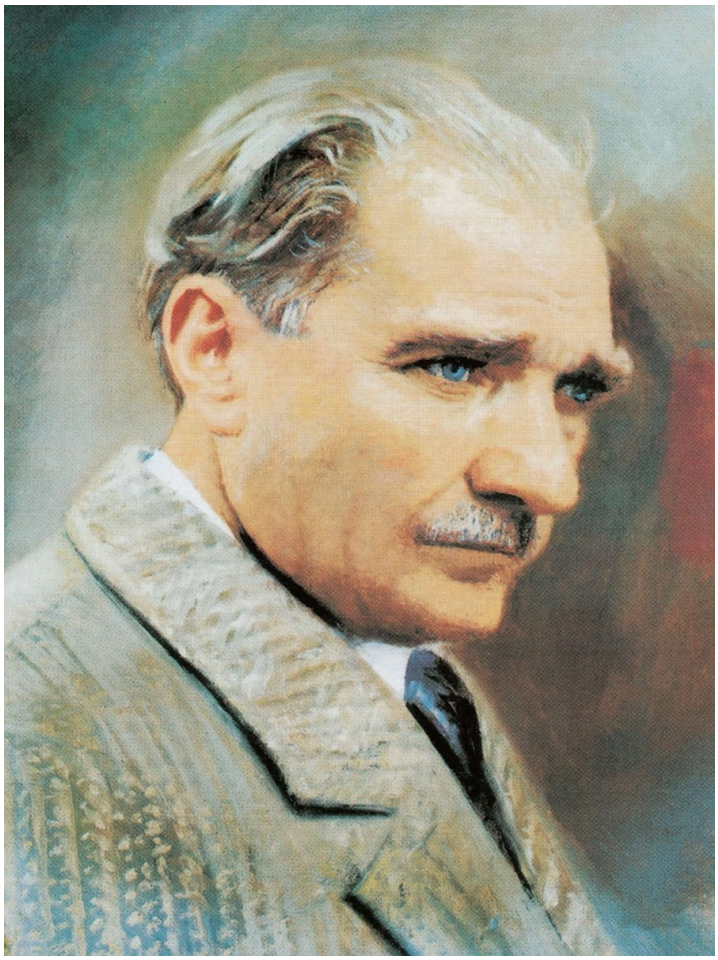
Kim bu cennet vatanın uğruna olmaz ki fedâ?
Şühedâ fışkıracak toprağı sıksan, şühedâ!
Cânı, cânânı, bütün varımı alsın da Hudâ,
Etmesin tek vatanımdan beni dünyada cüdâ.

Ruhumun senden, İlâhî, şudur ancak emeli:
Değmesin mabedimin göğsüne nâmahrem eli.
Bu ezanlar –ki şehadetleri dinin temeli–
Ebedî yurdumun üstünde benim inlemeli.

O zaman vecd ile bin secde eder –varsa– taşım,
Her cerîhamdan, İlâhî, boşanıp kanlı yaşım,
Fışkırır ruh-ı mücerred gibi yerden na'şım;
O zaman yükselerek arşa değer belki başım.

Dalgalan sen de şafaklar gibi ey şanlı hilâl!
Olsun artık dökülen kanlarımın hepsi helâl.
Ebediyen sana yok, ırkıma yok izmihlâl:
Hakkıdır, hür yaşamış, bayrağımın hürriyet;
Hakkıdır, Hakk'a tapan, milletimin istiklâl!

Mehmet Âkif ERSOY



Mustafa Kemal ATATÜRK

1881 – 1938

Önsöz

Bu kitap, Yönetim Bilişim Sistemleri ikinci sınıfta okutulan Nesne Tabanlı Programlama II dersi için hazırlandı. Dersin sunumlarında anlatılan konular, çözülen örnekler ve verilen uygulamalar tek bir kaynakta, düzenli ve kolay okunur biçimde bir araya getirildi. Sunumlarda birden fazla kez geçen konular birleştirildi, eksik kalan açıklamalar tamamlandı.

Programlama okuyarak değil, yazarak öğrenilir. Bu nedenle her konu çalışan bir örnekle verildi. Kitaptaki kod örneklerinin tamamı derlenerek denetlendi; çıktı ve hata davranışları tek tek çalıştırılarak doğrulandı. Form ekranları, Visual Studio'daki görünümü örnek alacak biçimde yeniden çizildi. Böylece kodu ve kodun ekranda ne ürettiğini aynı sayfada görebilirsiniz.

Her bölümün sonunda **Kendini Test Et** soruları ve açıklamalı **Cevap Anahtarı** yer alıyor. Sorular ezber yoklamaz. Çıktı tahmini, hata avı, izleme tablosu, tasarım kararı ve kod yazma gibi bir programcının gerçekte karşılaştığı görevlerden oluşur. Soruları cevap anahtarına bakmadan, mümkünse Visual Studio'da deneyerek çözmenizi öneririm.

Metin içindeki kutuların her birinin bir görevi var. **Dikkat** kutuları sık yapılan hataları, **Not** kutuları tamamlayıcı bilgileri, **Nesne Tabanlı Bakış** kutuları ise form uygulamalarının arkasındaki sınıf ve nesne mantığını gösterir. Dersin adındaki “nesne tabanlı” ifadesi, formdaki her butonun, her kutunun aslında bir nesne olduğu fikrine dayanır.

Bu kitap derse katılmanın yerini tutmaz; dersi destekleyen bir kaynaktır. Kitap, ders ilerledikçe yeni bölümlerle güncellenecektir. Fark ettiğiniz hata ve eksikleri künyedeki iletişim adresine bildirirseniz sonraki sürümler daha iyi olacaktır.

Kitabın tüm öğrencilerimize faydalı olmasını dilerim.

Yunus Emre Demir

İçindekiler

Önsöz v

BÖLÜM 1 Visual Studio ve Windows Forms Ortamı 1

1.1 Konsoldan Forma: Olay Güdümlü Programlama 2
1.2 Yeni Bir Windows Forms Projesi Oluşturma 2
1.3 Visual Studio Pencereleri 3
1.4 Programı Çalıştırma 4
1.5 Form1.cs Kodunun Yapısı 4
1.6 Programın Çalışma Sırası: Main, Kurucu Metot ve Load 5
1.7 Yazım Kuralları ve Açıklama Satırları 6
1.8 Ekrana Mesaj Gönderme: MessageBox 7
1.9 Olaylar ve Olay Metotları 8
1.10 Kendini Test Et 9
1.11 Cevap Anahtarı 11

BÖLÜM 2 Toolbox ve Genel Kontroller 14

2.1 Kontrol, Sınıf ve Nesne 15
2.2 Properties Penceresi ve Ortak Özellikler 16
2.3 Text, BackColor, Visible ve Enabled Uygulaması 16
2.4 Genel Kontroller 18
2.5 Nesne Tabanlı Bakış: Kod ile Kontrol Oluşturma 28
2.6 Kendini Test Et 29
2.7 Cevap Anahtarı 32

BÖLÜM 3 Veri Tipleri, Tür Dönüşümü ve Kontrol Yapıları 35

3.1 Veri Tipleri 36
3.2 Formdan Veri Almak: Tür Dönüşümleri 37
3.3 Operatörler 40
3.4 if – else Yapısı 40
3.5 Döngüler: for ve while 41
3.6 Math Sınıfı 43
3.7 Form_Load ile Başlangıç Değerleri 44
3.8 Çözümlü Uygulamalar 45
3.9 Kendini Test Et 48
3.10 Cevap Anahtarı 51

BÖLÜM 4 ComboBox ve ListBox ile Liste İşlemleri 56

4.1 Items Koleksiyonu ve İndis Mantığı 57
4.2 ComboBox'a Veri Ekleme 57
4.3 ComboBox'tan Veri Silme 58

4.4 ListBox'a Veri Ekleme 60
4.5 ListBox'tan Veri Silme 60
4.6 Sütunlu Kayıt Listesi 61
4.7 SelectedIndexChanged Olayı ve Personel Uygulaması 62
4.8 Kendini Test Et 64
4.9 Cevap Anahtarı 66

BÖLÜM 5 **Çoklu Form Uygulamaları** 70

5.1 Projeye Yeni Form Ekleme 71
5.2 Formun Başlangıç Konumu: StartPosition 71
5.3 Form İçinden Yeni Form Açma 71
5.4 Show ve ShowDialog 72
5.5 Ana Formu Gizleme ve Formu Kapatma 73
5.6 Uygulama: Kullanıcı Giriş Formu 74
5.7 Nesne Tabanlı Bakış: Formlar Arası Veri Aktarma 75
5.8 Kendini Test Et 77
5.9 Cevap Anahtarı 78

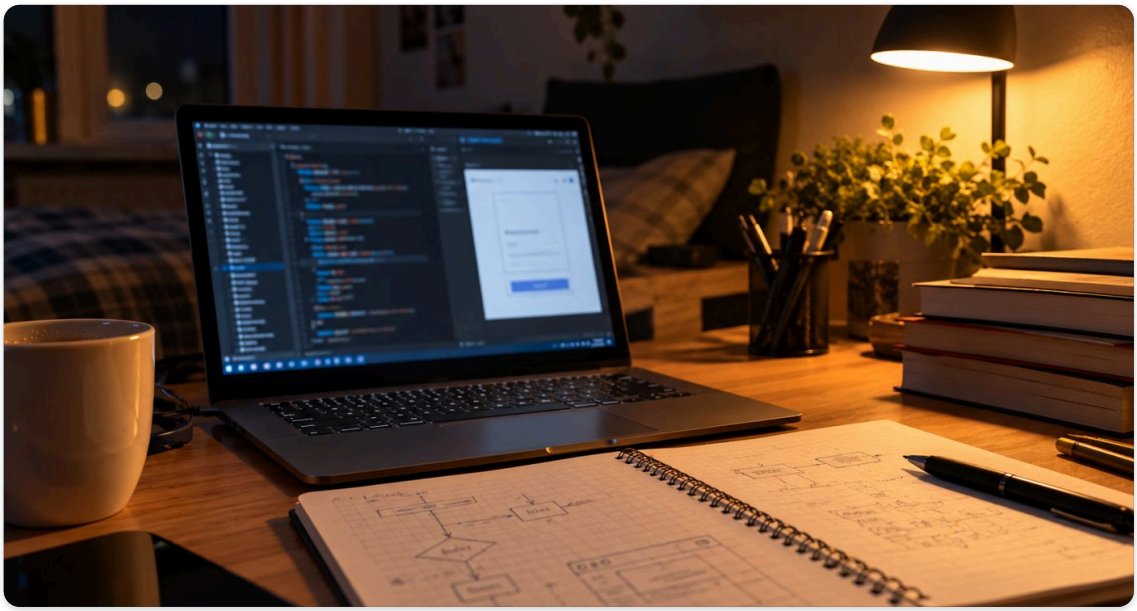
BÖLÜM 6 **Veri Arama, Rastgele Sayılar ve Metin İşlemleri** 82

6.1 Listede Veri Arama: Items.Contains 83
6.2 Items.Count ve Döngüyle Arama 84
6.3 Verinin Yerini Bulma: FindStringExact 85
6.4 Üç Sayının En Büyüğü, En Küçüğü ve Ortalaması 86
6.5 Rastgele Sayı Üretme: Random Sınıfı 88
6.6 Checked Özelliği: CheckBox ve RadioButton 89
6.7 Harf Dönüştürme: ToUpper ve ToLower 91
6.8 Kelimelerin İlk Harfini Büyütme 93
6.9 Kendini Test Et 95
6.10 Cevap Anahtarı 98

BÖLÜM

1

Visual Studio ve Windows Forms Ortamı



BU BÖLÜMÜN SONUNDA

- Konsol uygulaması ile form uygulaması arasındaki farkı açıklayabilecek,
- Visual Studio'da Windows Forms projesi oluşturup çalıştırabilecek,
- Toolbox, Solution Explorer, Properties ve kod/tasarım pencerelerini kullanabilecek,
- Form1.cs dosyasındaki sınıf, kurucu metot ve olay metotlarını tanıyabilecek,
- Programın Main → kurucu → Load → olaylar sırasıyla nasıl çalıştığını izleyebilecek,
- MessageBox ile kullanıcıya mesaj gösterebileceksiniz.

Konsolda yazdığımız programlar kullanıcıyla yalnızca metin üzerinden konuşur. Windows Forms ise pencereler, butonlar ve kutularla çalışan, kullanıcının hamlesini bekleyen bir program türüdür. Bu bölümde bu ortamı tanıyor ve ilk form uygulamamızı yazıyoruz.

1.1 Konsoldan Forma: Olay Güdümlü Programlama

Konsol uygulamalarında program **yukarıdan aşağıya** akar: `Main` metodundaki komutlar sırayla çalışır, gerektiğinde `Console.ReadLine()` ile kullanıcıdan veri beklenir ve son satıra gelindiğinde program biter. Programın akışını tamamen programcı belirler.



Her tıklama bir olaydır; olaya bağlı kod o anda çalışır

Windows Forms uygulamasında ise program açıldıktan sonra bir pencere gösterir ve **bekler**. Bundan sonra neyin, ne zaman çalışacağına kullanıcının hamleleri karar verir: bir butona tıklanması, bir kutuya yazı yazılması, listeden bir eleman seçilmesi. Bu hamlelerin her birine **olay (event)**, bu şekilde kurulan programlara da **olay güdümlü (event-driven)** programlar denir. Programcı, her olay gerçekleştiğinde çalışacak kodu ayrı bir metot içine yazar.

	Konsol uygulaması	Windows Forms uygulaması
Arayüz	Siyah ekranda metin	Pencere, buton, metin kutusu, liste...
Akış	Komutlar sırayla çalışır, sonra program biter	Form açılır, kullanıcının yaptığı olaylar beklenir
Ekrana yazma	`Console.WriteLine(...)`	`MessageBox.Show(...)` veya bir kontrolün `Text` özelliği
Veri alma	`Console.ReadLine()`	TextBox, ComboBox gibi kontrollerin özellikleri
Kodun yeri	Çoğunlukla `Main` içinde	Olay metotlarının içinde (ör. `button1_Click`)

1.2 Yeni Bir Windows Forms Projesi Oluşturma

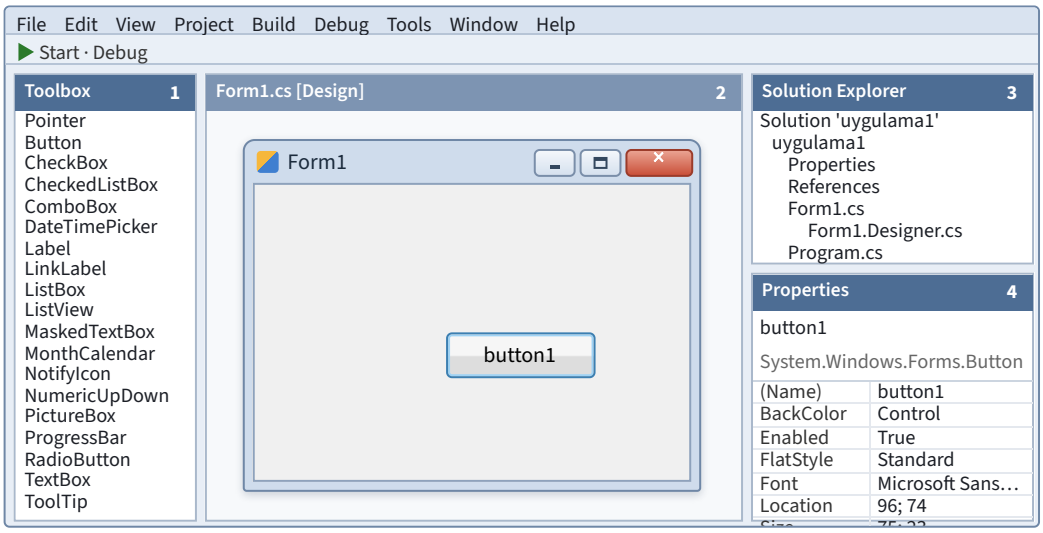
Visual Studio'da yeni bir form uygulaması açmak için şu adımlar izlenir:

1. **File** → **New** → **Project** menüleri sırasıyla tıklanır.
2. Açılan *New Project* penceresinde sol taraftan **Visual C#** → **Windows**, ortadan **Windows Forms Application** seçilir.
3. Alttaki **Name** alanına projenin adı yazılır (ör. `uygulama1`). Ad değiştirilmezse Visual Studio `WindowsFormsApplication1` gibi bir ad verir ve sonraki projelerde sayıyı 2, 3, 4... şeklinde artırır. **Location** alanı projenin kaydedileceği klasörü gösterir.
4. **OK** tuşuna basılır. Ekrana boş bir `Form1` gelir.

> **NOT**
>
> Yeni Visual Studio sürümlerinde (2019, 2022) şablonun adı **Windows Forms App (.NET Framework)** ya da **Windows Forms App** şeklinde görünür. Arama kutusuna *Windows Forms* yazarak şablonu bulabilirsiniz. Dersin örnekleri iki şablonda da aynı şekilde çalışır.

1.3 Visual Studio Pencereleri

Proje açıldığında ekran dört ana bölümden oluşur. Aşağıdaki şekil bu bölümleri, tabloda ise her birinin görevini görüyorsunuz.



Şekil 1.1 *Visual Studio'da bir Windows Forms projesinin çalışma alanı*

No	Pencere	Görevi	View menüsündeki adı
1	**Toolbox**	Forma eklenebilecek kontrollerin (Button, TextBox, Label...) listesi. Kontroller sürüklenip forma bırakılır.	Toolbox
2	**Tasarım penceresi**	Formun görsel olarak düzenlendiği alan. Sekmede `Form1.cs [Design]` yazar.	Designer
3	**Solution Explorer**	Projedeki dosyaları (Form1.cs, Program.cs...) ağaç yapısında gösterir.	Solution Explorer
4	**Properties**	Seçili nesnenin özelliklerini (Text, BackColor, Size...) ve olaylarını listeler.	Properties Window

Bu pencerelerden biri yanlışlıkla kapatılırsa **View** menüsünden yeniden açılır. Tasarım ile kod arasında geçiş için **View → Code** (**F7**) ve **View → Designer** (**Shift+F7**) komutları kullanılır; sekmelerdeki `Form1.cs` ve `Form1.cs [Design]` başlıklarına tıklamak da aynı işi görür. Toolbox başlığındaki raptiye simgesi, pencerenin sabit kalmasını ya da otomatik gizlenmesini sağlar.

> **İPUCU**
>
> Kısayol tuşları Visual Studio sürümüne ve seçili ayar profiline göre değişebilir. Örneğin Properties penceresi çoğu sürümde **F4** ile açılır. Kısayolu hatırlamıyorsanız en güvenli yol **View** menüsüdür.

1.4 Programı Çalıştırma

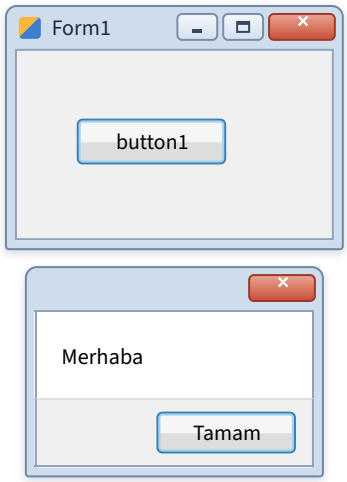
Yazılan program üç yoldan biriyle çalıştırılır: **F5** tuşu, **Debug → Start Debugging** menüsü ya da araç çubuğundaki yeşil ▶ **Start** düğmesi. Üçü de programı hata ayıklama (debug) modunda başlatır. **Ctrl+F5** (*Start Without Debugging*) ise programı hata ayıklayıcı olmadan çalıştırır. Kod derlenemiyorsa program açılmaz ve hatalar **Error List** penceresinde listelenir.

1.5 Form1.cs Kodunun Yapısı

Forma bir buton ekleyip butona çift tıkladığımızda Visual Studio bizi kod penceresine götürür ve tıklama olayı için boş bir metot hazırlar. Metodun içine `MessageBox.Show("Merhaba");` yazdığımızda `Form1.cs` dosyası aşağıdaki gibi görünür.

FORM1.CS

```
1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel;
4  using System.Data;
5  using System.Drawing;
6  using System.Text;
7  using System.Windows.Forms;
8
9  namespace makine1
10 {
11     public partial class Form1 : Form
12     {
13         public Form1()
14         {
15             InitializeComponent();
16         }
17
18         private void button1_Click(object sender,
   EventArgs e)
19         {
20             MessageBox.Show("Merhaba");
21         }
22     }
23 }
```



Bu kısa dosyada nesne tabanlı programlamanın temel kavramlarının neredeyse hepsi vardır:

Satır	Kod	Anlamı
1–7	`using System.Windows.Forms;`	Kullanılacak hazır sınıfların bulunduğu ad alanlarını (namespace) projeye tanıtır. `Form`, `Button`, `MessageBox` gibi sınıflar `System.Windows.Forms` içindedir.
9	`namespace makine1`	Projenin kendi ad alanı. Proje oluşturulurken **Name** alanına yazılan isim buraya gelir.
11	`public partial class Form1 : Form`	**Form1** bir **sınıftır**. İki nokta (`:`) işareti, Form1'in hazır **Form** sınıfından **kalıtım** aldığını gösterir: Form1, bir pencerenin sahip olduğu her şeye (başlık, boyut, renk...) kendiliğinden sahiptir.
11	`partial`	Sınıfın tanımı birden fazla dosyaya bölünmüştür. Tasarımda eklenen kontrollerin kodu Visual Studio tarafından `Form1.Designer.cs` dosyasına yazılır; bizim kodumuz `Form1.cs` dosyasında durur.
13–16	`public Form1() { InitializeComponent(); }`	**Kurucu metot (constructor).** Form1 nesnesi oluşturulurken çalışır. İçindeki `InitializeComponent()`, Designer dosyasında tanımlıdır ve formdaki tüm kontrolleri oluşturup yerleştirir.
18–21	`private void button1_Click(...)`	**Olay metodu.** button1'e tıklandığında çalışır.

NESNE TABANLI BAKIŞ

Dersin adındaki *nesne tabanlı* ifadesi tam olarak burada karşımıza çıkar. `Form1` bir **sınıftır**, yani bir kalıptır. Program çalıştığında bu kalıptan bir **nesne** üretilir ve ekranda gördüğümüz pencere o nesnedir. Forma sürüklediğimiz `button1` de `Button` sınıfından üretilmiş bir nesnedir. Kalıtım sayesinde Form1'e pencere davranışlarını tek satır kod yazmadan kazandırırız. Kalıtımı yazan `: Form` ifadesidir.

1.6 Programın Çalışma Sırası: Main, Kurucu Metot ve Load

Bir form uygulamasının gerçek başlangıç noktası, konsol uygulamalarında olduğu gibi **Main** metodudur. Bu metot Solution Explorer'da görünen `Program.cs` dosyasındadır ve Visual Studio tarafından otomatik yazılır:

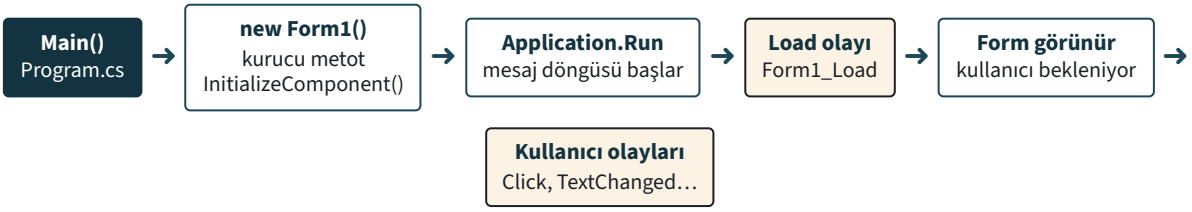
PROGRAM.CS

```
using System;
using System.Windows.Forms;

namespace Uygulama1
{
    static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
```

```
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());   // Form1 nesnesi oluşturulur ve ekrana gelir
        }
    }
}
```

`Application.Run(new Form1())` satırı önce `new` ile bir Form1 nesnesi oluşturur (bu sırada kurucu metot çalışır), sonra bu formu ekrana getirip uygulamanın **mesaj döngüsünü** başlatır. Form ekrana gelmeden hemen önce formun **Load** olayı tetiklenir. Form kapandığında `Application.Run` sona erer ve program biter.



Şekil 1.2 *Bir Windows Forms uygulamasında kodların çalışma sırası*

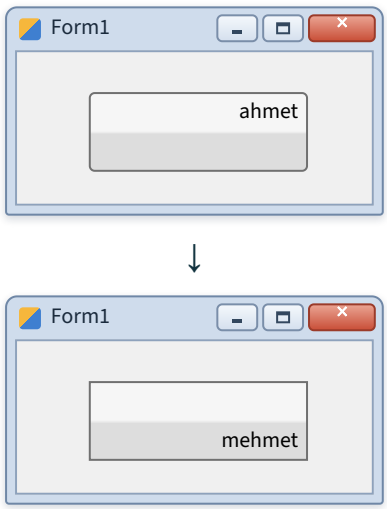
> **DİKKAT**
>
> Bazı kaynaklarda `Form1_Load` için "ana metot gibidir" denir. Bu benzetme yalnızca, Load olayına yazılan kodların kullanıcı hiçbir şey yapmadan, form açılırken **kendiliğinden** çalıştığını anlatmak içindir. Programın başlangıç noktası `Main`'dir ve **kurucu metot Load'dan önce çalışır**.

Load olayına ulaşmak için tasarım penceresinde formun boş bir yerine çift tıklamak yeterlidir. Açılış değerleri (kutulara varsayılan değer yazmak, listeleri doldurmak gibi) genellikle buraya yazılır. Aşağıdaki örnekte form açıldığında buton yazısı **ahmet** olur ve sağ üst köşeye hizalanır. Butona tıklandığında ise yazı **mehmet** olur ve sağ alta kayar.

FORM1.CS — LOAD VE CLİCK OLAYLARI

```
private void Form1_Load(object sender, EventArgs e)
{
    button1.Text = "ahmet";
    button1.TextAlign = ContentAlignment.TopRight;
}

private void button1_Click(object sender, EventArgs e)
{
    button1.Text = "mehmet";
    button1.TextAlign = ContentAlignment.BottomRight;
    button1.FlatStyle = FlatStyle.System;
}
```



1.7 Yazım Kuralları ve Açıklama Satırları

- C# **büyük-küçük harf duyarlıdır**. `MessageBox` ile `messagebox`, `Text` ile `text` farklı adlardır. Yanlış yazılan ad derleme hatasına yol açar.
- Her komut **noktalı virgül** (`;`) ile biter. Kod blokları **süslü parantez** (`{ }`) ile açılıp kapatılır.

- Metinler **çift tırnak** içinde yazılır: `"Merhaba"`. Tırnak konmazsa derleyici bunu bir değişken adı sanar.
- **Açıklama satırları** derleyici tarafından dikkate alınmaz. Tek satır için `//`, birden çok satır için `/* … */` kullanılır. Visual Studio açıklamaları yeşil renkte gösterir.

AÇIKLAMA SATIRLARI

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("DENEME");

    // Tek satırlık açıklama: bugün hava çok güzel

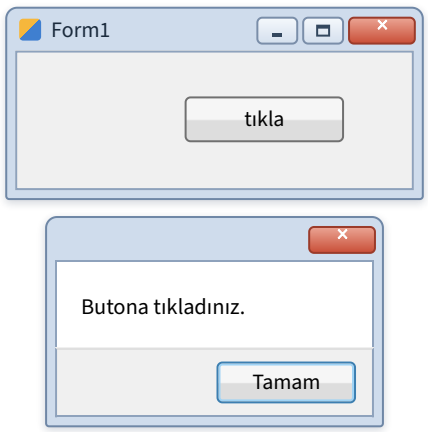
    /* Çok satırlı açıklama:
       Bu satırlar derleyici tarafından
       dikkate alınmaz. */
}
```

Açıklamalar, kodu daha sonra okuyacak kişiye (çoğu zaman bir süre sonraki kendimize) ne yapıldığını anlatmak için yazılır. Ayrıca bir kod parçasını silmeden geçici olarak devre dışı bırakmak için de kullanılır.

1.8 Ekrana Mesaj Gönderme: MessageBox

Konsolda kullandığımız `Console.Write` ve `Console.WriteLine` komutları form ortamında kullanılmaz. Kullanıcıya bir mesaj göstermek için `MessageBox.Show("mesaj")` komutu yazılır. Komut, içinde mesaj ve **Tamam** düğmesi bulunan küçük bir pencere açar. Bu pencere kapatılmadan forma dönülemez.

```
private void button1_Click(object sender, EventArgs
e)
{
    MessageBox.Show("Butona tıkladınız.");
}
```



İPUCU

`MessageBox.Show` yalnızca metin (string) kabul eder. Bir sayıyı göstermek için onu metne çevirmek gerekir: `MessageBox.Show(sonuc.ToString());` ya da metinle birleştirerek `MessageBox.Show("Sonuç: " + sonuc);`. Bu konu 3. bölümde ayrıntılı işlenecektir.

1.9 Olaylar ve Olay Metotları

Tasarım penceresinde bir kontrole çift tıklandığında Visual Studio o kontrolün **varsayılan olayı** için boş bir metot oluşturur. En sık kullanılan tetikleyici kontrol butondur ve varsayılan olayı **Click**'tir:

```
private void btnTikla_Click(object sender, EventArgs e)
{
    // Butona tıklandığında buradaki kodlar çalışır.
}
```

Metodun adı *kontrolAdı_OlayAdı* biçimindedir. Parantez içindeki iki parametre her olay metodunda bulunur: `sender` olayı tetikleyen nesneyi, `e` ise olayla ilgili ek bilgileri taşır. Kontrollerin diğer olaylarına **Properties** penceresindeki **şimşek** simgesine (Events) tıklanarak ulaşılır.

Kontrol	Varsayılan olay	Ne zaman tetiklenir?
Form	`Load`	Form ilk kez ekrana gelmeden hemen önce
Button	`Click`	Butona tıklandığında
TextBox	`TextChanged`	Kutudaki metin her değiştiğinde
CheckBox, RadioButton	`CheckedChanged`	İşaret durumu değiştiğinde
ComboBox, ListBox	`SelectedIndexChanged`	Seçili eleman değiştiğinde
DateTimePicker	`ValueChanged`	Seçili tarih değiştiğinde
LinkLabel	`LinkClicked`	Bağlantıya tıklandığında

DİKKAT

Olay metodunun çalışması için metodun olaya **bağlanmış** olması gerekir. Çift tıklama ile oluşturulan metotlarda Visual Studio bu bağlantıyı `Form1.Designer.cs` dosyasına kendisi yazar:

```
// Form1.Designer.cs dosyasında Visual Studio'nun otomatik yazdığı satırlar:
this.Load += new System.EventHandler(this.Form1_Load);
this.button1.Click += new System.EventHandler(this.button1_Click);
```

Bir metodu başka bir projeden kopyalayıp yapıştırmak bu bağlantıyı oluşturmaz. Metot yazılmış görünür ama hiçbir zaman çalışmaz. Böyle bir durumda metot, Properties → Events listesinden ilgili olaya seçilerek bağlanır.

NESNE TABANLI BAKIŞ

Bir olay metodu, olayı tetikleyen nesneye `sender` parametresi üzerinden ulaşabilir. `sender` genel `object` türündedir. Nesnenin bir buton olduğunu biliyorsak tür dönüşümü yaparak onun özelliklerini okuyabiliriz:

```
private void button1_Click(object sender, EventArgs e)
{
    Button tiklanan = (Button)sender;   // olayı tetikleyen nesne
    MessageBox.Show(tiklanan.Text + " butonuna tıklandı.");
}
```

Aynı metot birden çok butonun Click olayına bağlanırsa, her tıklamada hangi butona basıldığı bu yöntemle anlaşılır. Bu kullanımı 2. bölümde kod ile buton oluştururken tekrar göreceğiz.

ÖZET

- Windows Forms uygulamaları **olay güdümlüdür**. Kod, kullanıcının yaptığı olaylara bağlı metotların içine yazılır.
- Proje **File → New → Project → Visual C# → Windows → Windows Forms Application** yoluyla oluşturulur. Program **F5** ile çalıştırılır.
- Toolbox, Solution Explorer ve Properties pencereleri kapanırsa **View** menüsünden yeniden açılır. Kod/tasarım geçişi **F7** / **Shift+F7** ile yapılır.
- `Form1` bir sınıftır ve `: Form` ifadesiyle Form sınıfından kalıtım alır. `partial` ifadesi, sınıfın Form1.cs ile Form1.Designer.cs arasında bölündüğünü gösterir.
- Çalışma sırası: **Main → new Form1() (kurucu, InitializeComponent) → Load → form görünür → kullanıcı olayları**.
- C# büyük-küçük harf duyarlıdır. Komutlar `;` ile biter. Açıklamalar `//` ve `/* */` ile yazılır.
- Form ortamında ekrana mesaj `MessageBox.Show("...")` ile verilir.

1.10 Kendini Test Et

Soruları önce kâğıt üzerinde çözün, sonra Visual Studio'da deneyerek kontrol edin. Açıklamalı cevaplar Cevap Anahtarı başlığındadır. Zorluk düzeyi soru başlığındaki noktalarla gösterilmiştir.

Soru 1.1 HATA AVI Derlenmeyen kod ◆◇◇

Bir öğrenci, butona tıklandığında iki mesaj gösterip buton yazısını değiştiren aşağıdaki kodu yazmış ancak program derlenmemiştir.

```
1 private void button1_Click(object sender, EventArgs e)
2 {
3     messagebox.Show("Kayıt tamamlandı");
4     button1.text = "Tamam"
5     MessageBox.Show(Merhaba);
6     /* Bu satır bir açıklamadır
7 }
```

a. Kodda **beş** ayrı hata vardır. Her hatanın satır numarasını yazınız ve derleyicinin bu satırı neden kabul etmediğini bir cümleyle açıklayınız.

b. Hatalardan biri, kendisinden sonraki satırların tamamının derleyici tarafından yanlış okunmasına neden olur. Hangisi olduğunu ve nedenini açıklayınız.

c. Kodun düzeltilmiş hâlini yazınız.

Soru 1.2 İZLEME TABLOSU Kurucu metot, Load ve Click ◆◆◇

Aşağıdaki Form1 sınıfında form üzerinde yalnızca `button1` bulunmaktadır. Tüm olay metotları olaylarına doğru şekilde bağlanmıştır.

```
1  using System;
2  using System.Windows.Forms;
3
4  namespace Soru12
5  {
6      public partial class Form1 : Form
7      {
8          public Form1()
9          {
10             InitializeComponent();
11             button1.Text = "A";
12             this.Text = "Kurucu";
13         }
14
15         private void Form1_Load(object sender, EventArgs e)
16         {
17             button1.Text = button1.Text + "B";
18             this.Text = this.Text + " + Load";
19         }
20
21         private void button1_Click(object sender, EventArgs e)
22         {
23             button1.Text = button1.Text + "C";
24         }
25     }
26 }
```

a. Form ekrana geldiği anda butonun üzerinde ve formun başlık çubuğunda ne yazar?

b. Kullanıcı butona art arda **3 kez** tıkladıktan sonra butonun üzerinde ne yazar?

c. Kurucu metotta 11. satır (`button1.Text = "A";`) 10. satırın **üstüne** taşınırsa program açılırken ne olur? Nedenini açıklayınız.

d. Öğrenci `Form1_Load` metodunu tasarım penceresinde çift tıklayarak oluşturmamış, başka bir projeden kopyalayıp yapıştırmıştır. Bu durumda (a) şıkkının cevabı ne olur? Neden?

Soru 1.3 TASARIM KARARI Hangi kontrol, hangi olay? ◆◆◇

Aşağıdaki her istek için kodun hangi kontrolün hangi olay metoduna yazılacağını belirleyiniz ve o metodun

gövdesine yazılacak kodu (1–3 satır) yazınız.

No	İstek
1	Program açılır açılmaz formun başlığı *Öğrenci Kayıt* olsun.
2	Kullanıcı textBox1'e her harf yazdığında, label1'de kutudaki karakter sayısı görünsün.
3	checkBox1 (*Koşulları okudum*) işaretlenince Kaydet butonu (button1) tıklanabilir, işaret kaldırılınca tıklanamaz olsun.
4	comboBox1'den bir şehir seçildiğinde seçilen şehir mesaj olarak gösterilsin.
5	linkLabel1'e tıklanınca üniversitenin web sitesi açılsın.
6	dateTimePicker1'de tarih değiştirildiğinde bu yıl ile seçilen yıl arasındaki fark label2'ye yazılsın.

Soru 1.4 KOD OKUMA Form1.cs'yi okumak ◆◆◇

Bölüm 1.5'teki `Form1.cs` dosyasını (makine1 projesi) dikkate alarak aşağıdaki soruları cevaplayınız.

a. Projenin adı *makine1* yerine *hesap* olsaydı kodda hangi satır değişirdi?

b. Form1 sınıfı hangi sınıftan türetilmiştir? Bu türetme olmasaydı Form1'in neden `Text`, `BackColor`, `Show()` gibi üyeleri olmazdı?

c. `InitializeComponent` metodu Form1.cs dosyasında tanımlanmadığı hâlde nasıl çağrılabilmektedir? `partial` anahtar kelimesiyle ilişkisini açıklayınız.

d. Aynı Click metodu hem button1'in hem de button2'nin Click olayına bağlanmıştır. Metot içinde hangi butona tıklandığını nasıl anlarsınız? İlgili kodu yazınız.

1.11 Cevap Anahtarı

Soru 1.1

a)

Satır	Hata	Neden?
3	`messagebox`	C# büyük-küçük harf duyarlıdır. Sınıfın adı `MessageBox`'tır; `messagebox` adında bir şey tanımlı değildir.
4	`button1.text`	Button sınıfında `text` adlı bir üye yoktur. Özelliğin adı `Text`'tir.
4	Noktalı virgül eksik	Komut `;` ile bitmediği için derleyici bir sonraki satırı da aynı komutun devamı sanar ve beklenmeyen ifade hatası verir.
5	`MessageBox.Show(Merhaba)`	Tırnak işareti olmadığı için `Merhaba` bir değişken adı sayılır. Böyle bir değişken tanımlı olmadığından hata oluşur.
6	`/* ...` kapatılmamış	Çok satırlı açıklama `*/` ile kapatılmamıştır.

b) 6. satırdaki kapatılmamış açıklama. `/*` görüldükten sonra derleyici `*/` görene kadar **her şeyi** açıklama sayar. Bu yüzden metodu kapatan `}` ve sonrasındaki tüm sınıf kodu açıklamanın içinde kalır. Derleyici dosya sonuna kadar `*/` bulamadığı için "dosya sonuna ulaşıldı, `*/` bekleniyordu" hatası verir.

c)

```
1 private void button1_Click(object sender, EventArgs e)
2 {
3     MessageBox.Show("Kayıt tamamlandı");
4     button1.Text = "Tamam";
5     MessageBox.Show("Merhaba");
6     /* Bu satır bir açıklamadır */
7 }
```

Denetim: Beş hatanın her biri doğru koda tek başına eklendiğinde de derleme başarısız olmaktadır.

Soru 1.2

a) Buton: **AB**, başlık: **Kurucu + Load**. Önce kurucu metot çalışır ve butona `A`, başlığa `Kurucu` yazar. Form ekrana gelmeden önce Load olayı tetiklenir ve mevcut metinlerin sonuna `B` ve `+ Load` ekler.

b) ABCCC. Her tıklamada mevcut metnin sonuna bir `C` eklenir.

c) Program açılırken **NullReferenceException** hatası oluşur. `button1` nesnesi `InitializeComponent()` içinde `new Button()` ile oluşturulur. Bu satırdan önce `button1` değişkeni henüz hiçbir nesneyi göstermez (`null`), dolayısıyla `Text` özelliğine erişilemez. Kural: kontrollere kurucu metotta ancak `InitializeComponent()` satırından **sonra** erişilebilir.

d) Buton: **A**, başlık: **Kurucu**. Kopyalanan metot olaya bağlı olmadığı için (Designer dosyasında `this.Load += …` satırı yoktur) hiç çalışmaz. Çözüm: Properties → Events (şimşek) listesinde **Load** olayına bu metodu seçmek ya da formun boş bir yerine çift tıklamaktır.

Denetim: (a) ve (b) şıklarındaki sıra, olaylar Designer'daki gibi bağlanmış bir formda çalıştırılarak doğrulanmıştır.

Soru 1.3

No	Kontrol ve olay
1	Form → `Load`
2	textBox1 → `TextChanged`
3	checkBox1 → `CheckedChanged`
4	comboBox1 → `SelectedIndexChanged`
5	linkLabel1 → `LinkClicked`
6	dateTimePicker1 → `ValueChanged`

```
// 1) Form açılır açılmaz başlık
private void Form1_Load(object sender, EventArgs e)
{
    this.Text = "Öğrenci Kayıt";
}

// 2) Her tuş vuruşunda karakter sayısı
private void textBox1_TextChanged(object sender, EventArgs e)
{
    label1.Text = textBox1.Text.Length + " karakter";
}
```

```
// 3) Onay kutusu ile butonu aktif/pasif yapma
private void checkBox1_CheckedChanged(object sender, EventArgs e)
{
    button1.Enabled = checkBox1.Checked;
}

// 4) Açılır listeden seçim yapılınca
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    if (comboBox1.SelectedIndex != -1)
    {
        MessageBox.Show(comboBox1.SelectedItem.ToString());
    }
}

// 5) Bağlantıya tıklanınca
private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    System.Diagnostics.Process.Start("https://www.firat.edu.tr");
}

// 6) Tarih değişince yıl farkını yazma
private void dateTimePicker1_ValueChanged(object sender, EventArgs e)
{
    int fark = DateTime.Today.Year - dateTimePicker1.Value.Year;
    label2.Text = "Yıl farkı: " + fark;
}
```

3. istek: `if` yazmaya gerek yoktur. `Checked` zaten `true` / `false` değeri taşır ve doğrudan `Enabled`'a atanabilir. Tasarımda button1'in `Enabled` özelliği başlangıçta `False` yapılmalıdır.

4. istek: Listedeki seçim kaldırılırsa (ör. liste temizlenirse) olay yine tetiklenir ve `SelectedIndex` değeri `-1` olur. Bu durumda `SelectedItem` boştur (`null`) ve `.ToString()` hata verir. Kontrol bu yüzden yazılmıştır.

Soru 1.4

a) 9. satır: `namespace hesap`. (Program.cs'deki ad alanı da aynı şekilde değişir.)

b) `Form` sınıfından. `Text`, `BackColor`, `Show()` gibi üyeler Form sınıfında (ve onun da türediği Control sınıfında) tanımlıdır. Form1 bunları **kalıtım** yoluyla devralır. `: Form` yazılmasaydı Form1 boş bir sınıf olurdu ve bu üyelerin hiçbiri bulunmazdı.

c) Form1 sınıfı `partial` olarak tanımlandığı için sınıfın bir parçası `Form1.cs`'de, diğer parçası `Form1.Designer.cs`'de bulunur. Derleyici iki parçayı tek bir sınıf olarak birleştirir. `InitializeComponent` Designer dosyasındaki parçada tanımlı olduğu için Form1.cs'den çağrılabilir.

d) `sender` parametresi olayı tetikleyen nesnedir:

```
private void button1_Click(object sender, EventArgs e)
{
    Button tiklanan = (Button)sender;   // olayı tetikleyen nesne
    MessageBox.Show(tiklanan.Text + " butonuna tıklandı.");
}
```

İstenirse `if (sender == button1)` karşılaştırması da yapılabilir.

BÖLÜM

2

Toolbox ve Genel Kontroller



BU BÖLÜMÜN SONUNDA

- Kontrol, sınıf ve nesne kavramları arasındaki ilişkiyi açıklayabilecek,
- Properties penceresinden ve kod ile kontrol özelliklerini değiştirebilecek,
- Visible ile Enabled arasındaki farkı ayırt edebilecek,
- Toolbox'taki genel kontrollerin amacını, temel özelliklerini ve olaylarını bilecek,
- Bir ihtiyaç için en uygun kontrolü seçip gerekçelendirebilecek,
- Kod ile çalışma anında kontrol oluşturabileceksiniz.

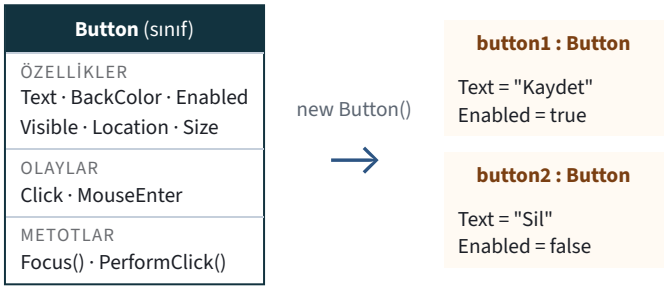
Bir form uygulamasının yüzü kontrollerdir. Butonlar, metin kutuları, listeler, takvimler... Bu bölümde Toolbox'taki genel kontrolleri tanıyor, her birinin hangi ihtiyaca cevap verdiğini ve en çok kullanılan özelliklerini öğreniyoruz.

2.1 Kontrol, Sınıf ve Nesne

Toolbox'ın **Common Controls** bölümünde forma eklenebilecek genel kontroller listelenir. C#'taki her kontrol, `System.Windows.Forms` içindeki **Control** sınıfından türetilmiş bir sınıftır. Toolbox'tan bir Button sürükleyip forma bıraktığımızda Visual Studio, `Button` sınıfından `button1` adında bir **nesne** oluşturur. İkinci butonu bıraktığımızda aynı sınıftan ikinci bir nesne (`button2`) oluşur. Sınıf tektir, nesneler istendiği kadar çoğaltılabilir.



Tek kalıp (sınıf), birbirinden farklı süslenmiş ürünler (nesneler)



Şekil 2.1 *Button sınıfı ve bu sınıftan üretilmiş iki nesne. Her nesnenin özellik değerleri kendine aittir.*

Kavram	Anlamı	Örnek
Özellik (property)	Nesnenin bir niteliği; okunabilir ve çoğu zaman değiştirilebilir.	`button1.Text = "Kaydet";`
Olay (event)	Nesnenin başına gelen ve koda haber verilen durum.	`button1_Click`
Metot (method)	Nesneye yaptırılan iş; adının sonunda parantez bulunur.	`textBox1.Clear();`

NOT

Kontrolün kodda kullanılan adı **(Name)** özelliğidir (`button1` , `textBox1` ...). Ekranda görünen yazı ise **Text** özelliğidir. Büyük projelerde adlar anlamlı verilir: `btnKaydet` , `txtAd` , `lblSonuc` gibi. Bu kitapta, ders sunumlarıyla uyumlu olması için Visual Studio'nun varsayılan adları kullanılmıştır.

2.2 Properties Penceresi ve Ortak Özellikler

Bir kontrolün özellikleri iki yoldan değiştirilebilir. **Tasarım anında** kontrol seçilir ve sağ alttaki **Properties** penceresinden istenen özellik ayarlanır. **Çalışma anında** ise aynı özellik kod ile değiştirilir. Properties penceresinde yapılan her ayar, Visual Studio tarafından `Form1.Designer.cs` dosyasına kod olarak yazılır. Yani iki yol aslında aynı işi yapar.

Özellik	Görevi	Kod ile örnek
`Text`	Kontrolün üzerinde/içinde görünen yazı	`button1.Text = "Tamam";`
`BackColor`	Arka plan rengi	`this.BackColor = Color.Red;`
`ForeColor`	Yazı (karakter) rengi	`label1.ForeColor = Color.Blue;`
`Visible`	Kontrolün görünür (`true`) ya da görünmez (`false`) olması	`button3.Visible = false;`
`Enabled`	Kontrolün aktif (`true`) ya da pasif (`false`) olması	`button4.Enabled = false;`
`Font`	Yazı tipi ve boyutu	`new Font("Arial", 12)`
`Location`	Kontrolün form üzerindeki konumu (x; y)	`new Point(20, 40)`
`Size`	Genişlik ve yükseklik	`new Size(120, 30)`
`RightToLeft`	Yazının sağdan sola yazılması	`RightToLeft.Yes`
`TabIndex`	Tab tuşuyla kontroller arasında geçiş sırası	`textBox2.TabIndex = 1;`
`ContextMenuStrip`	Sağ tıklayınca açılacak menüyü kontrole bağlar	Tasarımda seçilir

DİKKAT

Kontrolü aktif/pasif yapan özelliğin adı `Enable` değil **`Enabled`** 'dır. Properties penceresinde de *Enabled* olarak görünür. `button1.Enable = false;` yazılırsa "Button sınıfında Enable adında bir tanım yok" hatası alınır.

2.3 Text, BackColor, Visible ve Enabled Uygulaması

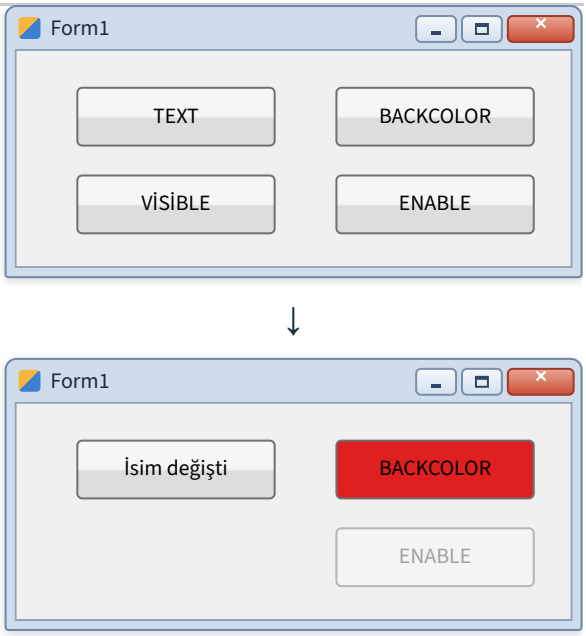
1. adım: Forma 4 buton eklenir ve Properties penceresinin **Text** kısmından adları sırasıyla TEXT, BACKCOLOR, VISIBLE ve ENABLE yapılır. **2. adım:** Her butona çift tıklanarak aşağıdaki kodlar yazılır ve program F5 ile çalıştırılır.

```
private void button1_Click(object sender,
EventArgs e)
{
    button1.Text = "İsim değişti";
}

private void button2_Click(object sender,
EventArgs e)
{
    button2.BackColor = Color.Red;
}

private void button3_Click(object sender,
EventArgs e)
{
    button3.Visible = false;   // buton tamamen
görünmez olur
}

private void button4_Click(object sender,
EventArgs e)
{
    button4.Enabled = false;   // buton görünür
kalır ama tıklanamaz
}
```



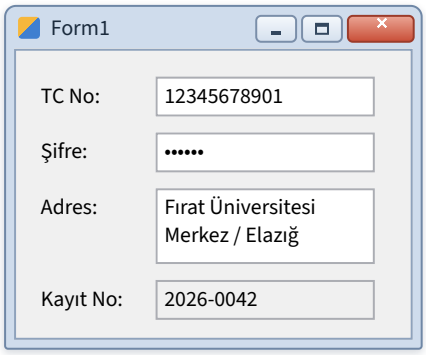
Dört butona da tıklandıktan sonra (alttaki görüntü): birinci butonun yazısı değişmiş, ikincisi kırmızı olmuş, üçüncüsü tamamen **kaybolmuş**, dördüncüsü ise yerinde duruyor ama **soluk** görünüyor ve artık tıklanamıyor.

	Visible = false	Enabled = false
Ekranda	Görünmez, yeri boş kalır	Görünür, soluk (gri) çizilir
Tıklanabilir mi?	Hayır (görülmüyor ki)	Hayır, olayları tetiklenmez
Kullanım örneği	Yetkisiz kullanıcıdan bir butonu tamamen gizlemek	Form doldurulmadan Kaydet'e basılmasını engellemek

3. adım: Aynı işlemler kod yazmadan, her butonun Properties penceresindeki **Text**, **BackColor**, **Visible** ve **Enabled** satırlarından da yapılabilir. Fark şudur: Properties penceresinden yapılan ayar formun **açılış** durumunu belirler, kod ise durumu program **çalışırken** değiştirir.

TextBox'a özgü bazı özellikler

Özellik	Görevi
`MaxLength`	Yazılabilecek en fazla karakter sayısı (varsayılan 32767)
`Multiline`	`true` ise kutu birden çok satır alır
`PasswordChar`	Yazılan karakterleri gizler, ör. `'*'`
`ReadOnly`	`true` ise kutu yalnızca okunur, içine yazılamaz



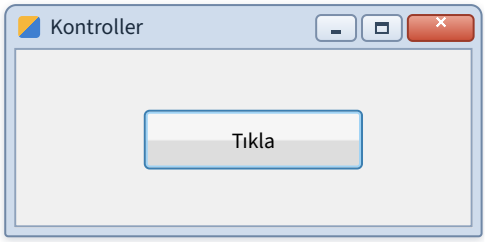

```
private void Form1_Load(object sender, EventArgs e)
{
    textBox1.MaxLength = 11;          // en fazla 11 karakter (ör. TC kimlik no)
    textBox2.PasswordChar = '*';      // yazılanlar * olarak görünür
    textBox3.Multiline = true;        // birden çok satır yazılabilir
    textBox4.ReadOnly = true;         // yalnızca okunur, kullanıcı değiştiremez
}
```

2.4 Genel Kontroller

Bu bölümde Toolbox'taki genel kontroller tek tek ele alınmıştır. Her kontrol için kısa bir tanım, en çok kullanılan özellikler ve form üzerindeki görünüm verilmiştir. Kod örneği gerektirenlerde örnek kod da eklenmiştir. Kontroller Toolbox'taki alfabetik sırayla verilmiştir.

1. Button

Genellikle en çok kullanılan tetikleyici kontroldür. Üzerine çift tıklandığında varsayılan olarak açılan olay **Click**'tir.

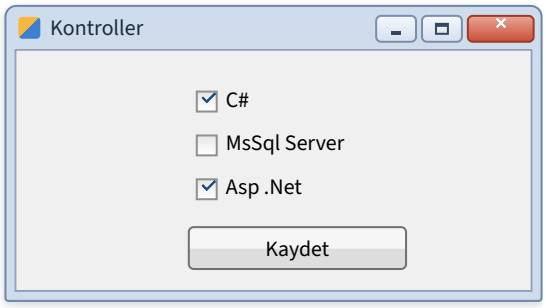


Özellik / Üye	Açıklama
`Text`	Buton yazısı
`TextAlign`	Yazının buton içindeki hizası (ör. `ContentAlignment.TopRight`)
`FlatStyle`	Görünüm biçimi: Flat, Popup, Standard, System

> **NOT**
>
> Sunumlarda özelliğin adı *FlatSytle* diye geçmektedir. Doğru yazım **FlatStyle**'dır.

2. CheckBox

Kullanıcıya metin içeren bir seçenek sunar (evet/hayır, doğru/yanlış gibi). Birden fazla CheckBox aynı anda işaretlenebilir. İşaret durumu değişince **CheckedChanged** olayı tetiklenir.

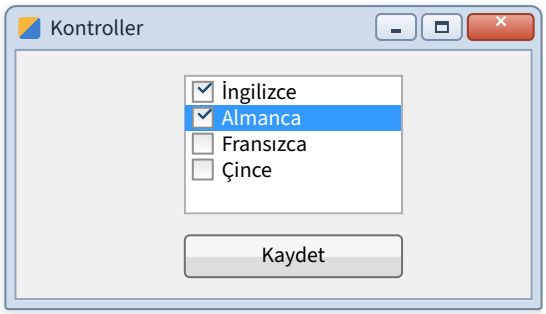


Özellik / Üye	Açıklama
`Text`	Kutunun yanındaki yazı
`Checked`	İşaretli ise `true`
`CheckAlign`	Kutucuğun yazıya göre konumu

```
private void button1_Click(object sender, EventArgs e)
{
    string secilenler = "";
    if (checkBox1.Checked) secilenler += checkBox1.Text + "  ";
    if (checkBox2.Checked) secilenler += checkBox2.Text + "  ";
    if (checkBox3.Checked) secilenler += checkBox3.Text + "  ";
    MessageBox.Show("Seçilenler: " + secilenler);
}
```

3. CheckedListBox

CheckBox'a benzer, ancak tek bir kontrolde birden fazla işaret kutusu oluşturmayı sağlar. Elemanlarına bir dizinin elemanlarıymış gibi indisle erişilir.



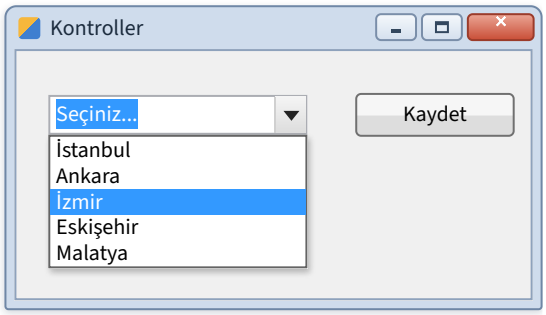
Özellik / Üye	Açıklama
`Items`	Eleman listesi
`SelectedItem`	/ `SelectedIndex` seçili (mavi) eleman ve indisi
`CheckedItems`	İşaretli elemanların koleksiyonu
`GetItemChecked(i)`	**Metottur.** i. eleman işaretliyse `true` döndürür
`SetItemChecked(i, d)`	i. elemanın işaretini değiştirir

```
private void button1_Click(object sender, EventArgs e)
{
    string diller = "";
    foreach (object dil in checkedListBox1.CheckedItems)
    {
        diller += dil + "\n";
    }
    MessageBox.Show("İşaretlenen diller:\n" + diller);

    // Tek bir elemanın durumunu sorgulama ve değiştirme (indis 0'dan başlar):
    bool almancaSecili = checkedListBox1.GetItemChecked(1);
    checkedListBox1.SetItemChecked(0, true);
}
```

4. ComboBox

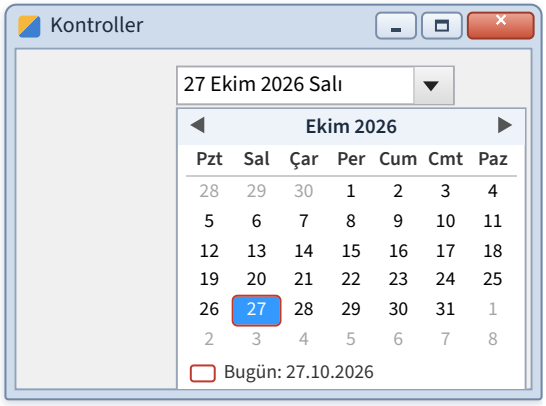
Aşağı doğru açılan bir seçim listesi sunar. Listedeki elemanlar bir dizinin elemanları gibi indislidir. ComboBox ile yapılan ekleme ve silme işlemleri 4. bölümde ayrıntılı işlenecektir.



Özellik / Üye	Açıklama
`Text`	Kutuda görünen metin (ör. *Seçiniz...*)
`Items`	Elemanlar: `Add`, `AddRange`, `Clear`, `Remove`
`SelectedItem`	Seçili eleman
`SelectedIndex`	Seçili elemanın indisi; seçim yoksa `-1`
`DropDownStyle`	`DropDownList` yapılırsa kutuya elle yazılamaz
`MaxDropDownItems`	Açılan listede aynı anda görünen eleman sayısı

5. DateTimePicker

Açılan kutu içinde bir takvim görünümü sunar. Bugünün tarihini gösterir ve istenen herhangi bir tarihin seçilmesini sağlar. Tarih değişince **ValueChanged** olayı tetiklenir.



Özellik / Üye	Açıklama
`Value`	Seçili tarih (`DateTime` türünde)
`MinDate`	/ `MaxDate` seçilebilecek en erken / en geç tarih
`Format`	Long, Short, Time ya da Custom
`CustomFormat`	Format Custom iken kalıp, ör. `"dd.MM.yyyy"`

DİKKAT

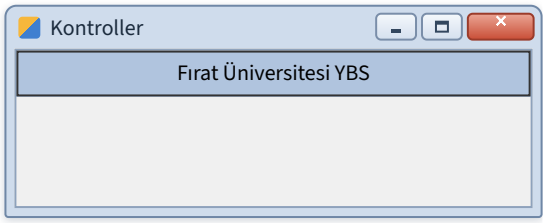
Bazı kaynaklarda DateTimePicker'ın özellikleri arasında *MaxDropDownItems* sayılır. Bu özellik DateTimePicker'da **yoktur**, ComboBox'a aittir. Kod içinde kullanılırsa derleme hatası verir.

```
private void Form1_Load(object sender, EventArgs e)
{
    dateTimePicker1.MaxDate = DateTime.Today;          // ileri bir tarih seçilemez
    dateTimePicker1.Format = DateTimePickerFormat.Custom;
    dateTimePicker1.CustomFormat = "dd.MM.yyyy";
}

private void button1_Click(object sender, EventArgs e)
{
    DateTime secilen = dateTimePicker1.Value;
    MessageBox.Show("Seçilen tarih: " + secilen.ToShortDateString());
}
```

6. Label

Form üzerinde kullanıcıya bilgi vermek, kutuların ne işe yaradığını yazmak ya da bir sonucu göstermek için kullanılır. Kullanıcı Label'ın içine yazı yazamaz.

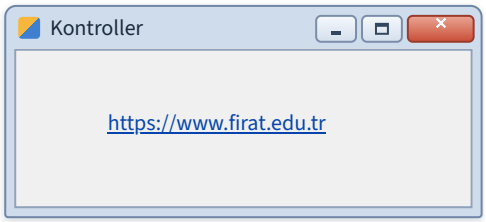


Özellik / Üye	Açıklama
`Text`	Görünen yazı
`BorderStyle`	Kenarlık: None, FixedSingle, Fixed3D
`BackColor`	Arka plan rengi
`AutoSize`	`true` ise boyut yazıya göre ayarlanır

```
private void Form1_Load(object sender, EventArgs e)
{
    label1.Text = "Fırat Üniversitesi YBS";
    label1.AutoSize = false;
    label1.BorderStyle = BorderStyle.FixedSingle;
    label1.BackColor = Color.LightSteelBlue;
}
```

7. LinkLabel

Uygulamadan bir dosyaya, klasöre ya da internet sayfasına yönlendirme yapmak için kullanılır. Tek bir LinkLabel üzerine birden fazla bağlantı verilebilir. Bağlantıya tıklanınca **LinkClicked** olayı tetiklenir.



Özellik / Üye	Açıklama
`Text`	Görünen yazı
`Links`	Bağlantılar koleksiyonu: `Links.Add(başlangıç, uzunluk, adres)`
`LinkColor`	Bağlantı rengi
`VisitedLinkColor`	Ziyaret edilmiş bağlantı rengi

```
private void Form1_Load(object sender, EventArgs e)
{
    string link = "https://www.firat.edu.tr";
    linkLabel1.Text = link;
    linkLabel1.Links.Add(0, link.Length, link);
}

private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    System.Diagnostics.Process.Start(e.Link.LinkData.ToString());
    linkLabel1.LinkVisited = true;      // bağlantı "ziyaret edildi" rengine geçer
}
```

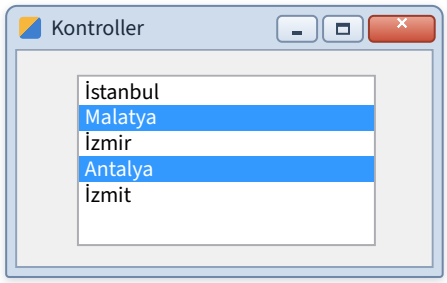
NOT

`Process.Start(adres)` yazımı .NET Framework projelerinde adresi varsayılan tarayıcıda açar. Yeni **.NET** (Core, 5 ve sonrası) şablonlarıyla açılan projelerde ise aynı iş şu satırla yapılır:

```
// .NET Core / .NET 5 ve sonrası projelerde adres şu şekilde açılır:
System.Diagnostics.Process.Start(new
System.Diagnostics.ProcessStartInfo("https://www.firat.edu.tr") { UseShellExecute = true
});
```

8. ListBox

Kullanıcının bir veya daha fazla seçim yapabildiği ve bu seçimler üzerinde işlem yapılabilen bir liste sunar. Ekleme ve silme işlemleri 4. bölümde işlenecektir.

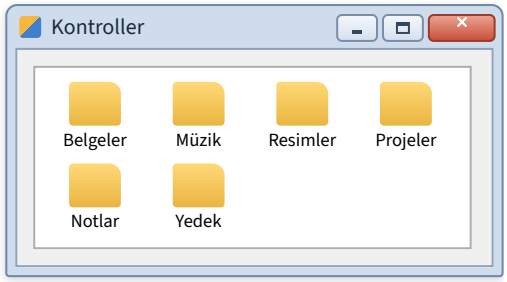


Özellik / Üye	Açıklama
`Items`	Elemanlar: `Add`, `AddRange`, `Clear`, `Remove`, `RemoveAt`
`SelectedItem`	/ `SelectedIndex` seçili eleman ve indisi
`SelectionMode`	One, MultiSimple, MultiExtended (çoklu seçim)
`Sorted`	`true` ise elemanlar alfabetik sıralanır

```
private void Form1_Load(object sender, EventArgs e)
{
    listBox1.Items.AddRange(new string[] { "İstanbul", "Malatya", "İzmir", "Antalya", "İzmit"
});
    listBox1.SelectionMode = SelectionMode.MultiExtended;   // Ctrl/Shift ile çoklu seçim
}
```

9. ListView

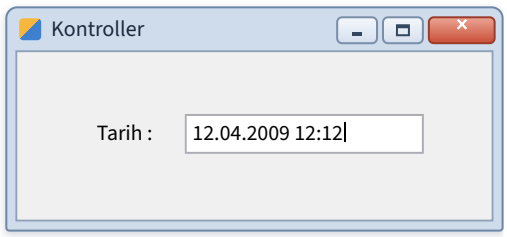
Farklı görünüm tipleri içeren bir liste yapısı sunar. Windows Gezgini'ndeki klasör görünümü buna örnektir: aynı öğeler büyük simge, küçük simge, liste ya da ayrıntılı tablo olarak gösterilebilir.



Özellik / Üye	Açıklama
`View`	LargeIcon, SmallIcon, List, Details, Tile
`Items`	Satırlar (her biri `ListViewItem`)
`Columns`	Details görünümündeki sütun başlıkları

10. MaskedTextBox

Normal TextBox'tan farklı olarak kullanıcıyı belirlenmiş bir kalıba uygun yazmaya zorlar. Böylece hatalı giriş baştan engellenir.



Özellik / Üye	Açıklama
`Mask`	Giriş kalıbı
`Text`	Girilen metin
`MaskCompleted`	Kalıbın zorunlu yerleri doldurulduysa `true`

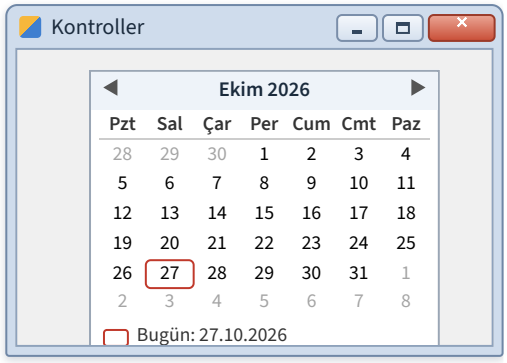
Mask karakteri	Anlamı
`0`	Rakam, zorunlu
`9`	Rakam ya da boşluk, isteğe bağlı
`L`	Harf, zorunlu
`?`	Harf, isteğe bağlı
`/` ve `:`	Tarih ve saat ayırıcısı (Türkçe sistemde tarih ayırıcısı nokta görünür)
`\`	Kendisinden sonraki karakteri kalıp karakteri değil, sabit yazı yapar

```
private void Form1_Load(object sender, EventArgs e)
{
    maskedTextBox1.Mask = "00/00/0000 90:00";   // tarih ve saat kalıbı
}

private void button1_Click(object sender, EventArgs e)
{
    if (maskedTextBox1.MaskCompleted)
        MessageBox.Show("Girilen: " + maskedTextBox1.Text);
    else
        MessageBox.Show("Lütfen kalıbı eksiksiz doldurunuz.");
}
```

11. MonthCalendar

DateTimePicker gibi formda takvim bulunmasını sağlar. Farkı, takvimi açılır kutu yerine doğrudan form üzerinde göstermesidir. Bir tarih aralığı da seçilebilir.



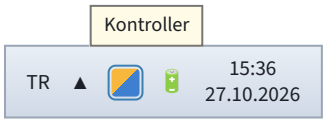
Özellik / Üye	Açıklama
`ShowWeekNumbers`	Hafta numaralarını gösterir
`ShowToday`	Alttaki *Bugün* yazısını gösterir
`ShowTodayCircle`	Bugünün tarihini çerçeve içine alır
`TodayDate`	Takvimin “bugün” kabul ettiği tarih

```
private void Form1_Load(object sender, EventArgs e)
{
    monthCalendar1.ShowWeekNumbers = true;
    monthCalendar1.ShowToday = true;
    monthCalendar1.ShowTodayCircle = true;
}

private void monthCalendar1_DateSelected(object sender, DateRangeEventArgs e)
{
    MessageBox.Show("Seçilen gün: " + e.Start.ToShortDateString());
}
```

12. NotifyIcon

Uygulamanın simgesini görev çubuğunun sağındaki bildirim alanında (saatin yanında) gösterir. Arka planda çalışan programlarda kullanılır.

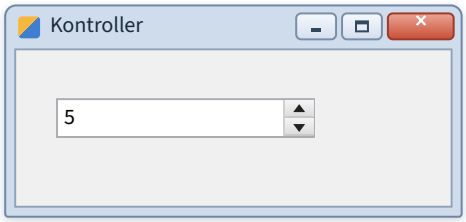


Özellik / Üye	Açıklama
`Icon`	Gösterilecek simge (atanmazsa görünmez)
`Text`	Fare üzerine gelince çıkan yazı
`Visible`	Simgenin görünmesi için `true` olmalıdır

```
private void Form1_Load(object sender, EventArgs e)
{
    notifyIcon1.Icon = this.Icon;        // simge atanmazsa bildirim alanında görünmez
    notifyIcon1.Text = "Kontroller";     // fare üzerine gelince görünen yazı
    notifyIcon1.Visible = true;
}
```

13. NumericUpDown

Form üzerinde, yukarı/aşağı oklarla değiştirilebilen bir sayı sayacı oluşturur. Kullanıcının sayı dışında bir şey yazmasını engeller.



Özellik / Üye	Açıklama
`Value`	Güncel değer (**decimal** türünde)
`Minimum`	/ `Maximum` alt ve üst sınır (varsayılan 0 ve 100)
`Increment`	Her ok tıklamasındaki artış miktarı
`DecimalPlaces`	Gösterilecek ondalık basamak sayısı

```
private void Form1_Load(object sender, EventArgs e)
{
    numericUpDown1.Minimum = 1;
    numericUpDown1.Maximum = 10;
    numericUpDown1.Increment = 1;
    numericUpDown1.Value = 5;
}

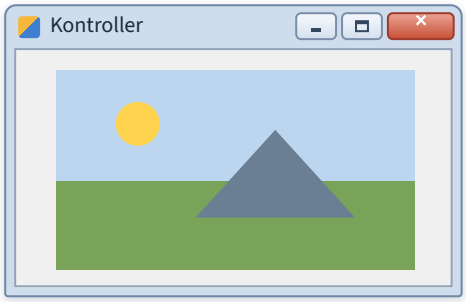
private void button1_Click(object sender, EventArgs e)
{
    int adet = Convert.ToInt32(numericUpDown1.Value);   // Value decimal türündedir
    MessageBox.Show(adet + " adet seçildi.");
}
```

DİKKAT

`Value` değeri `Minimum` – `Maximum` aralığı dışında bir sayıya ayarlanırsa program çalışırken **ArgumentOutOfRangeException** hatası oluşur. Aynı kural ProgressBar için de geçerlidir.

14. PictureBox

Form üzerinde resim göstermek için kullanılır. Resim, tasarımda **Image** özelliğinin yanındaki düğmeyle bilgisayardan seçilir.

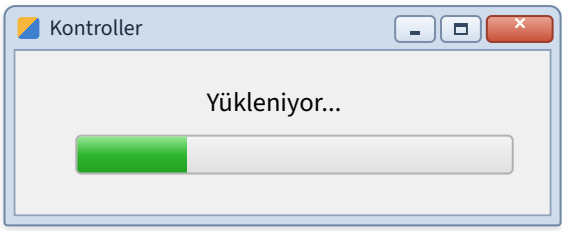


Özellik / Üye	Açıklama
`Image`	Gösterilecek resim
`SizeMode`	Normal, StretchImage, Zoom, CenterImage, AutoSize

```
private void Form1_Load(object sender, EventArgs e)
{
    pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;     // resmi oranını bozmadan sığdırır
}
```

15. ProgressBar

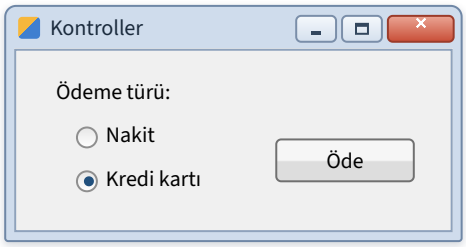
Bir işlemin ilerleme durumunu adım adım gösterir (dosya kopyalama, yükleme gibi).



Özellik / Üye	Açıklama
`Value`	Güncel değer
`Minimum`	/ `Maximum` alt ve üst sınır
`Style`	Blocks, Continuous, Marquee (süresi belirsiz işlemler)

```
private void button1_Click(object sender, EventArgs e)
{
    if (progressBar1.Value + 25 <= progressBar1.Maximum)
    {
        progressBar1.Value += 25;          // her tıklamada %25 ilerler
    }
    else
    {
        MessageBox.Show("İşlem tamamlandı.");
    }
}
```

16. RadioButton

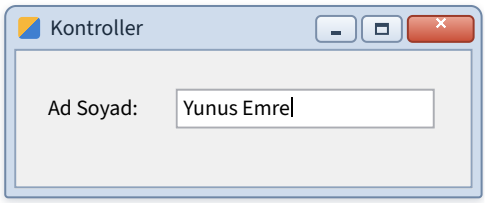


Bir grup seçenekten **yalnızca birinin** seçilmesini sağlar. Aynı kapsayıcıdaki (form ya da GroupBox) radyo butonlardan biri seçilince diğerlerinin seçimi kendiliğinden kalkar. Bu özelliğiyle CheckBox'tan ayrılır.

Özellik / Üye	Açıklama
`Text`	Seçeneğin yazısı
`Checked`	Seçili ise `true`
`CheckedChanged`	Seçim durumu değişince tetiklenen olay

```
private void button1_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked)
        MessageBox.Show("Nakit ödeme seçildi.");
    else if (radioButton2.Checked)
        MessageBox.Show("Kredi kartı seçildi.");
    else
        MessageBox.Show("Lütfen bir ödeme türü seçiniz.");
}
```

17. TextBox

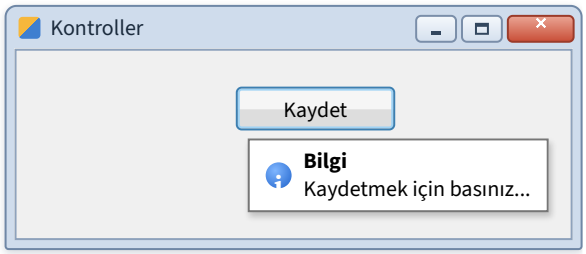


Kullanıcıdan metin almak için kullanılan temel kontroldür. Kutuya yazılan değer her zaman **string** türündedir. Sayı olarak kullanılacaksa dönüştürülmesi gerekir (3. bölüm). Özellikleri Bölüm 2.3'te verilmiştir.

Özellik / Üye	Açıklama
`Text`	Kutudaki metin
`TextChanged`	Metin her değiştiğinde tetiklenen olay
`Clear()`	Kutuyu boşaltan metot
`Focus()`	İmleci kutuya getiren metot

18. ToolTip

Fare bir kontrolün üzerine geldiğinde açıklama gösterir. Forma eklendiğinde formun üzerinde değil, tasarım penceresinin altındaki bileşen tepsisinde görünür. Tek bir ToolTip, formdaki tüm kontroller için kullanılabilir.

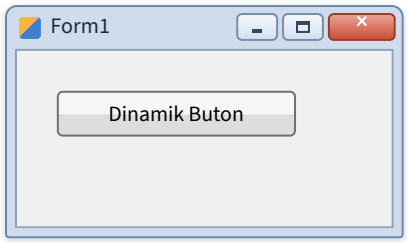


Özellik / Üye	Açıklama
`SetToolTip(kontrol, yazı)`	**Metottur.** Bir kontrole açıklama atar
`ToolTipTitle`	Açıklama kutusunun başlığı
`ToolTipIcon`	Başlık simgesi: Info, Warning, Error

```
private void Form1_Load(object sender, EventArgs e)
{
    toolTip1.SetToolTip(button1, "Kaydetmek için basınız...");
    toolTip1.ToolTipTitle = "Bilgi";
    toolTip1.ToolTipIcon = ToolTipIcon.Info;
}
```

2.5 Nesne Tabanlı Bakış: Kod ile Kontrol Oluşturma

Toolbox'tan sürüklenen her kontrol için Visual Studio, Designer dosyasına `this.button1 = new System.Windows.Forms.Button();` benzeri satırlar yazar. Aynı işi biz de kod ile, program çalışırken yapabiliriz. Bu, kontrollerin gerçekten birer nesne olduğunu en açık biçimde gösteren örnektir:




```
private void Form1_Load(object sender, EventArgs e)
{
    Button yeniButon = new Button();          // Button sınıfından yeni bir nesne
    yeniButon.Text = "Dinamik Buton";
    yeniButon.Location = new Point(20, 20);
    yeniButon.Width = 120;
    yeniButon.Click += YeniButon_Click;       // olaya metot bağlanır
    this.Controls.Add(yeniButon);             // nesne formun kontrollerine eklenir
}

private void YeniButon_Click(object sender, EventArgs e)
{
    MessageBox.Show("Kod ile oluşturulan butona tıklandı.");
}
```

1. `new Button()` ile **Button sınıfından** yeni bir nesne oluşturulur. Bu nesne şu an yalnızca bellektedir.
2. Nesnenin özellikleri (`Text`, `Location`, `Width`) ayarlanır.
3. `+=` işleci ile nesnenin **Click** olayına bir metot bağlanır. Designer dosyasındaki olay bağlantısı da tam olarak böyle yazılır.
4. `this.Controls.Add(...)` ile nesne formun kontrol koleksiyonuna eklenir ve ancak bu satırdan sonra ekranda görünür.

ÖZET

- Her kontrol bir **sınıftır**. Forma eklenen `button1`, `textBox1` gibi öğeler bu sınıfların **nesneleridir**.
- Özellikler tasarımda Properties penceresinden, çalışma anında kod ile değiştirilir. Properties ayarları Designer dosyasına kod olarak yazılır.
- `Visible = false` kontrolü gizler. `Enabled = false` ise kontrolü görünür bırakır ama pasif yapar. Özelliğin adı **Enabled**'dır.
- Tek seçim için **RadioButton** ya da **ComboBox**, çoklu seçim için **CheckBox** ya da **CheckedListBox** kullanılır. Kalıplı giriş için **MaskedTextBox**, sınırlı sayı girişi için **NumericUpDown**, tarih için **DateTimePicker** seçilir.
- `GetItemChecked`, `SetToolTip`, `Clear`, `Focus` birer **metottur** ve parantezle çağrılır. Bunlara `=` ile değer atanamaz.
- NumericUpDown'ın `Value` özelliği **decimal** türündedir. Value ile ProgressBar değerleri sınırların dışına çıkarsa çalışma anında hata oluşur.
- Kontroller kod ile de oluşturulabilir: `new` → özellikleri ayarla → olaya bağla (`+=`) → `Controls.Add`.

2.6 Kendini Test Et

Soru 2.1 TASARIM KARARI Doğru kontrol, doğru ayar ◆◆◇

Bir öğrenci kayıt formu tasarlanmaktadır. Aşağıdaki her ihtiyaç için (a) kullanılacak kontrolü, (b) Load olayında yapılması gereken özellik ayarlarını kod olarak ve (c) en akla yatkın alternatif kontrol yerine bunu neden seçtiğinizi bir cümleyle yazınız.

No	İhtiyaç
1	Cep telefonu numarası mutlaka **0(5xx) xxx xx xx** biçiminde girilmeli. Baştaki 0 kullanıcıya yazdırılmamalı, ekranda sabit durmalıdır.
2	Öğrencinin yaşı 18 ile 65 arasında bir tam sayı olmalı. Harf ya da aralık dışı değer girilememelidir.
3	Doğum tarihi seçilecek. Bugünden ileri bir tarih seçilememelidir.
4	Bölüm, sabit bir listeden (YBS, İşletme, İktisat) seçilecek. Listede olmayan bir bölüm elle yazılamamalıdır.
5	Şifre girilirken karakterler ekranda görünmemelidir.
6	Adres en fazla 200 karakter olmalı ve birden çok satıra yazılabilmelidir.

Soru 2.2 HATA AVI Altı satır, altı hata ◆◆◇

Aşağıdaki Load metodunun her satırında bir hata vardır ve kod derlenmemektedir. Her satır için hatanın nedenini açıklayınız ve satırı, yazanın amacını koruyarak düzeltiniz. 5. satırdaki amaç bir ComboBox'ta aynı anda 5 eleman göstermektir.

```
1 private void Form1_Load(object sender, EventArgs e)
2 {
3     button1.Enable = false;
4     button2.FlatSytle = FlatStyle.Flat;
5     dateTimePicker1.MaxDropDownItems = 5;
6     toolTip1.SetToolTip = "Kaydet";
7     label1.Text = 25;
8     numericUpDown1.Value = 2.5;
9 }
```

Soru 2.3 İZLEME TABLOSU Visible ve Enabled izleme tablosu ◆◆◆

Formda `button1`, `button2` ve `button3` bulunmaktadır. Başlangıçta üç buton da görünür (Visible) ve aktiftir (Enabled). Butonların Click olaylarına aşağıdaki metotlar bağlanmıştır.

```
private void button1_Click(object sender, EventArgs e)
{
    button2.Enabled = !button2.Enabled;
}

private void button2_Click(object sender, EventArgs e)
{
    button3.Visible = !button3.Visible;
    button1.Enabled = false;
}

private void button3_Click(object sender, EventArgs e)
{
    button1.Enabled = true;
    button2.Visible = false;
}
```

Kullanıcı sırasıyla şu butonlara tıklamaya **çalışıyor**: **button2** → **button1** → **button3** → **button1** → **button2** → **button3**. Her adımdan sonra butonların durumunu tabloya işleyiniz (G: görünür, Gz: görünmez, A: aktif, P: pasif). Tıklamanın hiç işlem yapmadığı adımları ve nedenini belirtiniz.

Adım	Tıklanan	İşlem yapıldı mı?	button1	button2	button3
0	—	—	G, A	G, A	G, A
1	button2				
2	button1				
3	button3				
4	button1				

Adım	Tıklanan	İşlem yapıldı mı?	button1	button2	button3
5	button2				
6	button3				

Son durumda kullanıcı button2'yi bir daha görebilir mi? Kodu inceleyerek gerekçelendiriniz.

Soru 2.4 KOD YAZMA Mini proje: Kayıt özeti formu ◆◆◆

Aşağıdaki kontrollerden oluşan bir form tasarlayınız ve kodlayınız:

- textBox1 (ad soyad), comboBox1 (bölüm: YBS, İşletme, İktisat; yalnızca listeden seçilebilir), numericUpDown1 (sınıf: 1–4), maskedTextBox1 (telefon, Soru 2.1'deki kalıp), dateTimePicker1 (doğum tarihi, ileri tarih seçilemez), checkBox1 (*Duyuru bültenine katıl*), button1 (*Özetle*).
- Fare button1'in üzerine gelince *Bilgileri kontrol edip özetler* açıklaması görünsün.
- button1'e tıklandığında; ad soyad boşsa, bölüm seçilmemişse ya da telefon eksikse uygun bir uyarı verilsin ve işlem durdurulsun. Her şey doğruysa bütün bilgiler tek bir MessageBox'ta, alt alta satırlar hâlinde gösterilsin.

Soru 2.5 KOD OKUMA Kod ile oluşturulan butonlar ◆◆◆

```
1  private void Form1_Load(object sender, EventArgs e)
2  {
3      for (int i = 1; i <= 3; i++)
4      {
5          Button b = new Button();
6          b.Text = "Buton " + i;
7          b.Location = new Point(20, 40 * i);
8          b.Click += Dinamik_Click;
9          this.Controls.Add(b);
10     }
11 }
12
13 private void Dinamik_Click(object sender, EventArgs e)
14 {
15     Button tiklanan = (Button)sender;
16     MessageBox.Show(tiklanan.Text);
17 }
```

a. Form açıldığında kaç buton görünür? Her birinin üzerindeki yazıyı ve konumunu (x, y) yazınız.

b. Kullanıcı ortadaki butona tıklarsa ne olur? Metodun, tıklananın ortadaki buton olduğunu nasıl "bildiğini" açıklayınız.

c. 8. satır silinirse ne değişir? 9. satır silinirse ne değişir? İki durumu ayrı ayrı açıklayınız.

d. Döngüde `Button b = new Button();` satırı döngünün **dışına** (üstüne) alınırsa ne olur? Kaç nesne oluşur?

2.7 Cevap Anahtarı

Soru 2.1

No	Kontrol	Neden bu kontrol?
1	MaskedTextBox	TextBox biçimi denetlemez. Mask, her rakamın yerini belirler. `MaskCompleted` ile eksik giriş de yakalanır.
2	NumericUpDown	TextBox'a harf yazılabilir ve sayıya çevirmek gerekir. NumericUpDown yalnızca sayı kabul eder, sınırları kendisi uygular.
3	DateTimePicker	Tarih elle yazılırsa biçim hataları oluşur. MaxDate ile ileri tarihler takvimde seçilemez hâle gelir.
4	ComboBox (DropDownList)	Tek seçim gerektiği için CheckedListBox uygun değildir. DropDownList stili elle yazmayı kapatır.
5	TextBox + PasswordChar	Karakterler girilir ama ekranda `*` görünür.
6	TextBox + Multiline + MaxLength	Tek satırlı kutu adres için yetersizdir. MaxLength sınırı kendisi uygular.

```
private void Form1_Load(object sender, EventArgs e)
{
    // 1) Telefon: baştaki 0 kalıp karakteri sayılmasın diye önüne \ konur
    maskedTextBox1.Mask = "\\0(000) 000 00 00";
    // 2) Yaş aralığı
    numericUpDown1.Minimum = 18;
    numericUpDown1.Maximum = 65;
    // 3) Doğum tarihi bugünden ileri olamaz
    dateTimePicker1.MaxDate = DateTime.Today;
    // 4) Bölüm yalnızca listeden seçilsin, elle yazılamasın
    comboBox1.DropDownStyle = ComboBoxStyle.DropDownList;
    // 5) Şifre gizli görünsün
    textBox1.PasswordChar = '*';
    // 6) Adres alanı çok satırlı ve en fazla 200 karakter
    textBox2.Multiline = true;
    textBox2.MaxLength = 200;
}
```

1. ihtiyaçla ilgili ayrıntı: Mask içinde `0` bir kalıp karakteridir (zorunlu rakam). Kalıp `"0(000) 000 00 00"` yazılsaydı, baştaki 0 da kullanıcının yazacağı bir rakam yeri olurdu ve 5321234567 girildiğinde kutuda 5(321) 234 56 7_ görünürdü. Sabit bir 0 için ters bölü ile kaçış yapılır. Mask metni `\0(000) 000 00 00` olur, C# metni içinde ters bölü iki kez yazılır. Bu kalıpla aynı giriş 0(532) 123 45 67 olarak görünür.

Denetim: İki kalıp da aynı girişle denenmiştir.

Soru 2.2

Satır	Hata nedeni	Düzeltme
3	Özelliğin adı `Enabled`'dır, `Enable` diye bir üye yoktur.	`button1.Enabled = false;`
4	Yazım hatası: `FlatSytle` → `FlatStyle`.	`button2.FlatStyle = FlatStyle.Flat;`
5	DateTimePicker'da `MaxDropDownItems` yoktur. Bu özellik ComboBox'a aittir.	`comboBox1.MaxDropDownItems = 5;`

Satır	Hata nedeni	Düzeltme
6	`SetToolTip` bir metottur. Metoda değer atanamaz, metot parametreyle çağrılır.	`toolTip1.SetToolTip(button1, "Kaydet");`
7	`Text` string türündedir. int değer doğrudan atanamaz.	`label1.Text = "25";`
8	`Value` decimal türündedir. `2.5` double sabitidir ve kendiliğinden decimal'e dönüşmez.	`numericUpDown1.Value = 2.5m;`

```
private void Form1_Load(object sender, EventArgs e)
{
    button1.Enabled = false;
    button2.FlatStyle = FlatStyle.Flat;
    comboBox1.MaxDropDownItems = 5;          // bu özellik ComboBox'a aittir
    toolTip1.SetToolTip(button1, "Kaydet");  // SetToolTip bir metottur
    label1.Text = "25";                      // ya da 25.ToString()
    numericUpDown1.Value = 2.5m;             // decimal sabiti m son eki ile yazılır
}
```

Denetim: Hatalı kod derlendiğinde derleyici tam olarak bu altı satırda hata vermektedir. Düzeltilmiş kod hatasız derlenir.

Soru 2.3

Adım	Tıklanan	İşlem yapıldı mı?	button1	button2	button3
0	—	—	G, A	G, A	G, A
1	button2	Evet	G, **P**	G, A	**Gz**, A
2	button1	**Hayır**: button1 pasif	G, P	G, A	Gz, A
3	button3	**Hayır**: button3 görünmez	G, P	G, A	Gz, A
4	button1	**Hayır**: button1 hâlâ pasif	G, P	G, A	Gz, A
5	button2	Evet	G, P	G, A	**G**, A
6	button3	Evet	G, **A**	**Gz**, A	G, A

5. adımda `button3.Visible = !button3.Visible` görünmez olan butonu yeniden görünür yapar. Aynı adımda `button1.Enabled = false` zaten pasif olan butonu yine pasif bırakır. Pasif bir butona tıklandığında Click olayı tetiklenmez, görünmez bir butona ise tıklanamaz.

button2 bir daha görülemez. Kodda `button2.Visible` değerini `true` yapan hiçbir satır yoktur. button2'yi gizleyen satır button3'ün metodunda yazılıdır ve görünür hâle getiren bir karşılığı bulunmaz.

Soru 2.4

```
private void Form1_Load(object sender, EventArgs e)
{
    comboBox1.Items.AddRange(new string[] { "YBS", "İşletme", "İktisat" });
    comboBox1.DropDownStyle = ComboBoxStyle.DropDownList;
    maskedTextBox1.Mask = "\\0(000) 000 00 00";
    numericUpDown1.Minimum = 1;
    numericUpDown1.Maximum = 4;
    dateTimePicker1.MaxDate = DateTime.Today;
    toolTip1.SetToolTip(button1, "Bilgileri kontrol edip özetler");
```

```
}

private void button1_Click(object sender, EventArgs e)
{
    if (textBox1.Text.Trim() == "")
    {
        MessageBox.Show("Ad soyad boş bırakılamaz.");
        return;
    }
    if (comboBox1.SelectedIndex == -1)
    {
        MessageBox.Show("Lütfen bölüm seçiniz.");
        return;
    }
    if (!maskedTextBox1.MaskCompleted)
    {
        MessageBox.Show("Telefon numarasını eksiksiz giriniz.");
        return;
    }

    string bulten = "Hayır";
    if (checkBox1.Checked)
        bulten = "Evet";

    string ozet = "Ad Soyad: " + textBox1.Text + "\n"
                + "Bölüm: " + comboBox1.Text + "\n"
                + "Sınıf: " + numericUpDown1.Value + "\n"
                + "Telefon: " + maskedTextBox1.Text + "\n"
                + "Doğum tarihi: " + dateTimePicker1.Value.ToShortDateString() + "\n"
                + "Bülten: " + bulten;
    MessageBox.Show(ozet, "Kayıt Özeti");
}
```

`return;` satırı metodu o noktada bitirir. Böylece hatalı durumda özet gösterilmez. `\n` MessageBox içinde yeni satıra geçer. `numericUpDown1.Value` decimal olduğu hâlde metinle `+` ile birleştirilebilir, çünkü metinle toplanan her değer otomatik olarak metne çevrilir.

Soru 2.5

a) 3 buton: *Buton 1* (20, 40), *Buton 2* (20, 80), *Buton 3* (20, 120). y değeri `40 * i` ile hesaplanır.

b) Ekranda **Buton 2** yazan bir mesaj çıkar. Üç butonun Click olayı da aynı `Dinamik_Click` metoduna bağlanmıştır. Metot hangi butondan çağrıldığını `sender` parametresinden öğrenir: `sender` o anda tıklanan nesnedir. `(Button)sender` ile Button türüne dönüştürülür ve `Text` özelliği okunur.

c) 8. satır (`b.Click += …`) silinirse butonlar görünür ama tıklanınca hiçbir şey olmaz, çünkü olaya bağlı metot yoktur. **9. satır** (`Controls.Add`) silinirse butonlar bellekte oluşturulur ama forma eklenmediği için **hiçbiri görünmez**.

d) Yalnızca **bir** Button nesnesi oluşur. Döngü aynı nesnenin yazısını ve konumunu değiştirir; form aynı nesneyi tek kez tutar. Ekranda (20, 120) konumunda *Buton 3* yazan tek buton görünür. Click olayına aynı metot üç kez bağlandığı için tıklanınca mesaj **üç kez** gelir. Her tur için ayrı nesne gerekiyorsa `new` döngünün içinde olmalıdır.

BÖLÜM

3

Veri Tipleri, Tür Dönüşümü ve Kontrol Yapıları



BU BÖLÜMÜN SONUNDA

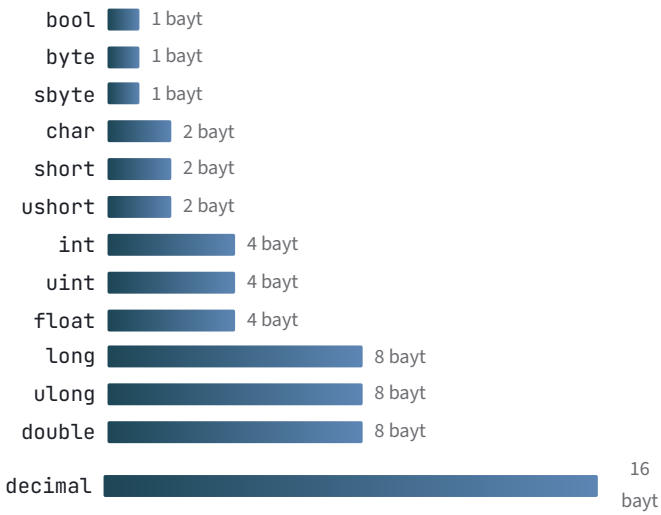
- C#'taki temel veri tiplerini, boyutlarını ve kullanım yerlerini bilecek,
- Formdan alınan string verileri sayıya, sayıları string'e dönüştürebilecek,
- Bölge ayarı, tamsayı bölmesi ve metin birleştirme tuzaklarını tanıyacak,
- Atama, karşılaştırma ve mantıksal operatörleri kullanabilecek,
- if–else, for ve while yapılarını form uygulamalarında kullanabilecek,
- Math sınıfının metotlarıyla hesaplama yapabileceksiniz.

Form ekranında görünen her şey metindir. Oysa programlar sayılarla hesap yapar, karar verir ve işleri tekrarlar. Bu bölümde formdan veri almayı, onu doğru türe çevirmeyi ve karar ile döngü yapılarıyla işlemeyi öğreniyoruz.

3.1 Veri Tipleri

Bir değişken tanımlanırken bellekte ne kadar yer kaplayacağı ve hangi değerleri alabileceği **veri tipi** ile belirlenir. C#'ta temel tipler üç grupta toplanır: **sayısal tipler** (byte, sbyte, short, ushort, int, uint, long, ulong, float, double, decimal), **metinsel tipler** (char, string) ve **mantıksal tip** (bool).

Tip	Boyut	Değer aralığı / açıklama
`byte`	1 bayt	İşaretsiz tamsayı: 0 … 255
`sbyte`	1 bayt	İşaretli tamsayı: −128 … 127
`short`	2 bayt	İşaretli tamsayı: −32 768 … 32 767
`ushort`	2 bayt	İşaretsiz tamsayı: 0 … 65 535
`int`	4 bayt	İşaretli tamsayı: yaklaşık ±2,1 milyar
`uint`	4 bayt	İşaretsiz tamsayı: 0 … yaklaşık 4,3 milyar
`long`	8 bayt	İşaretli tamsayı: yaklaşık $\pm 9{,}2 \times 10^{18}$
`ulong`	8 bayt	İşaretsiz tamsayı: 0 … yaklaşık $1{,}8 \times 10^{19}$
`float`	4 bayt	Ondalıklı sayı, yaklaşık 7 basamak duyarlık
`double`	8 bayt	Ondalıklı sayı, yaklaşık 15–16 basamak duyarlık
`decimal`	16 bayt	Ondalıklı sayı, 28–29 basamak; para hesapları için
`char`	2 bayt	Tek bir Unicode karakter: `'A'`
`string`	değişken	Karakter dizisi (metin): `"Merhaba"`
`bool`	1 bayt	Yalnızca `true` veya `false`



Şekil 3.1 *Tiplerin bellekte kapladığı alan*

DİKKAT

Bazı kaynaklarda `bool` tipinin 2 bayt olduğu yazar. C#'ta `sizeof(bool)` değeri **1**'dir. Benzer biçimde string için "2 milyar karaktere kadar" ifadesi geçer. .NET'te bir string'in alabileceği en fazla uzunluk yaklaşık **1 milyar** (1 073 741 791) karakterdir ve pratikte sınırı bilgisayarın belleği belirler.

DEĞİŞKEN TANIMLAMA ÖRNEKLERİ

```
int ogrenciSayisi = 45;
double ortalama = 72.5;
float oran = 0.75f;        // float sabitleri f son eki alır
decimal fiyat = 149.90m;   // decimal sabitleri m son eki alır
char harf = 'A';           // tek tırnak, tek karakter
string ad = "Yunus";       // çift tırnak, metin
bool aktif = true;         // yalnızca true veya false
```

Ondalıklı bir sabit (ör. `72.5`) C#'ta varsayılan olarak **double** kabul edilir. float'a atamak için sonuna `f`, decimal'e atamak için `m` eklenir. Bu son ek unutulursa derleyici "double türü kendiliğinden float/decimal'e dönüştürülemez" hatası verir. Kod içindeki ondalık ayırıcı, bilgisayarın dil ayarı ne olursa olsun her zaman **nokta**dır.

3.2 Formdan Veri Almak: Tür Dönüşümleri

Form üzerinden aldığımız veriler (TextBox, Label, ComboBox'ın `Text` özelliği) **string** türündedir. Girilen veri sayı olarak kullanılacaksa önce sayıya **çevrilmelidir**. Tersine, forma yazdıracağımız bir sayıyı da önce string'e çevirmemiz gerekir.



Tür dönüşümü: veriyi kullanılacağı yerin biçimine uyarlamak

TEMEL DÖNÜŞÜMLER

```
// string → sayı
int yas = Convert.ToInt32(textBox1.Text);
double boy = Convert.ToDouble(textBox2.Text);
int adet = int.Parse(textBox3.Text);

// sayı → string
label1.Text = yas.ToString();          // 1. yol: ToString()
label2.Text = Convert.ToString(boy);   // 2. yol: Convert.ToString()
```

Yöntem	Ne yapar?	Geçersiz giriş ("abc") olursa
`Convert.ToInt32(metin)`	Metni int'e çevirir	FormatException hatası
`Convert.ToDouble(metin)`	Metni double'a çevirir	FormatException hatası
`int.Parse(metin)`	Metni int'e çevirir	FormatException hatası
`double.TryParse(metin, out x)`	Çevirmeyi **dener**; başarılıysa `true` döner ve sonucu x'e yazar	Hata vermez, `false` döner
`sayi.ToString()`	Sayıyı metne çevirir	—
`Convert.ToString(sayi)`	Sayıyı metne çevirir	—

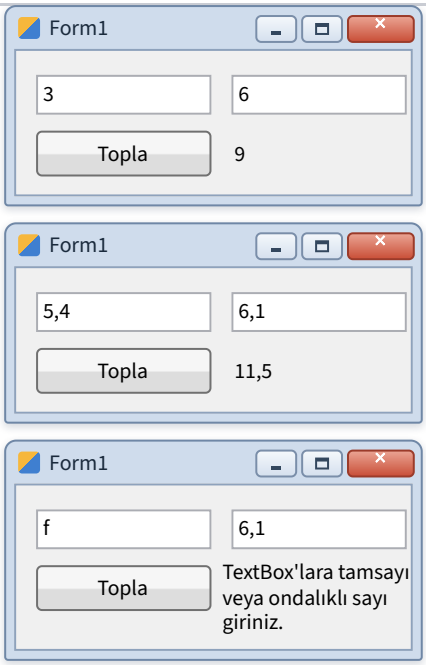
Kullanıcıdan gelen veriye hiçbir zaman güvenilmemelidir. Boş bırakılan bir kutu ya da yanlışlıkla yazılan bir harf, `Convert` kullanan programı hata vererek durdurur. Bu yüzden kullanıcı girişlerinde `TryParse` tercih edilir. Aşağıda, iki sayıyı toplayan programın hem ders sunumundaki hâli hem de hatalı girişe dayanıklı hâli verilmiştir.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs
e)
{
    double sayi1, sayi2, sonuc;
    sayi1 = Convert.ToDouble(textBox1.Text);
    sayi2 = Convert.ToDouble(textBox2.Text);
    sonuc = sayi1 + sayi2;
    label1.Text = Convert.ToString(sonuc);
}
```

DAYANIKLI ÇÖZÜM (TRYPARSE)

```
private void button1_Click(object sender, EventArgs
e)
{
    double sayi1, sayi2;
    if (double.TryParse(textBox1.Text, out sayi1) &&
double.TryParse(textBox2.Text, out sayi2))
    {
        label1.Text = (sayi1 + sayi2).ToString();
    }
    else
    {
        label1.Text = "TextBox'lara tamsayı veya
ondalıklı sayı giriniz.";
    }
}
```



DİKKAT

Bölge ayarı tuzağı. `Convert.ToDouble` ve `double.Parse`, bilgisayarın **bölge ayarına** göre çalışır. Türkçe ayarlı bir bilgisayarda ondalık ayırıcı **virgül**, binlik ayırıcı **nokta**dır. Bu yüzden:

Kutuya yazılan	Türkçe Windows'ta Convert.ToDouble sonucu
`5,4`	5,4
`5.4`	**54** (nokta binlik ayırıcı sayılır)

Sunumdaki ekran görüntülerinde sayıların 23,44 gibi virgülle yazılmasının nedeni budur. Kod içinde ise ondalık ayırıcı her zaman noktadır: `double x = 5.4;`

DİKKAT

Birleştirme tuzağı. `+` işleci iki sayı arasında toplama, bir metin ile herhangi bir değer arasında **birleştirme** yapar. `textBox1.Text + textBox2.Text` ifadesi, kutularda 3 ve 6 yazıyorsa 9 değil **"36"** sonucunu verir. Toplama yapabilmek için önce dönüştürmek şarttır.

3.3 Operatörler

Operatör	Eşdeğeri	Anlamı
`x += y`	`x = x + y`	x + y'nin değerini x'e ata
`x -= y`	`x = x - y`	x − y'nin değerini x'e ata
`x *= y`	`x = x * y`	x · y'nin değerini x'e ata
`x /= y`	`x = x / y`	x / y'nin değerini x'e ata
`x %= y`	`x = x % y`	x'in y'ye bölümünden kalanı x'e ata
`x++`	`x = x + 1`	x'i bir artır
`x--`	`x = x - 1`	x'i bir azalt

Operatör	Anlamı
`==`	Eşittir
`!=`	Eşit değil
`>` `<`	Büyük, küçük
`>=` `<=`	Büyük eşit, küçük eşit
`!`	Değil
`&&`	Ve (iki şart da doğru)
`||`	Veya (en az biri doğru)

DİKKAT

Tamsayı bölmesi. İki **int** değer bölündüğünde sonuç da int olur ve ondalık kısım **atılır** (yuvarlanmaz): `7 / 2` → **3**, `10 / 4` → **2**. Sonuç bir double değişkene atansa bile bölme önce int olarak yapılır: `double s = 7 / 2;` ifadesinde s'nin değeri 3'tür. Doğru sonuç için işlenenlerden en az biri ondalıklı olmalıdır: `7 / 2.0` → 3,5. Kalan operatörü `%` ise bölümden kalanı verir: `10 % 4` → 2.

Atama için tek eşittir (`=`), karşılaştırma için çift eşittir (`==`) kullanılır. `if (a = 3)` yazımı derleme hatası verir.

3.4 if – else Yapısı

Program, bir şartın doğru ya da yanlış olmasına göre farklı komutlar çalıştırır. Şart doğruysa `if` bloğu, yanlışsa `else` bloğu çalışır. `else` bloğu isteğe bağlıdır; yazılmazsa şart yanlış olduğunda hiçbir şey yapılmaz. Birden çok şart sırayla denenecekse `else if` kullanılır. Bu yapıda doğru bulunan **ilk** şartın bloğu çalışır, kalanlara bakılmaz.



if – else: şarta göre iki yoldan biri seçilir

TEXTBOX'A GİRİLEN SAYIYA GÖRE MESAJ

```
private void button1_Click(object sender, EventArgs e)
{
    int a = Convert.ToInt32(textBox1.Text);

    if (a < 0)
    {
        label1.Text = "TextBox'a pozitif bir sayı giriniz.";
    }
    else if (a == 3)
    {
        label1.Text = "TextBox'a 3 sayısını girdiniz.";
    }
    else
    {
        label1.Text = "";
    }
}
```

İPUCU

Blokta tek bir komut varsa süslü parantez yazılmayabilir: `if (a < 0) label1.Text = "…";` . Ancak sonradan ikinci bir komut eklendiğinde parantez unutulursa ikinci komut şarttan bağımsız çalışır. Bu yüzden parantez kullanmak iyi bir alışkanlıktır.

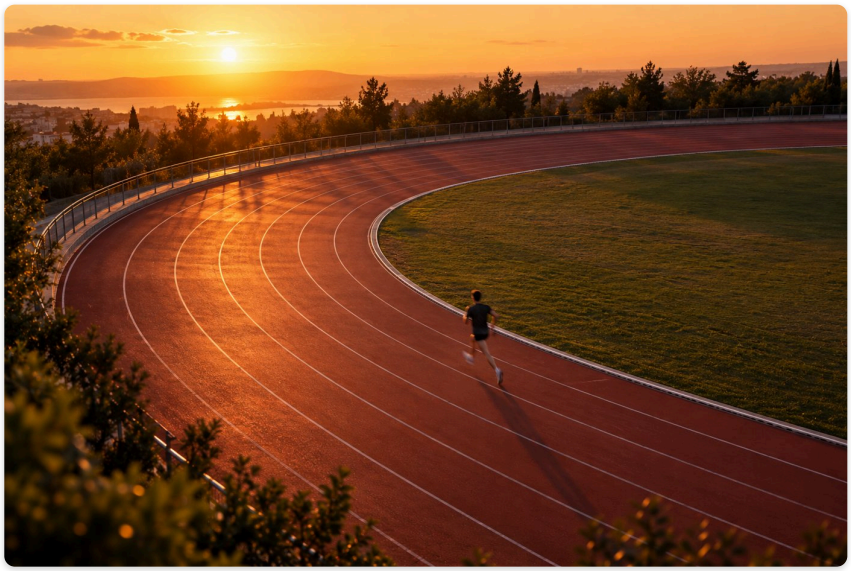
3.5 Döngüler: for ve while

GENEL YAPI

```
for (başlangıç işlemleri; devam şartı; ara işlemler)
{
    komutlar;
}

while (şart)
{
    komutlar;
}
```

for döngüsü, devam şartı sağlandığı sürece komutları tekrarlar. Başlangıç işlemleri yalnızca bir kez, en başta çalışır. Devam şartı her turdan önce kontrol edilir ve **bool** türündedir. Ara işlemler (genellikle sayacın artırılması) her turun sonunda çalışır.



Döngü: şart sağlandıkça aynı tur tekrar tekrar koşulur

FOR: FORMUN BAŞLIĞINDA SAYMA

```
private void button1_Click(object sender, EventArgs e)
{
    int N = 20;
    for (int i = 0; i < N; i++)
    {
        this.Text = i.ToString();   // formun başlık çubuğuna yazar
    }
}
```

Burada int tipinde N değişkenine 20 değeri atanmıştır. Döngüde int tipinde `i` tanımlanır ve başlangıç değeri 0 olur. `i`, N ile karşılaştırılır. i N'den küçük olduğu sürece döngü çalışır ve her turun sonunda `i++` ile bir artar. Her turda i'nin değeri formun başlık çubuğuna (`this.Text`) yazılır. Değer 0'dan 19'a kadar değişir, ancak döngü bir anda bittiği için kullanıcı pratikte yalnızca son değeri, **19**'u görür. Döngü bittiğinde i = 20 olmuştur ve şart (20 < 20) yanlıştır.

while döngüsü, şart doğru olduğu sürece çalışır. Sayacın artırılması bloğun içinde yapılmalıdır, aksi hâlde şart hep doğru kalır ve döngü sonsuza kadar sürer. Form uygulamasında sonsuz döngü, programın donmasına (yanıt vermemesine) yol açar.

WHİLE: 1'DEN A'YA KADAR TOPLAM

```
private void button1_Click(object sender, EventArgs e)
{
    int a = Convert.ToInt32(textBox1.Text);   // üst sınır, örneğin 5
    int b = 0;                                // toplam
    int c = 1;                                // sayaç

    while (c <= a)
    {
        b += c;
        c++;
    }
    label1.Text = "1 ile " + a + " arasındaki sayıların toplamı: " + b;
}
```

Tur	c (başta)	c <= 5 ?	b += c	c++
1	1	doğru	1	2
2	2	doğru	3	3
3	3	doğru	6	4
4	4	doğru	10	5
5	5	doğru	15	6
—	6	**yanlış** → döngü biter	15	—

Tabloda kutuya 5 girildiğinde döngünün her turu izlenmiştir. Bu tür tabloya **izleme tablosu** denir ve bir döngünün ne yaptığını anlamanın en güvenilir yoludur.

3.6 Math Sınıfı

Matematiksel işlemler için **Math** sınıfının hazır metotları ve sabitleri kullanılır. Hepsi **statik**tir. Kullanmak için nesne oluşturulmaz, doğrudan sınıf adıyla çağrılır: `Math.Sqrt(16)`.

Üye	Görevi	Örnek → sonuç
`Math.E`	e sayısı	≈ 2,71828
`Math.PI`	π sayısı	≈ 3,14159
`Math.Sin(b)` `Cos` `Tan`	Trigonometrik fonksiyonlar; b **radyan** cinsindendir	Dereceyi radyana çevirmek için π/180 ile çarpılır
`Math.Exp(b)`	e^b	Math.Exp(1) → 2,718…
`Math.Pow(b, c)`	b^c	Math.Pow(2, 3) → 8
`Math.Sqrt(b)`	Karekök	Math.Sqrt(16) → 4
`Math.Ceiling(b)`	Yukarı (büyük olan tamsayıya) yuvarlar	10,3 → 11 · −10,3 → **−10**

Üye	Görevi	Örnek → sonuç
`Math.Floor(b)`	Aşağı (küçük olan tamsayıya) yuvarlar	10,3 → 10 · −10,3 → **−11**
`Math.Round(b)`	En yakın tamsayıya yuvarlar	10,3 → 10 · 10,8 → 11
`Math.Round(b, c)`	Virgülden sonra c basamak kalacak şekilde yuvarlar	Math.Round(10.3867, 1) → 10,4
`Math.Min(b, c)` `Math.Max(b, c)`	Küçük / büyük olanı verir	Min(3, 4) → 3 · Max(3, 4) → 4
`Math.Abs(b)`	Mutlak değer	Math.Abs(−3) → 3
`Math.Log10(b)`	10 tabanında logaritma	Math.Log10(100) → 2
`Math.Log(b)`	Doğal logaritma (ln)	Math.Log(Math.E) → 1
`Math.Log(b, c)`	c tabanında logaritma	Math.Log(8, 2) → 3

DİKKAT

Yarım değerlerde yuvarlama. `Math.Round` tam ortadaki (,5) değerleri varsayılan olarak **çift sayıya** yuvarlar (bankacı yuvarlaması): `Math.Round(2.5)` → **2**, `Math.Round(3.5)` → **4**. Okulda öğrendiğimiz "5 ve üstü yukarı" kuralı isteniyorsa ikinci parametre verilir: `Math.Round(2.5, MidpointRounding.AwayFromZero)` → 3.

3.7 Form_Load ile Başlangıç Değerleri

Form1_Load olayına yazılan kodlar program yüklenirken çalışır (*load*: yüklemek). Aşağıdaki ikinci derece denklem çözücüsünde kutulara açılışta örnek değerler yazılmıştır: a = 1, b = 0, c = −4. Bu denklemin ($x^2 - 4 = 0$) kökleri 2 ve −2'dir. Hesaplanan sonuç forma yazdırılırken sayı iki farklı yolla string'e çevrilmiştir: **1. yol** `Convert.ToString(...)`, **2. yol** `.ToString()`.

İKİNCİ DERECE DENKLEM: AX² + BX + C = 0

```
private void Form1_Load(object sender, EventArgs e)
{
    textBox1.Text = "1";
    textBox2.Text = "0";
    textBox3.Text = "-4";
    textBox4.Text = "birinci kök";
    textBox5.Text = "ikinci kök";
}

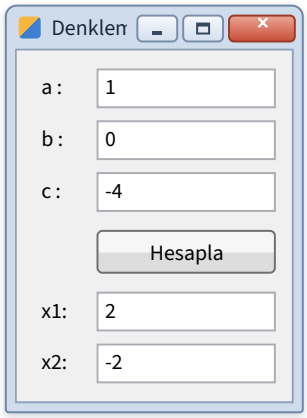
private void button1_Click(object sender, EventArgs e)
{
    double a, b, c, delta, x1, x2;
    a = Convert.ToDouble(textBox1.Text);   // textBox1'in metni double'a çevrilip a'ya atanır
    b = Convert.ToDouble(textBox2.Text);
    c = Convert.ToDouble(textBox3.Text);

    if (a == 0)
    {
        MessageBox.Show("a katsayısı 0 olamaz.");
        return;
    }
```

```
    delta = b * b - 4 * a * c;

    if (delta > 0)
    {
        x1 = (-b + Math.Sqrt(delta)) / (2 * a);
        x2 = (-b - Math.Sqrt(delta)) / (2 * a);
        textBox4.Text = x1.ToString();                    // 2. yol
        textBox5.Text = x2.ToString();
    }
    else if (delta == 0)
    {
        textBox4.Text = Convert.ToString(-b / (2 * a));   // 1. yol
        textBox5.Text = textBox4.Text;                    // çakışık (eşit) kök
    }
    else
    {
        textBox4.Text = "Reel kök yok";
        textBox5.Text = "Reel kök yok";
    }
}
```

Form açıldığında Load olayı kutulara 1, 0 ve −4 değerlerini yazar. **Hesapla** butonuna basıldığında delta = $0^2 - 4 \cdot 1 \cdot (-4) = 16$ bulunur. Delta pozitif olduğu için iki farklı kök hesaplanır: x1 = (0 + 4) / 2 = **2**, x2 = (0 − 4) / 2 = **−2**.



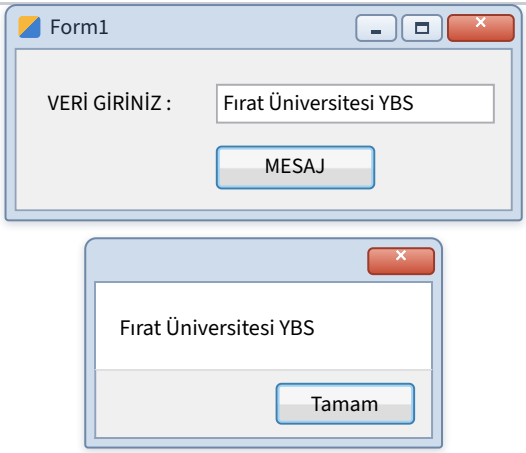
Sunumdaki örnekte delta kontrol edilmeden kökler hesaplanmıştı. Delta negatif olduğunda `Math.Sqrt` hata vermez, sonuç olarak **NaN** (Not a Number: sayı değil) üretir ve kutulara anlamsız bir değer yazılır. a = 0 olduğunda ise sıfıra bölme oluşur. Yukarıdaki kodda bu iki durum da kontrol edilmiştir. Delta sıfır olduğunda iki kök eşittir ve tek formülle (−b / 2a) hesaplanır.

3.8 Çözümlü Uygulamalar

Uygulama 1 — Kutudaki veriyi mesaj olarak gösterme

VERİ GİRİNİZ yazan bir Label, bir TextBox ve MESAJ yazan bir buton forma eklenir. Butona tıklanınca kutuya yazılan veri mesaj olarak gösterilir.

```
private void button1_Click(object sender,
EventArgs e)
{
    MessageBox.Show(textBox1.Text);
}
```

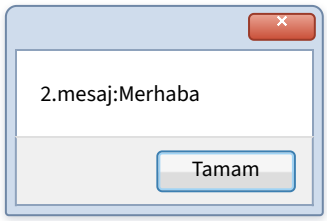


Uygulama 2 — Mesajı 10 kez gösterme

Aynı form kullanılarak kutudaki veri 10 kez, başında sıra numarasıyla gösterilir. `i + ".mesaj:"` ifadesinde int ile string toplandığı için sayı metne çevrilip birleştirilir.

HER MESAJ AYRI PENCEREDE

```
private void button1_Click(object sender, EventArgs e)
{
    for (int i = 1; i <= 10; i++)
    {
        MessageBox.Show(i + ".mesaj:" + textBox1.Text);
    }
}
```



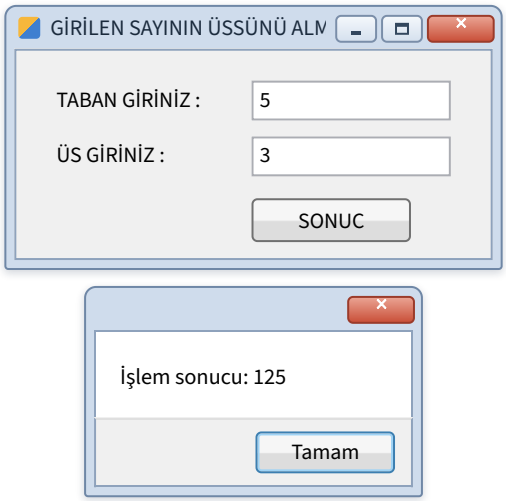
Bu çözümde kullanıcı on kez **Tamam**'a basmak zorundadır. Mesajları tek bir string'de biriktirip tek seferde göstermek daha kullanışlıdır. `\n` yeni satır karakteridir:

TÜM MESAJLAR TEK PENCEREDE

```
private void button1_Click(object sender, EventArgs e)
{
    string mesaj = "";
    for (int i = 1; i <= 10; i++)
    {
        mesaj += i + ". mesaj: " + textBox1.Text + "\n";
    }
    MessageBox.Show(mesaj);      // 10 satır tek pencerede
}
```

Uygulama 3 — Üs alma (Math.Pow)

```
private void button1_Click(object sender, EventArgs
e)
{
    double taban, us;
    taban = Convert.ToDouble(textBox1.Text);   //
metin ondalıklı sayıya çevrilir
    us = Convert.ToDouble(textBox2.Text);
    double sonuc = Math.Pow(taban, us);
    MessageBox.Show("İşlem sonucu: " + sonuc);
}
```



Uygulama 4 — Dört işlem (RadioButton)

İki sayı girilir, **İŞLEM SEÇ** bölümünden hangi radyo buton seçilmişse **İŞLEM SONUCU** butonuna tıklandığında o işlemin sonucu gösterilir. Sunumdaki çözüm aşağıdadır. 2 ve 4 sayıları için çarpma seçilirse 8, bölme seçilirse 0,5 sonucu görünür.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    double sayi1, sayi2, sonuc;

    sayi1 = Convert.ToDouble(textBox1.Text);
    sayi2 = Convert.ToDouble(textBox2.Text);
    if (radioButton1.Checked == true) { sonuc = sayi1 + sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton2.Checked == true) { sonuc = sayi1 - sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton3.Checked == true) { sonuc = sayi1 * sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton4.Checked == true) { sonuc = sayi1 / sayi2; MessageBox.Show("" + sonuc); }
}
```



Şekil 3.2 *Çarpma seçiliyken 2 ve 4 için sonuç*

`radioButton1.Checked == true` karşılaştırması gereksizdir. `Checked` zaten bool değerdir ve `if (radioButton1.Checked)` yazmak aynı anlama gelir. Bu çözüm çalışır, ancak geliştirilmeye açık noktaları vardır: hiçbir işlem seçilmezse kullanıcıya bilgi verilmez, sıfıra bölme kontrol edilmez ve aynı MessageBox satırı dört kez tekrarlanır. Bu çözümü iyileştirmek bu bölümün Kendini Test Et kısmında Soru 3.4 olarak size bırakılmıştır.

ÖZET

- Tamsayılar için çoğunlukla `int`, ondalıklı sayılar için `double`, para için `decimal`, metin için `string`, doğru/yanlış için `bool` (1 bayt) kullanılır.
- Formdan gelen her veri **string**'dir. Sayıya `Convert.ToInt32/ToDouble`, `Parse` ya da güvenli biçimde `TryParse` ile çevrilir. Sayı string'e `ToString()` ya da `Convert.ToString()` ile çevrilir.
- Türkçe sistemde ondalık ayırıcı virgüldür. `"5.4"` metni 54 olarak okunur.
- `+` metinle birlikte kullanılınca birleştirir. İki int'in bölümü int'tir (`7/2 = 3`).
- `if – else if – else` zincirinde doğru olan ilk bloğun kodu çalışır. `for` sayaçlı tekrarlar, `while` şarta bağlı tekrarlar içindir.
- Math sınıfının metotları statiktir. `Math.Round` yarım değerleri çift sayıya yuvarlar. Negatif sayının karekökü **NaN**'dır.

3.9 Kendini Test Et

Soru 3.1 ÇIKTI TAHMİNİ Ekranda ne yazar? ◆◆◇

Aşağıdaki her satır bir butonun Click metodunda tek başına çalıştırılmaktadır. Bilgisayarın bölge ayarı Türkçedir. Açılan mesaj kutusunda tam olarak ne yazacağını tahmin ediniz ve **nedenini** bir cümleyle açıklayınız.

	Kod
a	`MessageBox.Show("Toplam: " + 2 + 3);`
b	`MessageBox.Show(2 + 3 + " adet");`
c	`int a = 7, b = 2; double s = a / b; MessageBox.Show(s.ToString());`
d	`textBox1.Text = "12"; textBox2.Text = "8";` `MessageBox.Show(textBox1.Text + textBox2.Text);`
e	`MessageBox.Show(Math.Round(2.5) + " / " + Math.Round(3.5));`
f	`int s = 0; for (int i = 1; i <= 10; i++) if (i % 3 == 0) s += i; MessageBox.Show(s.ToString());`
g	`MessageBox.Show(Convert.ToDouble("5.4").ToString());` *(Türkçe Windows)*
h	`double t = 0.1 + 0.2; MessageBox.Show((t == 0.3).ToString());`
i	`MessageBox.Show(Math.Ceiling(-10.3) + " ve " + Math.Floor(-10.3));`

Soru 3.2 İZLEME TABLOSU İki döngüyü izlemek

◆◆◇

a) Aşağıdaki kod için izleme tablosunu doldurunuz ve label1'e yazılacak değeri bulunuz.

```
int a = 5, b = 0, c = 1;
while (c <= a)
{
    b += c;
    c += 2;
}
label1.Text = "b = " + b;
```

Tur	c (başta)	c <= a ?	b (tur sonunda)	c (tur sonunda)
1				
2				
3				
4				

b) Aşağıdaki iç içe döngü sonunda label1'de ne yazar? Dış döngünün her turunda `s` değişkeninin aldığı değeri yazarak gösteriniz.

```
string s = "";
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
    {
        s += j;
    }
    s += " ";
}
label1.Text = s;
```

Soru 3.3 HATA AVI Ortalama yanlış çıkıyor

◆◆◆

İki sayının ortalamasını hesaplaması beklenen aşağıdaki kod hatasız derlenmektedir.

```
private void button1_Click(object sender, EventArgs e)
{
    int sayi1 = Convert.ToInt32(textBox1.Text);
    int sayi2 = Convert.ToInt32(textBox2.Text);
    label1.Text = "Ortalama: " + sayi1 + sayi2 / 2;
}
```

a. Aşağıdaki test tablosunu, programın **gerçekte** ne yapacağını düşünerek doldurunuz.

textBox1	textBox2	Beklenen	Programın gerçek davranışı
10	20	Ortalama: 15	
7	8	Ortalama: 7,5	

textBox1	textBox2	Beklenen	Programın gerçek davranışı
4,5	3	Ortalama: 3,75	
(boş)	3	Uyarı	

b. Koddaki **üç** ayrı sorunu adlandırınız.

c. Kodu, dört test satırının hepsinde beklenen sonucu verecek şekilde yeniden yazınız.

Soru 3.4 YENİDEN DÜZENLEME Dört işlemi iyileştirmek ◆◆◇

Bölüm 3.8, Uygulama 4'teki sunum çözümünü aşağıdaki şartları sağlayacak şekilde yeniden yazınız:

1. Kutulara sayı dışında bir şey yazılırsa program hata vererek kapanmasın, kullanıcı uyarılsın.
2. Hiçbir işlem seçilmemişse *Lütfen bir işlem seçiniz.* mesajı verilsin.
3. Bölme seçili ve ikinci sayı 0 ise *Sıfıra bölme yapılamaz.* mesajı verilsin.
4. Sonucu gösteren `MessageBox.Show` satırı kodda **yalnızca bir kez** geçsin.

Ek soru: Sunum çözümünde ikinci sayıya 0 girilip bölme seçilirse program hata verip kapanır mı? Ekranda ne görünür? `double` ile `int` bölmesi bu açıdan nasıl farklıdır?

Soru 3.5 KOD YAZMA Harf notu hesaplayıcı ◆◆◆

textBox1'e vize, textBox2'ye final notu girilen ve butona tıklanınca label1'e ortalamayı ve harf notunu yazan programı kodlayınız. Kurallar:

- Ortalama = vize × 0,4 + final × 0,6, virgülden sonra 2 basamağa yuvarlanır.
- **Final barajı:** final notu 50'nin altındaysa ortalamaya bakılmaksızın harf notu FF olur.
- Bu soruda kullanılacak ölçek: 90–100 AA · 85–89,99 BA · 80–84,99 BB · 75–79,99 CB · 70–74,99 CC · 60–69,99 DC · 50–59,99 DD · 50 altı FF.
- Notlar sayı değilse ya da 0–100 aralığı dışındaysa uyarı verilsin.

Kodunuzu yazdıktan sonra aşağıdaki test tablosunu doldurunuz.

Vize	Final	Ortalama	Harf
40	70		
90	45		
85	95		
80	87		
70	86		
2	82		

Son satırı yuvarlama yapmadan (`Math.Round` kullanmadan) da deneyiniz. Ne fark ediyor?

Soru 3.6 HATA AVI Faiz büyümüyor ◆◆◆

Bir banka uygulamasında anapara (textBox1), yıllık faiz yüzdesi (textBox2) ve yıl sayısı (textBox3) girilmekte, her yılın sonundaki bakiye listBox1'e yazılmaktadır. Kullanıcı 10000, 40 ve 3 girdiğinde listede üç satırın da 10000 olduğu görülmüştür.

```
1  private void button1_Click(object sender, EventArgs e)
2  {
3      double bakiye = Convert.ToDouble(textBox1.Text);   // 10000
4      int faiz = Convert.ToInt32(textBox2.Text);         // 40
5      int yil = Convert.ToInt32(textBox3.Text);          // 3
6
7      for (int i = 1; i <= yil; i++)
8      {
9          bakiye = bakiye * (1 + faiz / 100);
10         listBox1.Items.Add(i + ". yıl: " + bakiye);
11     }
12 }
```

a. Bakiyenin hiç artmamasının nedenini 9. satır üzerinden açıklayınız.

b. Kodu düzeltiniz. Düzeltilmiş kodda her yılın bakiyesi 2 basamağa yuvarlanarak yazılsın. Döngü bittikten sonra aynı sonuç bileşik faiz formülüyle (anapara × (1 + faiz/100)^{yıl}) `Math.Pow` kullanılarak hesaplanıp label1'e yazılsın.

c. Düzeltilmiş program için listede ve label1'de görünecek değerleri yazınız.

3.10 Cevap Anahtarı

Soru 3.1

	Çıktı	Neden?
a	**Toplam: 23**	İşlem soldan sağa yapılır. Önce `"Toplam: " + 2` metin olur, sonra 3 bu metnin sonuna eklenir.
b	**5 adet**	Önce `2 + 3` iki sayı olarak toplanır ve 5 olur, sonra metinle birleşir.
c	**3**	`a / b` iki int'in bölümüdür ve 3 olur. Sonuç double'a atanırken zaten 3'tür.
d	**128**	İki string birleştirilir, toplama yapılmaz.
e	**2 / 4**	Bankacı yuvarlaması: tam ortadaki değerler çift sayıya yuvarlanır.
f	**18**	1–10 arasında 3'e tam bölünenler: 3 + 6 + 9.
g	**54**	Türkçe ayarda nokta binlik ayırıcıdır. `"5.4"` metni 54 olarak okunur.
h	**False**	0,1 ve 0,2 ikili sistemde tam gösterilemez. Toplamları 0,30000000000000004'tür. Ondalıklı sayılar `==` ile karşılaştırılmamalıdır.
i	**-10 ve -11**	Ceiling sayı doğrusunda yukarıya (−10), Floor aşağıya (−11) yuvarlar.

Denetim: Tüm çıktılar Türkçe bölge ayarlı bir .NET ortamında çalıştırılarak alınmıştır.

Soru 3.2

Tur	c (başta)	c <= a ?	b (tur sonunda)	c (tur sonunda)
1	1	doğru	1	3
2	3	doğru	4	5
3	5	doğru	9	7
4	7	**yanlış**, döngü biter	9	—

a) label1: **b = 9**. Kod 1'den 5'e kadar tek sayıları toplar (1 + 3 + 5).

b) i = 1 → `"1 "`, i = 2 → `"1 12 "`, i = 3 → `"1 12 123 "`, i = 4 → `"1 12 123 1234 "`. label1'de **1 12 123 1234** yazar (sonda bir boşluk vardır). İç döngü her dış turda 1'den i'ye kadar sayar. `s += j` sayıyı metnin sonuna ekler.

Soru 3.3

textBox1	textBox2	Programın gerçek davranışı
10	20	**Ortalama: 1010**
7	8	**Ortalama: 74**
4,5	3	**FormatException**, program hata verir
(boş)	3	**FormatException**, program hata verir

b) (1) **İşlem önceliği / birleştirme:** `/` işlemi `+`'dan önce yapılır, yani yalnızca sayi2 ikiye bölünür. Ardından `"Ortalama: " + sayi1` metin olduğu için kalan değer toplanmaz, metnin sonuna eklenir: "Ortalama: " + "10" + "10". (2) **Tamsayı bölmesi:** int / int, 7,5 yerine 7 verir (8 / 2 = 4 olduğu için ikinci satırda 7 ve 4 yan yana yazılır). (3) **Giriş denetimi yok:** `Convert.ToInt32` ondalıklı ya da boş girişte hata verir.

c)

```
private void button1_Click(object sender, EventArgs e)
{
    double sayi1, sayi2;
    if (!double.TryParse(textBox1.Text, out sayi1) || !double.TryParse(textBox2.Text, out sayi2))
    {
        label1.Text = "Lütfen iki geçerli sayı giriniz.";
        return;
    }
    double ortalama = (sayi1 + sayi2) / 2;
    label1.Text = "Ortalama: " + ortalama;
}
```

Parantez `(sayi1 + sayi2)` toplamanın önce ve **sayısal** olarak yapılmasını sağlar. double kullanımı ondalığı korur. TryParse ise hatalı girişi yakalar.

Soru 3.4

```
private void button1_Click(object sender, EventArgs e)
{
    double sayi1, sayi2, sonuc;
    if (!double.TryParse(textBox1.Text, out sayi1) || !double.TryParse(textBox2.Text, out sayi2))
```

```
    {
        MessageBox.Show("Lütfen iki geçerli sayı giriniz.");
        return;
    }

    if (radioButton1.Checked)
        sonuc = sayi1 + sayi2;
    else if (radioButton2.Checked)
        sonuc = sayi1 - sayi2;
    else if (radioButton3.Checked)
        sonuc = sayi1 * sayi2;
    else if (radioButton4.Checked)
    {
        if (sayi2 == 0)
        {
            MessageBox.Show("Sıfıra bölme yapılamaz.");
            return;
        }
        sonuc = sayi1 / sayi2;
    }
    else
    {
        MessageBox.Show("Lütfen bir işlem seçiniz.");
        return;
    }

    MessageBox.Show("Sonuç: " + sonuc);   // tek bir MessageBox yeterli
}
```

Ek soru: Program **kapanmaz**. double türünde sıfıra bölme hata fırlatmaz, sonuç **sonsuz** (∞) olur ve mesaj kutusunda sonsuz simgesi (∞) ya da "Infinity" gibi bir ifade görünür (gösterim .NET sürümüne göre değişir). 0/0 ise NaN verir. **int** türünde sıfıra bölme ise **DivideByZeroException** hatası oluşturur ve program durur. Her iki durumda da kullanıcıyı önceden uyarmak doğru yaklaşımdır.

Soru 3.5

```
private void button1_Click(object sender, EventArgs e)
{
    double vize, final;
    if (!double.TryParse(textBox1.Text, out vize) || !double.TryParse(textBox2.Text, out final))
    {
        MessageBox.Show("Notları sayı olarak giriniz.");
        return;
    }
    if (vize < 0 || vize > 100 || final < 0 || final > 100)
    {
        MessageBox.Show("Notlar 0 ile 100 arasında olmalıdır.");
        return;
    }

    double ortalama = Math.Round(vize * 0.4 + final * 0.6, 2);
    string harf;

    if (final < 50)
        harf = "FF";                // final barajı: ortalamaya bakılmaz
    else if (ortalama >= 90)
        harf = "AA";
    else if (ortalama >= 85)
        harf = "BA";
    else if (ortalama >= 80)
```

```
        harf = "BB";
    else if (ortalama >= 75)
        harf = "CB";
    else if (ortalama >= 70)
        harf = "CC";
    else if (ortalama >= 60)
        harf = "DC";
    else if (ortalama >= 50)
        harf = "DD";
    else
        harf = "FF";

    label1.Text = "Ortalama: " + ortalama + "   Harf notu: " + harf;
}
```

Vize	Final	Ortalama	Harf
40	70	58	DD
90	45	63	FF (final barajı)
85	95	91	AA
80	87	84,2	BB
70	86	79,6	CB
2	82	50	DD

Yuvarlamasız deneme: 2 × 0,4 + 82 × 0,6 işlemi double ile **49,99999999999999** sonucunu verir. Matematikte sonuç tam 50 olduğu hâlde, kayan nokta hatası yüzünden program öğrenciye **FF** verir. `Math.Round(…, 2)` bu küçük hatayı temizler. Not, para gibi kesinlik gerektiren hesaplarda ya yuvarlama yapılmalı ya da `decimal` tipi kullanılmalıdır.

Denetim: Tablo değerleri program çalıştırılarak elde edilmiştir. Vize 0–100 ve final 50–100 aralığındaki tüm tamsayı çiftleri tarandığında, yuvarlama yapılmazsa yanlış harf notu veren durum bu örnektir.

Soru 3.6

a) `faiz` int, `100` de int olduğu için `faiz / 100` bir **tamsayı bölmesidir**: 40 / 100 = **0**. Böylece çarpan `(1 + 0) = 1` olur ve bakiye hiç değişmez.

b) faiz double yapılır (ya da `faiz / 100.0` yazılır):

```
private void button1_Click(object sender, EventArgs e)
{
    double bakiye = Convert.ToDouble(textBox1.Text);
    double faiz = Convert.ToDouble(textBox2.Text);
    int yil = Convert.ToInt32(textBox3.Text);
    double baslangic = bakiye;

    listBox1.Items.Clear();
    for (int i = 1; i <= yil; i++)
    {
        bakiye = bakiye * (1 + faiz / 100);
        listBox1.Items.Add(i + ". yıl: " + Math.Round(bakiye, 2));
    }
```

```
    double formul = baslangic * Math.Pow(1 + faiz / 100, yil);
    label1.Text = "Formül ile: " + Math.Round(formul, 2);
}
```

c) Liste: **1. yıl: 14000 · 2. yıl: 19600 · 3. yıl: 27440**. label1: **Formül ile: 27440**. Yuvarlama yapılmasaydı formülün sonucu bazı .NET sürümlerinde 27439,999999999993 olarak görünürdü. Bu da yine kayan nokta hatasından kaynaklanır.

BÖLÜM

4

ComboBox ve ListBox ile Liste İşlemleri



BU BÖLÜMÜN SONUNDA

- Items koleksiyonunu ve sıfırdan başlayan indis mantığını açıklayabilecek,
- Add, AddRange ve döngü ile listelere veri ekleyebilecek,
- Clear, Remove ve RemoveAt ile veri silebilecek, SelectedIndex = −1 durumunu kontrol edebilecek,
- Silme sonrasında indislerin nasıl kaydığını izleyebilecek,
- SelectedIndexChanged olayıyla seçime tepki veren uygulamalar yazabileceksiniz.

Şehir listeleri, personel kayıtları, ürün sepetleri... Pek çok uygulamanın merkezinde bir liste vardır. Bu bölümde ComboBox ve ListBox kontrollerine veri eklemeyi, seçili veriyi bulmayı ve silmeyi öğreniyoruz.

4.1 Items Koleksiyonu ve İndis Mantığı

ComboBox ve ListBox'taki elemanlar **Items** adlı bir koleksiyonda tutulur. Koleksiyon bir dizi gibi davranır: her elemanın bir **indisi** (sıra numarası) vardır ve indisler **0'dan başlar**. Beş elemanlı bir listede indisler 0–4 arasındadır, eleman sayısı ise `Items.Count` ile öğrenilir.

0	1	2	3
Elazığ	Malatya	Ankara	İstanbul

Şekil 4.1 *Dört elemanlı bir liste. Count = 4, son elemanın indisi Count − 1 = 3.*

Üye	Görevi	Örnek
`Items.Add(x)`	Listenin sonuna bir eleman ekler	`listBox1.Items.Add("Van");`
`Items.AddRange(dizi)`	Bir dizinin tüm elemanlarını döngüye gerek kalmadan ekler	`comboBox1.Items.AddRange(ilce);`
`Items.Clear()`	Tüm elemanları siler	`comboBox1.Items.Clear();`
`Items.Remove(x)`	Değeri x olan **ilk** elemanı siler (yoksa bir şey yapmaz)	`listBox1.Items.Remove("Van");`
`Items.RemoveAt(i)`	i indisli elemanı siler	`listBox1.Items.RemoveAt(0);`
`Items.Count`	Eleman sayısı	`label1.Text = listBox1.Items.Count + " kayıt";`
`SelectedIndex`	Seçili elemanın indisi; **seçim yoksa −1**	`int i = listBox1.SelectedIndex;`
`SelectedItem`	Seçili elemanın kendisi; seçim yoksa boş (null)	`listBox1.SelectedItem.ToString()`

DİKKAT

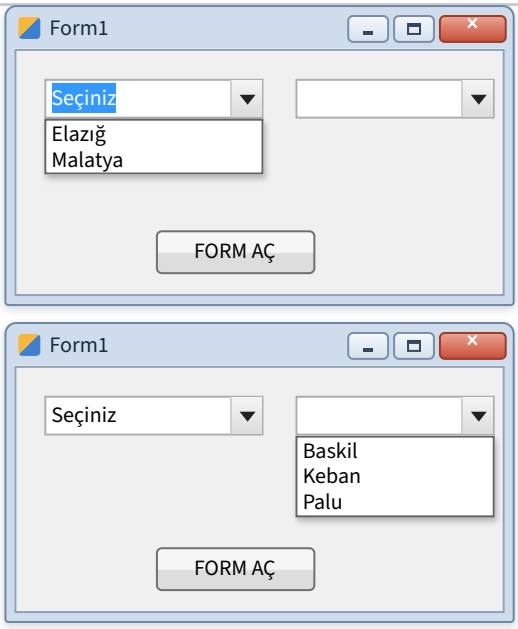
`RemoveAt` metoduna geçersiz bir indis verilirse (negatif ya da `Count` 'a eşit veya büyük) program **ArgumentOutOfRangeException** hatası verir. En sık görülen iki durum şunlardır: kullanıcı hiçbir şey seçmeden "Seçili Sil"e basar (`RemoveAt(-1)`) ya da boş listede "Sil"e basar (`RemoveAt(0)`). Silmeden önce `SelectedIndex != -1` ve `Items.Count > 0` kontrolleri yapılmalıdır.

4.2 ComboBox'a Veri Ekleme

Listeler genellikle form açılırken, yani **Form1_Load** olayında doldurulur. Üç yol vardır. **1. yol:** `Add` ile elemanlar tek tek eklenir. **2. yol:** önce bir dizi oluşturulur, sonra `AddRange` ile dizinin tüm elemanları döngüye gerek kalmadan eklenir.

```
private void Form1_Load(object sender, EventArgs
e)
{
    // 1. yol: Add ile tek tek ekleme
    comboBox1.Text = "Seçiniz";
    comboBox1.Items.Add("Elazığ");
    comboBox1.Items.Add("Malatya");

    // 2. yol: dizi oluşturulur, AddRange ile
topluca eklenir
    string[] ilce = { "Baskil", "Keban", "Palu"
};
    comboBox2.Items.AddRange(ilce);
}
```



NOT

Sunumdaki örnekte ilçe dizisinde *harput* de yer almaktadır. Harput, Elazığ'ın bir ilçesi değil, merkeze bağlı tarihî bir yerleşim yeridir. Bu nedenle örnekte Elazığ'ın ilçelerinden **Palu** kullanılmıştır.

3. yol: dizinin elemanları bir döngüyle tek tek `Add` edilir. Döngü sınırı olarak elle 4 yazmak yerine `sehir.Length` kullanmak, diziye eleman eklendiğinde kodu değiştirme ihtiyacını ortadan kaldırır.

4.3 ComboBox'tan Veri Silme

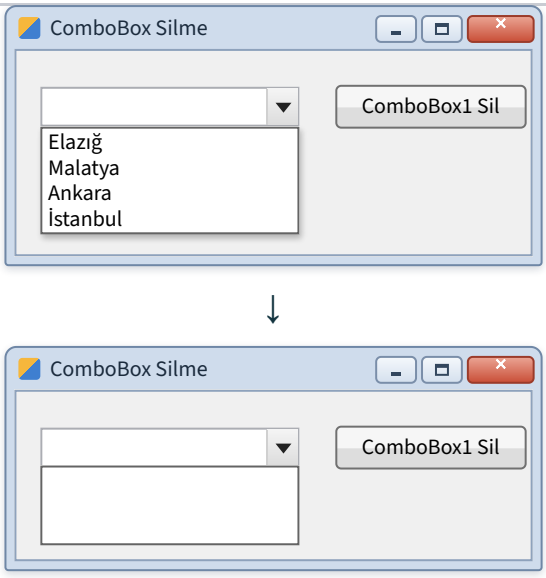
Tümünü silme: `Items.Clear()` komutu ile listedeki tüm veriler silinir.



Aradan bir eleman çekildiğinde arkadakiler bir sıra öne kayar

```
private void Form1_Load(object sender, EventArgs
e)
{
    // 3. yol: dizi elemanları döngüyle tek tek
eklenir
    string[] sehir = { "Elazığ", "Malatya",
"Ankara", "İstanbul" };
    for (int i = 0; i < sehir.Length; i++)
    {
        comboBox1.Items.Add(sehir[i]);
    }
}

private void button1_Click(object sender,
EventArgs e)
{
    comboBox1.Items.Clear();   // tüm elemanları
siler
}
```



Seçili veriyi silme: Önce ComboBox'ta seçili olan verinin indisi `SelectedIndex` ile öğrenilir, sonra `RemoveAt` ile o indisteki veri silinir. Seçim yapılmamışsa indis –1 olacağından silmeden önce kontrol edilir.

```
private void button1_Click(object sender,
EventArgs e)
{
    int indis = comboBox1.SelectedIndex;   //
seçili elemanın indisini alır
    if (indis != -1)
    {
        comboBox1.Items.RemoveAt(indis);   // o
indisteki elemanı siler
    }
    else
    {
        MessageBox.Show("Önce silinecek elemanı
seçiniz.");
    }
}
```



Seçili “Ankara” verisi (indis 2) silindikten sonra kendisinden sonraki “İstanbul” bir sıra yukarı kayar ve indisi 3'ten 2'ye düşer. Bu **kayma** davranışı, silme işlemlerinin döngüyle yapıldığı durumlarda çok önemlidir:

Silmeden önce

0	1	2	3
Elazığ	Malatya	~~Ankara~~	İstanbul

RemoveAt(2) sonrası

0	1	2
Elazığ	Malatya	İstanbul

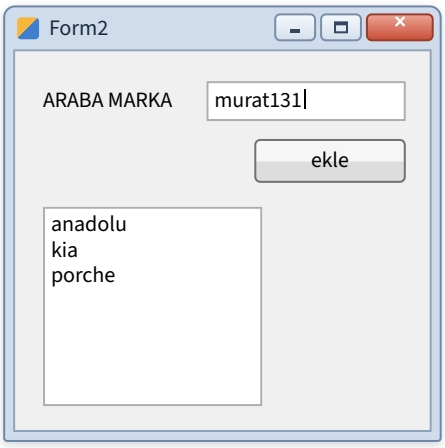
Şekil 4.2 *Bir eleman silindiğinde sonraki tüm elemanların indisi bir azalır.*

4.4 ListBox'a Veri Ekleme

ARABA MARKA yazan bir Label, bir TextBox, **ekle** butonu ve bir ListBox içeren form oluşturulur. Butona her tıklandığında kutudaki marka listeye eklenir.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    string marka;
    marka = textBox1.Text;
    listBox1.Items.Add(marka);
}
```



GELİŞTİRİLMİŞ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    string marka = textBox1.Text.Trim();   //
baştaki/sondaki boşluklar atılır
    if (marka == "")
    {
        MessageBox.Show("Marka adı boş bırakılamaz.");
        return;
    }
    listBox1.Items.Add(marka);
    textBox1.Clear();                      // kutu
temizlenir
    textBox1.Focus();                      // imleç
yeniden kutuya gelir
}
```

Sunumdaki çözüm, kutu boşken bile listeye boş bir satır ekler ve her eklemeden sonra kutunun elle temizlenmesini gerektirir. Geliştirilmiş çözümde `Trim()` baştaki ve sondaki boşlukları atar, boş giriş engellenir, `Clear()` kutuyu boşaltır ve `Focus()` imleci yeniden kutuya getirir. Kullanıcı Tab ya da fareye ihtiyaç duymadan art arda veri girebilir.

4.5 ListBox'tan Veri Silme

Forma **sil** ve **seçili sil** butonları eklenir. **sil** butonu listenin başından itibaren sırayla siler (`RemoveAt(0)`), **seçili sil** ise kullanıcının seçtiği satırı siler.

```
private void button2_Click(object sender, EventArgs e)
// "sil"
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);         // listenin
başından itibaren siler
    }
}

private void button3_Click(object sender, EventArgs e)
// "seçili sil"
{
    if (listBox1.SelectedIndex != -1)
    {

listBox1.Items.RemoveAt(listBox1.SelectedIndex);   //
seçilen satırı siler
    }
}
```



İPUCU

Listede çoklu seçim açıksa (`SelectionMode = MultiExtended`) `SelectedIndex` yalnızca seçilenlerden birini verir. Tüm seçili satırlar `SelectedItems` koleksiyonunda bulunur.

4.6 Sütunlu Kayıt Listesi

Dışarıdan ad, soyad, doğum yeri ve meslek bilgileri girilir. **KAYIT EKLE** butonuna tıklanınca bu bilgiler ListBox'a tek satır olarak eklenir, **KAYIT SİL** butonuna tıklanınca eklenmiş veriler satır satır silinir. Değerler arasına konan `"\t"` (sekme karakteri), ListBox'ta sütunların hizalı görünmesini sağlar. ListBox'ın `UseTabStops` özelliği varsayılan olarak açıktır.

```
private void button1_Click(object sender,
EventArgs e)   // KAYIT EKLE
{
    listBox1.Items.Add(textBox1.Text + "\t" +
textBox2.Text + "\t"
                    + textBox3.Text + "\t" +
textBox4.Text);
}

private void button2_Click(object sender,
EventArgs e)   // KAYIT SİL
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);
    }
}
```



Sunumdaki çözümde `RemoveAt(0)` satırı, liste boşken basıldığında hata vereceği için `Count > 0` kontrolü eklenmiştir.

4.7 SelectedIndexChanged Olayı ve Personel Uygulaması

ComboBox'ta seçili eleman her değiştiğinde **SelectedIndexChanged** olayı tetiklenir. Bu olay sayesinde kullanıcı bir butona basmadan, yalnızca seçim yaparak işlem başlatabilir. Aşağıdaki uygulamada AD ve SOYAD kutularına girilen veriler EKLE butonuyla PERSONELLER listesine aktarılır. Listedeki veriler seçili olarak ya da sırayla silinebilir. ARKA PLAN RENGİ listesinden bir renk seçildiğinde formun arka planı o renge dönüşür.



Şekil 4.3 *Personel uygulaması: listeden mavi seçildiğinde form arka planı maviye döner.*

PERSONEL UYGULAMASININ TÜM KODU

```
private void Form1_Load(object sender, EventArgs e)
{
    string[] dizi = { "kırmızı", "beyaz", "mavi" };
    comboBox1.Items.AddRange(dizi);
}

private void button1_Click(object sender, EventArgs e)   // EKLE
{
    listBox1.Items.Add(textBox1.Text + "\t" + textBox2.Text);
}

private void button2_Click(object sender, EventArgs e)   // SEÇİLİ SİL
{
    int sec = listBox1.SelectedIndex;          // seçili indis
    if (sec != -1)
    {
        listBox1.Items.RemoveAt(sec);          // o indisi siler
    }
}

private void button3_Click(object sender, EventArgs e)   // SIRAYLA SİL
{
    if (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);
    }
}
```

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // this: kodun içinde bulunduğu Form1 nesnesini ifade eder
    if (comboBox1.SelectedIndex == 0)
    {
        this.BackColor = Color.Red;
    }
    else if (comboBox1.SelectedIndex == 1)
    {
        this.BackColor = Color.White;
    }
    else if (comboBox1.SelectedIndex == 2)
    {
        this.BackColor = Color.Blue;
    }
}
```

NOT

Sunumdaki çözümde *beyaz* seçeneği (indis 1) için `Color.Yellow` yazılmıştır. Bu nedenle beyaz seçildiğinde form sarı olur. Doğrusu `Color.White`'tır. Ayrıca üç ayrı `if` yerine `else if` zinciri kullanılmıştır. Bir seçim yapıldığında diğer şartların boşuna kontrol edilmesine gerek yoktur.

NESNE TABANLI BAKIŞ

Renk uygulamasında kullanılan `this` anahtar kelimesi, kodun içinde bulunduğu **nesneyi**, yani çalışan Form1 penceresini ifade eder. `this.BackColor = Color.Red;` ifadesi "bu formun arka plan rengini kırmızı yap" demektir. Aynı Form1 sınıfından iki pencere açılsaydı, her pencerenin `this`'i kendisi olurdu ve biri değiştiğinde diğeri etkilenmezdi. Her nesnenin özellik değerleri kendine aittir.

ÖZET

- ComboBox ve ListBox elemanları **Items** koleksiyonunda tutulur. İndisler **0**'dan başlar, son indis `Count - 1`'dir.
- Ekleme `Add`, `AddRange` ya da döngüyle yapılır. Silme işlemleri `Clear` (tümü), `Remove` (değere göre ilk eşleşen) ve `RemoveAt` (indise göre) ile yapılır.
- Seçim yoksa `SelectedIndex = -1` olur. Geçersiz indisle `RemoveAt` çağrılırsa **ArgumentOutOfRangeException** oluşur.
- Bir eleman silindiğinde ondan sonraki elemanların indisi bir azalır. Döngüyle silerken bu kayma hesaba katılmalıdır.
- `"\t"` ile ListBox satırlarında sütun hizası sağlanır.
- Seçim değişince **SelectedIndexChanged** olayı tetiklenir. `this`, kodun ait olduğu form nesnesidir.

4.8 Kendini Test Et

Soru 4.1 İZLEME TABLOSU Liste nasıl değişir?

◆◆◇

listBox1 başlangıçta aşağıdaki gibidir. Bir butona tıklandığında altındaki yedi satır sırasıyla çalıştırılmaktadır.

0	1	2	3	4
Ankara	Bursa	Elazığ	İzmir	Malatya

Başlangıç durumu

```
listBox1.Items.RemoveAt(1);
listBox1.Items.Add("Van");
listBox1.Items.Remove("İzmir");
listBox1.Items.RemoveAt(listBox1.Items.Count - 1);
listBox1.Items.Add("Bursa");
listBox1.Items.RemoveAt(0);
listBox1.SelectedIndex = 1;
```

Her satırdan sonra listenin içeriğini (indisleriyle birlikte) ve `Items.Count` değerini yazınız. 7. satırdan sonra hangi şehir seçili (mavi) görünür?

Satır	Liste içeriği (indis: değer)	Count
1		
2		
3		
4		
5		
6		

Soru 4.2 HATA AVI Hepsini silmeyen “Tümünü Sil”

◆◆◆

Bir öğrenci listeyi döngüyle boşaltmak için iki farklı kod yazmıştır.

KOD A

```
private void button1_Click(object sender, EventArgs e)   // Tümünü Sil (A)
{
    for (int i = 0; i < listBox1.Items.Count; i++)
    {
        listBox1.Items.RemoveAt(i);
    }
}
```

KOD B

```
private void button2_Click(object sender, EventArgs e)   // Tümünü Sil (B)
{
    for (int i = 0; i < 5; i++)
    {
        listBox1.Items.RemoveAt(i);
    }
}
```

a. Listede **A, B, C, D, E, F** (6 eleman) varken Kod A çalıştırılırsa listede hangi elemanlar kalır? Döngünün her turunda `i`, `Items.Count` ve silinen elemanı göstererek açıklayınız.

b. Listede **A, B, C, D, E** (5 eleman) varken Kod B çalıştırılırsa ne olur? Program hangi turda, hangi hatayla durur? O anda listede neler vardır?

c. Listeyi döngüyle eksiksiz boşaltan **iki farklı** doğru çözüm yazınız ve her birinin neden doğru çalıştığını kayma kavramıyla açıklayınız.

Soru 4.3 HATA AVI Renkler karıştı

◆◆◇

```
private void Form1_Load(object sender, EventArgs e)
{
    comboBox1.Items.AddRange(new string[] { "kırmızı", "beyaz", "mavi" });
}

private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    if (comboBox1.SelectedIndex == 1)
        this.BackColor = Color.Red;
    if (comboBox1.SelectedIndex == 2)
        this.BackColor = Color.White;
    if (comboBox1.Text == "Mavi")
        this.BackColor = Color.Blue;
}

private void button1_Click(object sender, EventArgs e)   // listBox1: Seçili Sil
{
    listBox1.Items.RemoveAt(listBox1.SelectedIndex);
}
```

a. Kullanıcı sırasıyla *kırmızı*, *beyaz* ve *mavi*'yi seçtiğinde formun rengi her seferinde ne olur? Tablo hâlinde gösteriniz.

b. `comboBox1.Text == "Mavi"` şartı neden hiçbir zaman doğru olmaz?

c. Kullanıcı listBox1'de hiçbir satır seçmeden button1'e basarsa ne olur? Neden?

d. Olay metodunu `switch` yapısıyla, button1 metodunu ise güvenli hâle getirerek yeniden yazınız.

Soru 4.4 KOD YAZMA Alışveriş sepeti

◆◆◆

Bir kırtasiye için sepet uygulaması yazınız. Ürünler ve fiyatları: **Defter 45,50 TL · Kalem 12 TL · Silgi 7,25 TL**.

- comboBox1 ürünleri listeler ve yalnızca listeden seçim yapılabilir. numericUpDown1 adet seçtirir (1–20).
- **Sepete Ekle** (button1): ürün seçilmemişse uyarı verir. Seçilmişse listBox1'e *ürün · adet · tutar* satırını ekler ve label1'de güncel **sepet tutarını** gösterir.
- **Sepeti Boşalt** (button2): listeyi ve tutarı sıfırlar.

a. Programı yazınız. Ürün adlarını ve fiyatları iki ayrı dizide tutunuz, doğru fiyata seçili indis üzerinden ulaşınız.

b. Sırasıyla 2 Defter, 4 Silgi ve 3 Kalem eklendiğinde label1'de ne yazar?

c. Sepet tutarını tutan değişken neden olay metodunun içinde değil, sınıf düzeyinde tanımlanmalıdır? Metodun içine taşınırsa (b) şıkkındaki işlemlerden sonra label1'de ne görünür?

Soru 4.5 SENARYO ANALİZİ Birbirine bağlı iki liste ◆◆◇

comboBox1'den il seçildiğinde comboBox2'ye o ilin ilçeleri gelmektedir.

```
1  private void Form1_Load(object sender, EventArgs e)
2  {
3      comboBox1.Items.AddRange(new string[] { "Elazığ", "Malatya" });
4  }
5
6  private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
7  {
8      comboBox2.Items.Clear();
9      comboBox2.Text = "";
10     if (comboBox1.SelectedIndex == 0)
11         comboBox2.Items.AddRange(new string[] { "Baskil", "Keban", "Palu" });
12     else if (comboBox1.SelectedIndex == 1)
13         comboBox2.Items.AddRange(new string[] { "Battalgazi", "Darende", "Yeşilyurt" });
14 }
```

a. 8. satır (`comboBox2.Items.Clear();`) silinirse ve kullanıcı sırasıyla Elazığ, Malatya, Elazığ seçerse comboBox2'de kaç eleman olur? Elemanları sırasıyla yazınız.

b. 9. satırın görevi nedir? Hangi durumda gerekli olduğunu bir örnekle açıklayınız.

c. İl ve ilçe sayısı arttıkça `if – else if` zinciri uzar. Bu uzamanın önüne geçmek için verileri nasıl tutardınız? Kısaca açıklayınız (kod yazmanız gerekmez).

4.9 Cevap Anahtarı

Soru 4.1

Satır	Liste içeriği	Count
1	0: Ankara · 1: Elazığ · 2: İzmir · 3: Malatya	4
2	0: Ankara · 1: Elazığ · 2: İzmir · 3: Malatya · 4: Van	5
3	0: Ankara · 1: Elazığ · 2: Malatya · 3: Van	4
4	0: Ankara · 1: Elazığ · 2: Malatya	3

Satır	Liste içeriği	Count
5	0: Ankara · 1: Elazığ · 2: Malatya · 3: Bursa	4
6	0: Elazığ · 1: Malatya · 2: Bursa	3

7. satırdan sonra **Malatya** seçili görünür (indis 1). Dikkat: 1. satırda Bursa silinince Elazığ'ın indisi 2'den 1'e düşer. 4. satırda `Count - 1` o anki son indistir (3) ve **Van**'ı siler. 5. satırda Bursa listeye yeniden, bu kez sona eklenir.

Denetim: Satırlar gerçek bir ListBox üzerinde çalıştırılarak doğrulanmıştır.

Soru 4.2

a) Listede **B, D, F** kalır.

Tur	i	Count (tur başı)	Silinen	Liste (tur sonu)
1	0	6	A	B C D E F
2	1	5	C	B D E F
3	2	4	E	B D F
—	3	3	şart yanlış	B D F

Her silmede sonraki elemanlar bir indis geri kayar, i ise bir ileri gider. Bu yüzden her turda bir eleman atlanır.

b) i = 0'da A silinir (B C D E), i = 1'de C silinir (B D E), i = 2'de E silinir (B D). i = 3 turunda listede yalnızca 2 eleman vardır (geçerli indisler 0 ve 1), `RemoveAt(3)` çağrılır ve program **ArgumentOutOfRangeException** ile durur. O anda listede **B, D** vardır. Sabit sınır (5), listenin küçüldüğünü hesaba katmaz.

c)

```
// 1. çözüm: her seferinde ilk elemanı sil
private void button1_Click(object sender, EventArgs e)
{
    while (listBox1.Items.Count > 0)
    {
        listBox1.Items.RemoveAt(0);
    }
}

// 2. çözüm: sondan başa doğru sil (kayma, silinmemiş elemanları etkilemez)
private void button2_Click(object sender, EventArgs e)
{
    for (int i = listBox1.Items.Count - 1; i >= 0; i--)
    {
        listBox1.Items.RemoveAt(i);
    }
}

// En kısa yol: listBox1.Items.Clear();
```

1. çözümde her zaman 0. eleman silinir ve kayan eleman bir sonraki turda yine 0. konumdadır. Döngü liste boşalana kadar sürer. 2. çözümde sondan silindiği için kayma, henüz silinmemiş (daha küçük indisli) elemanları etkilemez.

Denetim: Her iki hatalı kod ve iki çözüm gerçek bir ListBox üzerinde çalıştırılmıştır.

Soru 4.3

Seçilen	SelectedIndex	Formun rengi
kırmızı	0	**Değişmez** (hiçbir şart 0'ı kontrol etmiyor)
beyaz	1	**Kırmızı**
mavi	2	**Beyaz**

b) String karşılaştırması büyük-küçük harfe duyarlıdır. Listedeki eleman *mavi*, karşılaştırılan metin *Mavi*'dir ve bu ikisi eşit değildir.

c) Seçim yokken `SelectedIndex = -1` olur ve `RemoveAt(-1)` çağrısı **ArgumentOutOfRangeException** hatası verir.

d)

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    switch (comboBox1.SelectedIndex)
    {
        case 0: this.BackColor = Color.Red; break;
        case 1: this.BackColor = Color.White; break;
        case 2: this.BackColor = Color.Blue; break;
    }
}

private void button1_Click(object sender, EventArgs e)
{
    if (listBox1.SelectedIndex != -1)
        listBox1.Items.RemoveAt(listBox1.SelectedIndex);
    else
        MessageBox.Show("Önce bir satır seçiniz.");
}
```

`switch`, aynı değişkenin farklı değerlerine göre dallanmanın okunaklı bir yoludur. Her `case` bir `break` ile biter.

Soru 4.4

a)

```
using System;
using System.Windows.Forms;

namespace Sepet
{
    public partial class Form1 : Form
    {
        string[] urunler = { "Defter", "Kalem", "Silgi" };
        double[] fiyatlar = { 45.5, 12, 7.25 };
        double sepetTutari = 0;   // alan (field): tüm olay metotları ortak kullanır

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
```

```
            comboBox1.Items.AddRange(urunler);
            comboBox1.DropDownStyle = ComboBoxStyle.DropDownList;
            numericUpDown1.Minimum = 1;
            numericUpDown1.Maximum = 20;
        }

        private void button1_Click(object sender, EventArgs e)   // Sepete Ekle
        {
            int i = comboBox1.SelectedIndex;
            if (i == -1)
            {
                MessageBox.Show("Lütfen bir ürün seçiniz.");
                return;
            }
            int adet = Convert.ToInt32(numericUpDown1.Value);
            double tutar = fiyatlar[i] * adet;
            listBox1.Items.Add(urunler[i] + "\t" + adet + " adet\t" + tutar + " TL");
            sepetTutari += tutar;
            label1.Text = "Sepet tutarı: " + sepetTutari + " TL";
        }

        private void button2_Click(object sender, EventArgs e)   // Sepeti Boşalt
        {
            listBox1.Items.Clear();
            sepetTutari = 0;
            label1.Text = "Sepet tutarı: 0 TL";
        }
    }
}
```

b) 2 × 45,50 = 91, 4 × 7,25 = 29, 3 × 12 = 36. label1: **Sepet tutarı: 156 TL**.

c) Yerel değişkenler, tanımlandıkları metot her çağrıldığında yeniden oluşturulur ve metot bitince yok olur. `double sepetTutari = 0;` satırı button1_Click'in içinde olsaydı her tıklamada tutar 0'dan başlardı ve yalnızca son eklenen ürünün tutarı görünürdü: **Sepet tutarı: 36 TL**. Sınıf düzeyindeki değişkene **alan (field)** denir. Alan, form nesnesi yaşadığı sürece değerini korur ve sınıfın tüm metotları (button2_Click dahil) onu kullanabilir.

Soru 4.5

a) 9 eleman: Baskil, Keban, Palu, Battalgazi, Darende, Yeşilyurt, Baskil, Keban, Palu. Liste temizlenmediği için her seçimde yeni ilçeler eskilerin sonuna eklenir.

b) 9. satır, kutuda önceki ilden kalan metni temizler. Elazığ'da *Keban* seçilip il Malatya yapılırsa, bu satır olmadan kutuda Keban yazmaya devam edebilir.

c) Her ilin ilçeleri, il listesiyle aynı sırada bir dizi dizisinde (`string[][]`) tutulur. Seçilen il indisiyle doğru ilçe dizisine ulaşılır: `comboBox2.Items.AddRange(ilceler[comboBox1.SelectedIndex]);`

BÖLÜM

5

Çoklu Form Uygulamaları



BU BÖLÜMÜN SONUNDA

- Projeye yeni form ekleyip formun başlangıç konumunu ayarlayabilecek,
- Bir formdan başka bir formu nesne oluşturarak açabilecek,
- Show ile ShowDialog arasındaki farkı kod akışı üzerinden açıklayabilecek,
- Hide, Close ve Application.Exit komutlarının etkisini ayırt edebilecek,
- Gizlenen ana form yüzünden kapanmayan program sorununu çözebilecek,
- Formlar arasında kurucu metot ile veri aktarabileceksiniz.

Gerçek uygulamalar tek pencereden oluşmaz: önce bir giriş ekranı, ardından ana menü, oradan da ayar ya da rapor pencereleri açılır. Bu bölümde bir formdan başka bir formu açmayı, gizlemeyi ve kapatmayı öğreniyoruz.

5.1 Projeye Yeni Form Ekleme

1. **Project** menüsünden **Add Windows Form...** alt menüsüne tıklanır.
2. Açılan *Add New Item* penceresinde **Windows Form** seçilir. **Name** alanında varsayılan ad `Form2.cs`'dir. İstenirse değiştirilir.
3. **Add** butonuna basılır. Solution Explorer'da `Form2.cs` görünür ve yeni formun tasarım penceresi açılır.

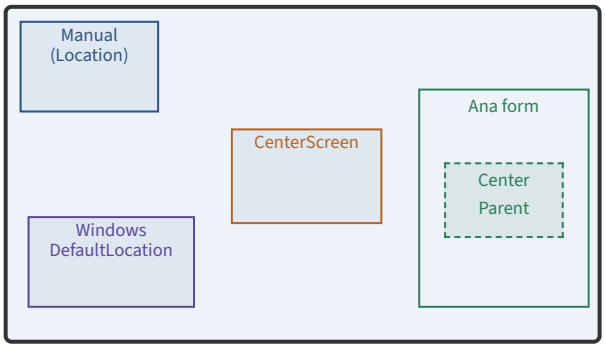
> **NOT**
> Yeni Visual Studio sürümlerinde aynı komut **Project → Add Form (Windows Forms)...** ya da Solution Explorer'da projeye sağ tıklayıp **Add → Form (Windows Forms)...** şeklinde bulunur.

5.2 Formun Başlangıç Konumu: StartPosition

Hazırlanan formun ekrana ilk geldiği konum ayarlanabilir. Bunun için form seçildikten sonra Properties penceresindeki **StartPosition** özelliğinden ilgili değer seçilir. Kod ile de ayarlanabilir:

```
this.StartPosition = FormStartPosition.CenterScreen;
```

Değer	Form ekrana nasıl gelir?
`Manual`	Location (konum) özelliğinde belirtilen koordinatlarda
`CenterScreen`	Ekranın tam ortasında
`WindowsDefaultLocation`	Windows'un belirlediği konumda (yeni projelerde varsayılan)
`WindowsDefaultBounds`	Konumu ve boyutu Windows'un belirlediği şekilde
`CenterParent`	Kendisini açan ana formun ortasında. En güvenilir biçimde ShowDialog ile açılan formlarda çalışır.

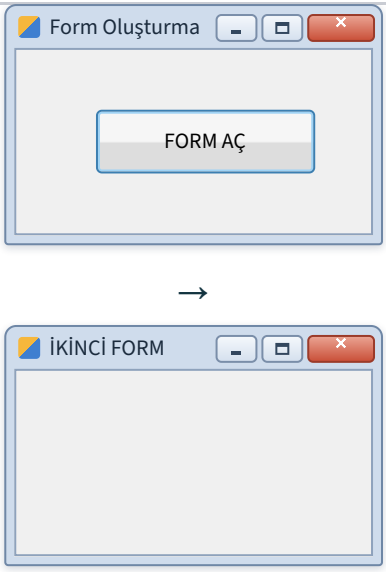


Şekil 5.1 *StartPosition değerleri*

5.3 Form İçinden Yeni Form Açma

Form1'e **FORM AÇ** butonu eklenir ve butona çift tıklanarak Click metoduna aşağıdaki kodlar yazılır:

```
private void button1_Click(object sender,
EventArgs e)
{
    Form2 yeni = new Form2();   // Form2
sınıfından yeni bir nesne oluşturulur
    yeni.Show();                // Form2
ekranda gösterilir
}
```



NESNE TABANLI BAKIŞ

Projeye eklenen `Form2` bir **sınıftır**. Tasarım penceresinde gördüğümüz yalnızca kalıptır. `new Form2()` bu kalıptan bellekte gerçek bir pencere **nesnesi** üretir ve bu nesneye `yeni` adını verir. Nesne oluşturmak, onu ekranda göstermek demek değildir. `yeni.Show()` metodu çağrılana kadar pencere görünmez. Butona her tıklandığında `new` yeniden çalışır ve her seferinde **yeni bir Form2 nesnesi**, yani yeni bir pencere oluşur.

5.4 Show ve ShowDialog

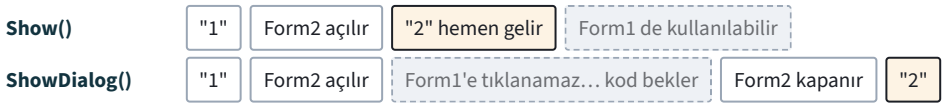
Form iki farklı yöntemle gösterilebilir:



ShowDialog: önceki işlem bitmeden sıradaki başlayamaz

```
private void button1_Click(object sender, EventArgs e)
{
    Form2 yeni = new Form2();
    yeni.ShowDialog();
    // ShowDialog: açılan pencere kapatılmadan diğer pencerelere ERİŞİLEMEZ
    // Show      : açılan pencere kapatılmadan da diğer pencerelere ERİŞİLİR
}
```

	Show()	ShowDialog()
Pencere türü	Modsuz (modeless)	Kalıcı / modal
Açan forma erişim	Açılan pencere kapanmadan da diğer pencerelere erişilir	Açılan pencere kapatılmadan diğer pencerelere **erişilemez**
Sonraki kod satırları	Hemen çalışır	Açılan pencere kapanana kadar **bekler**
Tipik kullanım	Ana menüden bağımsız pencereler	Ayarlar, onay, bilgi girişi pencereleri



Şekil 5.2 `MessageBox.Show("1"); f.Show…(); MessageBox.Show("2");` *kodunun iki yöntemdeki akışı*

5.5 Ana Formu Gizleme ve Formu Kapatma

Yeni form açıldığında ana formun ekranda kalması istenmiyorsa ana form gizlenir. `this` kodun içinde bulunduğu formu (burada Form1'i) ifade eder:

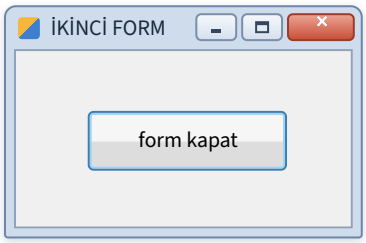
FORM1: FORM2'Yİ AÇIP KENDİNİ GİZLER

```
private void button1_Click(object sender, EventArgs e)
{
    this.Hide();               // şu anki formu (Form1) gizler
    Form2 yeni = new Form2();  // yeni formun nesnesi oluşturulur
    yeni.Show();
}
```

Form2'ye bir **form kapat** butonu konursa, Click metoduna yazılan `this.Close()` yalnızca kullanıldığı formu kapatır:

FORM2: KENDİNİ KAPATIR

```
private void button1_Click(object sender, EventArgs e)
// Form2 üzerindeki "form kapat"
{
    this.Close();   // yalnızca içinde bulunulan formu
(Form2) kapatır
}
```



Komut	Ne yapar?	Uygulamaya etkisi
`this.Hide()`	Formu gizler (`Visible = false`). Nesne bellekte durur, daha sonra `Show()` ile yeniden gösterilebilir.	Uygulama çalışmaya devam eder
`this.Close()`	Formu kapatır.	Kapatılan form **ana form** (Program.cs'de `Application.Run` 'a verilen) ise tüm uygulama kapanır. Değilse yalnızca o form kapanır.
`Application.Exit()`	Uygulamanın tüm formlarını kapatır.	Program sonlanır

DİKKAT

Kapanmayan program sorunu. Form1 `Hide()` ile gizlenip Form2 açıldıktan sonra kullanıcı Form2'yi sağ üstteki X ile kapatırsa ekranda hiç pencere kalmaz, ancak **program çalışmaya devam eder**. Uygulamanın ana formu Form1'dir ve Form1 kapatılmamış, yalnızca gizlenmiştir. Program Görev Yöneticisi'nde açık kalır, Visual Studio da hâlâ "çalışıyor" durumunda bekler. Çözüm, Form2 kapandığında ne olacağını açıkça belirtmektir:

```
private void button1_Click(object sender, EventArgs e)
{
    Form2 yeni = new Form2();
    yeni.FormClosed += Yeni_FormClosed;   // Form2 kapanınca çalışacak metodu bağla
    yeni.Show();
    this.Hide();
}

private void Yeni_FormClosed(object sender, FormClosedEventArgs e)
{
    this.Show();              // Form2 kapanınca ana form yeniden görünür
    // Uygulamanın tamamen kapanması istenirse: Application.Exit();
}
```

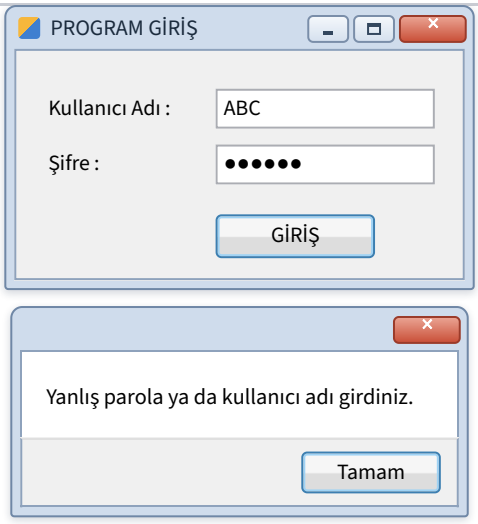
Buradaki `yeni.FormClosed += Yeni_FormClosed;` satırı, 1. bölümde Designer dosyasında gördüğümüz olay bağlama işleminin aynısıdır. Bu kez bağlantıyı biz kod ile kurarız. Form2 hangi yolla kapanırsa kapansın (X, Close, Alt+F4) FormClosed olayı tetiklenir ve Form1 yeniden görünür.

5.6 Uygulama: Kullanıcı Giriş Formu

Form1 üzerine kullanıcı adı ve şifre kutuları ile GİRİŞ butonu eklenir. Kullanıcı bilgileri doğruysa Form2 açılsın, değilse hata uyarı mesajı verilsin.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs
e)
{
    string kadi = "a", ksifre = "a";
    string ad, sifre;
    ad = textBox1.Text;
    sifre = textBox2.Text;
    if (ad == kadi && sifre == ksifre)
    {
        Form2 ali = new Form2();
        ali.Show();
        this.Hide();
    }
    else
    {
        MessageBox.Show("Yanlış parola ya da
kullanıcı adı girdiniz.");
    }
}
```



Uygulamayı geliştirmek için şu noktalara dikkat edilmelidir:

- Şifre kutusunun `PasswordChar` özelliği `*` yapılmalıdır. Şifre ekranda açık görünmemelidir.
- Giriş başarılı olunca Form1 gizlendiği için, Form2 kapatıldığında uygulamanın da kapanması sağlanmalıdır (Bölüm 5.5).
- Gerçek uygulamalarda kullanıcı adı ve şifre koda yazılmaz, veri tabanında (şifre özetlenerek) saklanır. Buradaki sabit değerler yalnızca öğretim amaçlıdır.
- Sınırsız deneme hakkı tanımak güvenlik açığıdır. Deneme sayısını sınırlamak bu bölümün Kendini Test Et kısmında Soru 5.4 olarak size bırakılmıştır.

5.7 Nesne Tabanlı Bakış: Formlar Arası Veri Aktarma

Giriş yapan kullanıcının adını Form2'de göstermek istediğimizi düşünelim. Form2, Form1'deki `textBox1`'e doğrudan erişemez, çünkü o kontrol Form1 nesnesinin **private** (özel) bir parçasıdır. Doğru yol, veriyi Form2 nesnesi oluşturulurken ona **göndermektir**. Bunun için Form2'ye **parametreli bir kurucu metot** yazılır:

FORM2.CS

```
using System;
using System.Windows.Forms;

namespace Giris
{
    public partial class Form2 : Form
    {
        public Form2(string kullaniciAdi)       // parametreli kurucu metot
        {
            InitializeComponent();
```

```
            this.Text = "Hoş geldin, " + kullaniciAdi;
        }
    }
}
```

FORM1.CS

```
using System;
using System.Windows.Forms;

namespace Giris
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Form2 anaSayfa = new Form2(textBox1.Text);   // veri kurucuya gönderilir
            anaSayfa.Show();
        }
    }
}
```

`new Form2(textBox1.Text)` ifadesi, Form2 nesnesi oluşturulurken kurucu metodu çalıştırır ve kutudaki metni `kullaniciAdi` parametresine aktarır. Kurucu metot bu değeri formun başlığına yazar. Bu yöntemde her form kendi kontrollerinden sorumludur ve veri formlar arasında açıkça tanımlanmış bir kapıdan, kurucu metotdan geçer. Nesne tabanlı programlamada buna **kapsülleme** denir.

DİKKAT

Form2'ye parametreli kurucu yazıldıktan sonra parametresiz `new Form2()` çağrısı derleme hatası verir. Projede iki kullanım birden gerekiyorsa Form2'de her iki kurucu da yazılır.

ÖZET

- Yeni form **Project → Add Windows Form** ile eklenir. Başlangıç konumu **StartPosition** ile belirlenir.
- Form açmak iki adımdır: `Form2 f = new Form2();` nesneyi oluşturur, `f.Show();` ya da `f.ShowDialog();` onu gösterir. Her `new` yeni bir pencere demektir.
- **Show** açan forma erişime izin verir ve kod beklemeden devam eder. **ShowDialog** ise açılan pencere kapanana kadar hem kullanıcıyı hem kodu bekletir.
- `Hide` gizler, `Close` kapatır, `Application.Exit` tüm uygulamayı sonlandırır. Ana form kapatılırsa program biter.
- Ana form gizlendiyse ikinci formun **FormClosed** olayında ana form yeniden gösterilmeli ya da uygulama kapatılmalıdır.
- Formlar arası veri, parametreli kurucu metot ile aktarılır.

5.8 Kendini Test Et

Soru 5.1 ÇIKTI TAHMİNİ Mesajların sırası

◆◆◇

Form2'nin Load olayına aşağıdaki metot bağlanmıştır:

```
// Form2.cs içinde
private void Form2_Load(object sender, EventArgs e)
{
    MessageBox.Show("Form2 yüklendi");
}
```

Form1'deki butona iki farklı kod yazılmıştır:

KOD A

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("1");
    Form2 f = new Form2();
    f.Show();
    MessageBox.Show("2");
}
```

KOD B

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("1");
    Form2 f = new Form2();
    f.ShowDialog();
    MessageBox.Show("2");
}
```

Her iki kod için, butona tıklandıktan sonra ekrana gelen üç mesajın (1, *Form2 yüklendi*, 2) sırasını yazınız. 2 mesajının Form2 kapatılmadan **önce** mi **sonra** mı geldiğini belirtiniz. Ayrıca Kod B'de, Form2 açıkken kullanıcının Form1'deki butona tekrar tıklayıp tıklayamayacağını açıklayınız.

Soru 5.2 SENARYO ANALİZİ Üç tık, kaç pencere?

◆◆◆

Bölüm 5.3'teki kod (`new Form2()` + `Show()`) kullanılmaktadır.

a. Kullanıcı FORM AÇ butonuna art arda 3 kez tıklarsa ekranda kaç tane Form2 penceresi olur? Neden?

b. Aynı kodda `Show()` yerine `ShowDialog()` yazılsaydı aynı anda en fazla kaç Form2 açık olabilirdi? Neden?

c. Show kullanmaya devam ederek, Form2 zaten açıksa yenisini açmayıp mevcut pencereyi öne getiren, kapatılmışsa yeniden açan kodu yazınız. İpucu: Form2 değişkenini sınıf düzeyinde tanımlayınız. Bir formun kapatılıp kapatılmadığı `IsDisposed` özelliğinden anlaşılır.

Soru 5.3 SENARYO ANALİZİ Program neden kapanmıyor? ◆◆◇

Bir öğrenci giriş ekranını Bölüm 5.6'daki gibi yazmıştır. Doğru şifreyle giriş yapıp Form2 açıldıktan sonra Form2'yi X ile kapatmıştır. Ekranda pencere kalmamış ama Visual Studio'da program "çalışıyor" görünmeye devam etmiştir.

a. Durumu Program.cs'deki `Application.Run(new Form1());` satırıyla ilişkilendirerek açıklayınız.

b. Sorunu yalnızca **Form2.cs** dosyasında değişiklik yaparak çözünüz.

c. Sorunu yalnızca **Form1.cs** dosyasında değişiklik yaparak, bu kez Form2 kapanınca giriş ekranı yeniden görünecek şekilde çözünüz.

d. Öğrenci `this.Hide()` yerine `this.Close()` yazsaydı ne olurdu?

Soru 5.4 KOD YAZMA Üç deneme hakkı ve karşılama ◆◆◆

Bölüm 5.6'daki giriş uygulamasını aşağıdaki şartlarla yeniden yazınız. Kullanıcı adı *admin*, şifre 1234 olsun.

- Şifre kutusunda karakterler görünmesin. label3'te kalan deneme hakkı (*Kalan hak:* 3) gösterilsin.
- Her hatalı girişte hak bir azalsın ve kullanıcıya kalan hak bildirilsin. Hak 0'a düşünce uygun bir mesajla program tamamen kapansın.
- Doğru girişte giriş formu gizlensin ve Form2 açılsın. Form2'nin başlığında *Hoş geldin, admin* yazsın. Kullanıcı adı Form2'ye kurucu metot ile aktarılsın.
- Form2 kapatıldığında program tamamen kapansın.

Ek soru: Kalan hak sayısını tutan değişkeni button1_Click metodunun içinde tanımlasaydınız program nasıl davranırdı?

Soru 5.5 KOD OKUMA Hide mı, Close mu, Exit mi? ◆◆◇

Ana formu Form1 olan bir uygulamada aşağıdaki durumların her birinde ne olacağını açıklayınız. Uygulama açık kalır mı, hangi pencereler görünür?

No	Durum
1	Form1'de `this.Hide();` çalışıyor. Başka açık form yok.
2	Form1, Form2'yi `Show()` ile açmış. Form2'de `this.Close();` çalışıyor.
3	Form1, Form2'yi `Show()` ile açmış. Form1'de `this.Close();` çalışıyor.
4	Form1 gizli, Form2 açık. Form2'de `Application.Exit();` çalışıyor.
5	Form1, Form2'yi `ShowDialog()` ile açmış. Form2'de `this.Close();` çalışıyor.

5.9 Cevap Anahtarı

Soru 5.1

Kod A: 1 → *Form2 yüklendi* → 2. 2 mesajı Form2 kapatılmadan **önce**, Form2 açıldıktan hemen sonra gelir. `Show()` pencereyi gösterir ve kod beklemeden bir sonraki satıra geçer.

Kod B: 1 → *Form2 yüklendi* → (kullanıcı Form2'yi kapatır) → 2. `ShowDialog()` satırında kod durur ve Form2 kapanana kadar sonraki satır çalışmaz. Her iki kodda da Load olayı, form gösterilirken (Show/ShowDialog çağrısı sırasında) tetiklenir.

Kod B'de Form2 açıkken Form1'deki butona **tıklanamaz**. Modal pencere kapanmadan sahibi olan forma erişilemez.

Denetim: Mesaj sırası, iki yöntem de çalıştırılarak doğrulanmıştır.

Soru 5.2

a) 3 pencere. Her tıklamada `new Form2()` yeniden çalışır ve bağımsız, yeni bir nesne oluşturur. `Show()` Form1'i kilitlemediği için kullanıcı tekrar tıklayabilir.

b) 1 pencere. ShowDialog açıkken Form1'e tıklanamaz, butona ikinci kez basılamaz.

c)

```
Form2 ikinci;   // sınıf düzeyinde alan: tıklamalar arasında değerini korur

private void button1_Click(object sender, EventArgs e)
{
    if (ikinci == null || ikinci.IsDisposed)
    {
        ikinci = new Form2();
        ikinci.Show();
    }
    else
    {
        ikinci.Activate();      // zaten açıksa yenisini açma, öne getir
    }
}
```

`ikinci` değişkeni sınıf düzeyinde (alan) olduğu için tıklamalar arasında açılan pencereyi hatırlar. İlk tıklamada değeri `null` 'dır ve form oluşturulur. Form kapatıldıktan sonra nesne **dispose** edilir (kaynakları serbest bırakılır) ve `IsDisposed` true olur. Kapatılmış bir form yeniden `Show()` edilemeyeceği için yeni nesne oluşturulur. Form açıksa `Activate()` onu öne getirir.

Soru 5.3

a) `Application.Run(new Form1())` , uygulamanın ömrünü Form1'e bağlar: program, Form1 **kapanana** kadar sürer. `Hide()` Form1'i kapatmaz, yalnızca görünmez yapar. Form2'nin kapanması da ana formu etkilemez. Böylece ekranda pencere olmadığı hâlde mesaj döngüsü çalışmaya devam eder.

b) Form2'nin **FormClosed** olayına (Properties → Events) bağlanan metotta:

```
// Form2.cs içinde: FormClosed olayına bağlanan metot
private void Form2_FormClosed(object sender, FormClosedEventArgs e)
{
    Application.Exit();      // gizli Form1 dahil tüm uygulamayı kapatır
}
```

c) Form2'yi açarken FormClosed olayına Form1'deki bir metodu bağlayıp orada `this.Show()` çağırmak gerekir (Bölüm 5.5'teki "Kapanmayan program sorunu" kutusundaki kod).

d) Form1 ana form olduğu için `this.Close()` **tüm uygulamayı** kapatır, az önce açılan Form2 de kapanır. Kullanıcı Form2'yi bir an görür ya da hiç göremez.

Soru 5.4

FORM1.CS

```
using System;
using System.Windows.Forms;

namespace GirisSistemi
{
    public partial class Form1 : Form
    {
        int kalanHak = 3;   // alan: her tıklamada sıfırlanmaması için sınıf düzeyinde

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            textBox2.PasswordChar = '*';
            label3.Text = "Kalan hak: " + kalanHak;
        }

        private void button1_Click(object sender, EventArgs e)
        {
            if (textBox1.Text == "admin" && textBox2.Text == "1234")
            {
                Form2 anaSayfa = new Form2(textBox1.Text);
                anaSayfa.FormClosed += AnaSayfa_FormClosed;
                anaSayfa.Show();
                this.Hide();
            }
            else
            {
                kalanHak--;
                label3.Text = "Kalan hak: " + kalanHak;
                if (kalanHak == 0)
                {
                    MessageBox.Show("3 kez hatalı giriş yapıldı. Program kapatılıyor.");
                    Application.Exit();
                }
                else
                {
                    MessageBox.Show("Hatalı giriş! Kalan hak: " + kalanHak);
                }
            }
        }

        private void AnaSayfa_FormClosed(object sender, FormClosedEventArgs e)
        {
            Application.Exit();
        }
    }
}
```

FORM2.CS

```
using System;
using System.Windows.Forms;

namespace GirisSistemi
{
    public partial class Form2 : Form
    {
        public Form2(string kullaniciAdi)
        {
            InitializeComponent();
            this.Text = "Hoş geldin, " + kullaniciAdi;
        }
    }
}
```

Ek soru: `int kalanHak = 3;` metodun içinde olsaydı her tıklamada yeniden 3 olarak oluşturulurdu. Her hatalı girişte 2'ye düşer ama bir sonraki tıklamada tekrar 3'ten başlardı. Hak hiçbir zaman 0'a ulaşmaz, kullanıcı sınırsız deneme yapabilirdi. Değerini tıklamalar arasında koruması gereken bilgi, sınıf düzeyinde bir **alan** olarak tutulmalıdır.

Soru 5.5

No	Sonuç
1	Ekranda hiç pencere kalmaz ama uygulama **açık kalır**. Ana form kapanmamıştır, yalnızca gizlenmiştir.
2	Yalnızca Form2 kapanır. Form1 görünür kalır, uygulama devam eder.
3	Ana form kapandığı için **uygulama sonlanır** ve Form2 de kapanır.
4	Tüm formlar kapanır, **uygulama sonlanır**. Gizli Form1 de buna dahildir.
5	Form2 kapanır ve `ShowDialog()` satırı geri döner. Form1 yeniden kullanılabilir hâle gelir, Form1'de ShowDialog'dan sonraki satırlar çalışmaya başlar. Uygulama devam eder.

BÖLÜM

6

Veri Arama, Rastgele Sayılar ve Metin İşlemleri



BU BÖLÜMÜN SONUNDA

- Items.Contains, Items.Count ve FindStringExact ile listede veri arayabilecek,
- Bir listeye aynı verinin ikinci kez eklenmesini engelleyebilecek,
- Math.Max ve Math.Min ile en büyük ve en küçük değeri bulabilecek,
- Random sınıfıyla benzersiz rastgele sayılar üretebilecek,
- Checked özelliğini CheckBox ve RadioButton ile kullanabilecek, kontrolleri dizide toplayabilecek,
- ToUpper ve ToLower ile metnin harflerini dönüştürüp kelimelerin ilk harfini büyütebileceksiniz.

Bir listeye aynı kaydın iki kez girmesi, aranan kaydın bulunamaması, metinlerin karışık harflerle yazılması... Gerçek uygulamalarda en sık karşılaşılan küçük sorunlar bunlardır. Bu bölümde listede veri aramayı, rastgele sayı üretmeyi ve metinlerle çalışmayı öğreniyoruz.

6.1 Listede Veri Arama: Items.Contains

`Items.Contains(veri)` metodu, verinin daha önce listeye (ListBox, ComboBox) eklenip eklenmediğini arar. Dönüş değeri **bool** türündedir: veri listede varsa `true`, yoksa `false` döner.

```
string data = textBox1.Text;
bool durum = listBox1.Items.Contains(data);   // varsa true, yoksa false
```

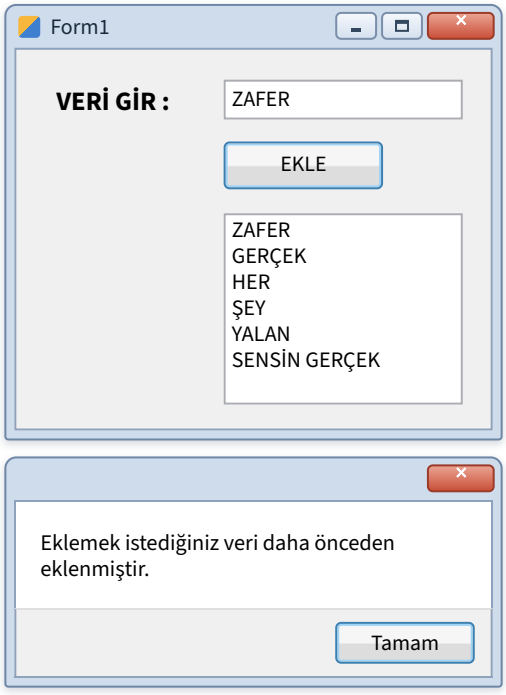
DİKKAT

Contains **birebir eşleşme** arar ve **büyük-küçük harfe duyarlıdır**. Listede *ayşe* varken `Contains("Ayşe")` ve sonunda boşluk bulunan `Contains("ayşe ")` çağrıları `false` döner. Bu yüzden aynı kişi farklı yazılışlarla listeye tekrar eklenebilir. Sunumdaki `Bool` yazımı da hatalıdır: C#'ta tür adı küçük harfle `bool` yazılır.

Uygulama — Aynı veriyi ikinci kez ekletmeyen liste

Forma VERİ GİR yazan bir Label, bir TextBox, EKLE butonu ve bir ListBox eklenir. EKLE'ye basıldığında yazılan veri listede zaten varsa kullanıcı uyarılır, yoksa veri listeye eklenir.

```
private void button1_Click(object sender, EventArgs
e)
{
    string veri = textBox1.Text;
    // Contains: aynı verinin daha önce eklenip
eklenmediğini verir
    if (listBox1.Items.Contains(veri) == true)
    {
        MessageBox.Show("Eklemek istediğiniz veri
daha önceden eklenmiştir.");
        textBox1.Text = "";
    }
    else
    {
        listBox1.Items.Add(textBox1.Text);
        textBox1.Text = "";
    }
}
```



`Contains(veri) == true` karşılaştırması gereksizdir. Metot zaten bool döndürdüğü için `if (listBox1.Items.Contains(veri))` yazmak yeterlidir. Değilini sormak için `!` kullanılır: `if (!listBox1.Items.Contains(veri))`.

6.2 Items.Count ve Döngüyle Arama

`Items.Count`, listeye eklenmiş toplam eleman sayısını verir. Dönüş değeri **int** türündedir. Bir listenin elemanları üzerinde döngü kurarken sınır olarak bu özellik kullanılır.

```
int toplam;
toplam = listBox1.Items.Count;   // listedeki eleman sayısı (int)
```

Contains metodu yokmuş gibi, aynı kontrol bir döngüyle de yapılabilir. Sunumda verilen ikinci çözüm aşağıdadır:

SUNUMDAKİ ÇÖZÜM 2

```
private void button1_Click(object sender, EventArgs e)
{
    string veri = textBox1.Text;

    for (int i = 0; i < listBox1.Items.Count; i++)
    {
        string data = Convert.ToString(listBox1.Items[i]);
        if (veri == data)
        {
            MessageBox.Show("Ekleyeceğiniz veri daha önce eklenmiştir.");
            listBox1.Items.RemoveAt(i);
        }
    }
    listBox1.Items.Add(veri);
}
```

DİKKAT

Bu çözüm görevini yerine getirmez. Veri listede bulunduğunda kullanıcı uyarılır, ama ardından **eski kayıt silinir** ve döngü bittikten sonra veri **yine eklenir**. Listede A, B, C varken B girilirse uyarı çıkar ve liste **A, C, B** olur. Tekrarlanan kayıt engellenmemiş, yalnızca listenin sonuna taşınmıştır. Döngü içinde silme yapıldığı için 4. bölümdeki **kayma** sorunu da vardır: listede A, B, B, C varken B girilirse ikinci B atlanır ve liste **A, B, C, B** olur.

Doğru yaklaşım, döngüde yalnızca **aramak**, sonucu bir **bayrak** değişkeninde tutmak ve kararı döngüden sonra vermektir:

BAYRAK DEĞİŞKENLİ DOĞRU ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    string veri = textBox1.Text;
    bool varMi = false;                      // bayrak (flag) değişkeni

    for (int i = 0; i < listBox1.Items.Count; i++)
    {
        if (Convert.ToString(listBox1.Items[i]) == veri)
        {
            varMi = true;
            break;                           // bulundu, aramaya gerek kalmadı
        }
```

```
    }

    if (varMi)
        MessageBox.Show("Ekleyeceğiniz veri daha önce eklenmiştir.");
    else
        listBox1.Items.Add(veri);
}
```

6.3 Verinin Yerini Bulma: FindStringExact

`FindStringExact(veri)` metodu, aranan veri listede varsa onun **indis numarasını** getirir. Dönüş değeri **int** türündedir. Sonuç 0 veya 0'dan büyükse veri listede vardır; **–1** dönüyorsa veri yoktur. Contains yalnızca "var mı?" sorusunu cevaplar, FindStringExact ise "nerede?" sorusunu da cevaplar.

```
string veri = textBox1.Text;
int indis;
indis = listBox1.FindStringExact(veri);   // bulunursa 0 veya büyük, yoksa -1
```

	Items.Contains(x)	FindStringExact(x)	FindString(x)
Döndürdüğü	bool (true / false)	int (indis ya da –1)	int (indis ya da –1)
Eşleşme türü	Birebir aynı	Metnin tamamı aynı	x ile **başlayan** ilk eleman
Büyük-küçük harf	**Duyarlı**	**Duyarsız**	Duyarsız
Hangi kontrolde?	Items koleksiyonu olan her kontrol	ListBox, ComboBox	ListBox, ComboBox

Uygulama — Benzersiz ekleme ve bulma

Forma EKLE ve BUL butonları eklenir. EKLE veriyi benzersiz olarak ekler; BUL ise veriyi listede bulup **seçili** gösterir, yoksa mesajla uyarır. Bulunan indisin `SelectedIndex` özelliğine atanması, o satırın mavi (seçili) görünmesini sağlar.

```
private void button1_Click(object sender, EventArgs e)
// EKLE
{
    string veri = textBox1.Text;
    // Contains ile veri daha önce eklenmişse tekrar
eklenmesini önlüyoruz
    if (listBox1.Items.Contains(veri) == false)
        listBox1.Items.Add(veri);
    else
        MessageBox.Show("Daha önce eklediniz!");
}

private void button2_Click(object sender, EventArgs e)
// BUL
{
    string veri = textBox1.Text;
    // FindStringExact: veri listede varsa indis
numarasını getirir
    int sonuc = listBox1.FindStringExact(veri);
    if (sonuc >= 0)
        listBox1.SelectedIndex = sonuc;       //
bulunan satır seçili gösterilir
    else
        MessageBox.Show("Aradığınız bulunamadı.");
}
```



NOT

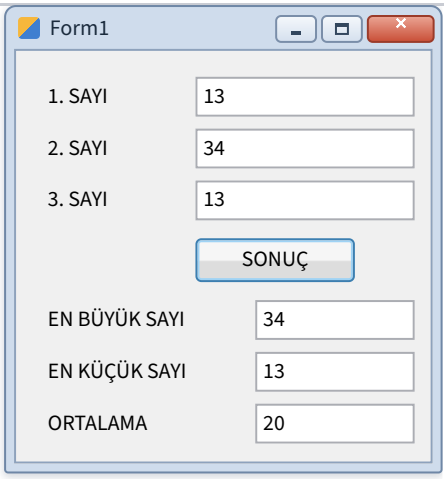
Bu uygulamada iki metot farklı kurala göre çalışır. EKLE büyük-küçük harfe duyarlı Contains kullandığı için listede *ayşe* varken AYŞE eklenebilir. BUL ise duyarsız FindStringExact kullandığı için AYŞE aranınca *ayşe* satırını bulur. Bu fark Kendini Test Et bölümünde ayrıca sorulmuştur.

6.4 Üç Sayının En Büyüğü, En Küçüğü ve Ortalaması

Forma üç sayı kutusu, SONUÇ butonu ve sonuçlar için üç kutu eklenir. `Math.Max(a, b)` iki sayıdan büyüğünü, `Math.Min(a, b)` küçüğünü verir. Üç sayı için metot iki kez kullanılır: önce a ile b karşılaştırılır, sonra kazanan c ile karşılaştırılır.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    int a, b, c, buyuk, kucuk, orta;
    a = Convert.ToInt32(textBox1.Text);   // 1. veriyi
tamsayıya çevir
    b = Convert.ToInt32(textBox2.Text);
    c = Convert.ToInt32(textBox3.Text);
    buyuk = Math.Max(a, b);               // a ve b
arasından büyüğü bulur
    buyuk = Math.Max(c, buyuk);           // c ve buyuk
arasından büyüğü bulur
    kucuk = Math.Min(a, b);               // a ve b
arasından küçüğü bulur
    kucuk = Math.Min(c, kucuk);           // c ve kucuk
arasından küçüğü bulur
    orta = (a + b + c) / 3;               // üç sayının
ortalaması
    textBox4.Text = Convert.ToString(buyuk);
    textBox5.Text = "" + kucuk;
    textBox6.Text = Convert.ToString(orta);
}
```



DİKKAT

Sunumdaki çözüm tamsayılarla çalışır. Ortalama `int` bölmesiyle hesaplandığı için ondalık kısım atılır: 13, 34, 13 için sonuç tam 20 olduğundan sorun görünmez, ama 1, 2, 2 için doğru ortalama 1,67 iken program **1** yazar. Kutulardan birine ondalıklı sayı yazılırsa `Convert.ToInt32` hata verir.

ONDALIKLI SAYILARLA ÇALIŞAN, GİRİŞİ DENETLEYEN ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    double a, b, c;
    if (!double.TryParse(textBox1.Text, out a) || !double.TryParse(textBox2.Text, out b)
        || !double.TryParse(textBox3.Text, out c))
    {
        MessageBox.Show("Lütfen üç geçerli sayı giriniz.");
        return;
    }
    textBox4.Text = Math.Max(Math.Max(a, b), c).ToString();   // iç içe çağrı
    textBox5.Text = Math.Min(Math.Min(a, b), c).ToString();
    textBox6.Text = Math.Round((a + b + c) / 3, 2).ToString();
}
```

`Math.Max(Math.Max(a, b), c)` yazımında içteki çağrının sonucu dıştaki çağrıya parametre olarak verilir. Böylece ara değişken (`buyuk`) kullanmaya gerek kalmaz.

6.5 Rastgele Sayı Üretme: Random Sınıfı

Rastgele sayı üretmek için **Random** sınıfından bir nesne oluşturulur ve nesnenin `Next` metodu çağrılır. `Next(alt, ust)` , **alt dahil, ust hariç** bir tamsayı üretir. 1 ile 49 arasında sayı istendiğinde `Next(1, 50)` yazılmasının nedeni budur.



Random: her çekilişte önceden bilinemeyen bir sayı

```
Random uret = new Random();      // Random sınıfından bir nesne
int zar = uret.Next(1, 7);       // 1, 2, 3, 4, 5 veya 6 (7 dahil değil)
int yuzde = uret.Next(100);      // 0 ile 99 arası
```

NESNE TABANLI BAKIŞ

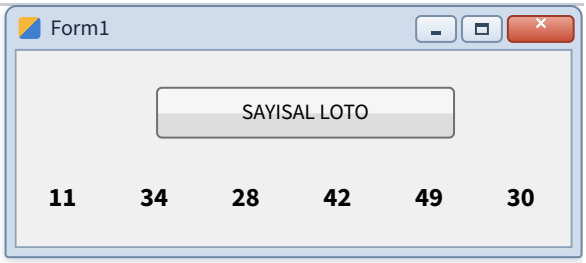
Math sınıfının metotları statiktir ve doğrudan `Math.Max(...)` şeklinde çağrılır. Random ise önce `new Random()` ile **nesnesi oluşturulması** gereken bir sınıftır, çünkü her nesne kendi sayı dizisini üretir ve bu durumu içinde saklar. Aynı Random nesnesinden art arda `Next` çağırmak doğru kullanımdır. Bir döngünün her turunda yeni bir `new Random()` oluşturmak, eski .NET Framework sürümlerinde aynı sayıların tekrar tekrar üretilmesine yol açabilir.

Uygulama — Sayısal loto

SAYISAL LOTO butonuna tıklandığında altı Label'a 1–49 arasında **birbirinden farklı** altı sayı yazılır. Her yeni sayı, kendisinden önce üretilen sayılarla karşılaştırılır. Aynı sayı daha önce çıkmışsa `i--` ile indis bir geri alınır ve `break` ile iç döngüden çıkılır. Dış döngünün `i++` adımı indisi yeniden aynı yere getirdiği için o konum için yeni bir sayı üretilir.

```
private void button1_Click(object sender,
EventArgs e)
{
    int[] dizi = new int[6];
    Random uret = new Random();   // sayı
üretecek Random nesnesi

    for (int i = 0; i < 6; i++)
    {
        dizi[i] = uret.Next(1, 50);          //
1-49 arası (50 dahil değil)
        for (int j = 0; j < i; j++)          //
yeni sayıyı öncekilerle karşılaştır
        {
            if (dizi[i] == dizi[j])
            {
                i--;     // aynı sayı çıktıysa
bu indis yeniden üretilsin
                break;   // iç döngüden çık
            }
        }
    }
    label1.Text = Convert.ToString(dizi[0]);
    label2.Text = Convert.ToString(dizi[1]);
    label3.Text = Convert.ToString(dizi[2]);
    label4.Text = Convert.ToString(dizi[3]);
    label5.Text = Convert.ToString(dizi[4]);
    label6.Text = Convert.ToString(dizi[5]);
}
```



Bu algoritma 20 000 kez çalıştırılarak denendiğinde her seferinde 1–49 aralığında altı farklı sayı üretmiştir. Algoritmanın adım adım nasıl ilerlediği Kendini Test Et bölümünde bir izleme tablosuyla sorulmuştur.

6.6 Checked Özelliği: CheckBox ve RadioButton

Checked özelliği, seçim yapılan kontrollerde (CheckBox, RadioButton) kontrolün seçili olup olmadığını tutar ve `true` ya da `false` değer alır. Hem okunabilir hem de kod ile değiştirilebilir.

```
radioButton1.Checked = true;    // seçili yap
checkBox1.Checked = false;      // seçimi kaldır
```

NOT

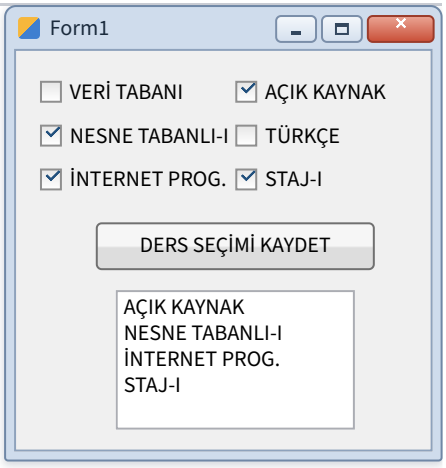
Sunumda bu örnek *radiobutton1.checked = true* şeklinde küçük harflerle yazılmıştır. C# büyük-küçük harfe duyarlı olduğu için bu yazım derlenmez. Doğrusu `radioButton1.Checked` şeklindedir.

Uygulama — Ders seçimi (CheckBox)

Formda altı CheckBox, DERS SEÇİMİ KAYDET butonu ve bir ListBox bulunur. Butona tıklandığında işaretli derslerin adları listeye aktarılır. Liste her tıklamada önce temizlenir; böylece butona birden fazla basıldığında aynı dersler tekrar eklenmez.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    // birden fazla tıklamada liste temiz kalsın
    listBox1.Items.Clear();
    // CheckBox seçili ise (true) listeye ekle, değilse
(false) ekleme
    if (checkBox1.Checked == true) {
listBox1.Items.Add(checkBox1.Text); }
    if (checkBox2.Checked == true) {
listBox1.Items.Add(checkBox2.Text); }
    if (checkBox3.Checked == true) {
listBox1.Items.Add(checkBox3.Text); }
    if (checkBox4.Checked == true) {
listBox1.Items.Add(checkBox4.Text); }
    if (checkBox5.Checked == true) {
listBox1.Items.Add(checkBox5.Text); }
    if (checkBox6.Checked == true) {
listBox1.Items.Add(checkBox6.Text); }
}
```



NESNE TABANLI BAKIŞ

Altı satırın birbirinden tek farkı kontrolün adıdır. Kontroller birer nesne olduğu için diğer nesneler gibi bir **diziye** konabilir. Dizi kurulduktan sonra aynı işlem tek bir döngüyle yapılır. Forma yeni bir ders eklendiğinde diziye bir eleman eklemek yeterlidir.

```
private void button1_Click(object sender, EventArgs e)
{
    // altı nesneyi bir dizide toplayınca aynı işlem döngüyle yapılır
    CheckBox[] dersler = { checkBox1, checkBox2, checkBox3, checkBox4, checkBox5,
checkBox6 };

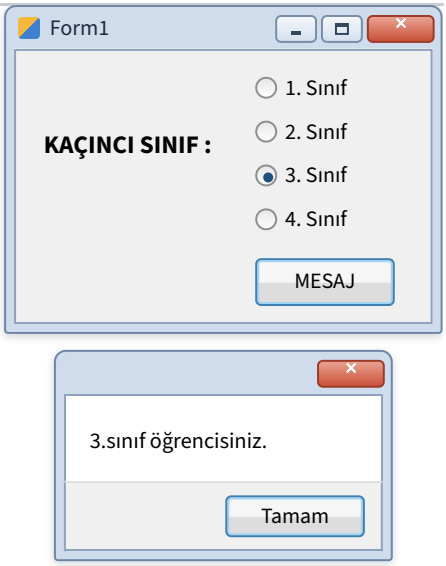
    listBox1.Items.Clear();
    foreach (CheckBox ders in dersler)
    {
        if (ders.Checked)
            listBox1.Items.Add(ders.Text);
    }
}
```

Uygulama — Sınıf seçimi (RadioButton)

KAÇINCI SINIF sorusu için dört RadioButton ve MESAJ butonu eklenir. Butona tıklandığında seçilen sınıf mesaj olarak gösterilir. RadioButton'larda aynı anda yalnızca biri seçili olabildiği için `else if` zinciri kullanılır.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked == true) {
MessageBox.Show("1.sınıf öğrencisiniz."); }
    else if (radioButton2.Checked == true) {
MessageBox.Show("2.sınıf öğrencisiniz."); }
    else if (radioButton3.Checked == true) {
MessageBox.Show("3.sınıf öğrencisiniz."); }
    else { MessageBox.Show("1.sınıf öğrencisiniz."); }
}
```



DİKKAT

Sunumdaki çözümde son `else` bloğu *1.sınıf öğrencisiniz.* mesajını verir. Dördüncü sınıf seçildiğinde ya da hiçbir seçenek seçilmediğinde de bu blok çalışır ve kullanıcıya yanlış sınıf söylenir. Bu hatanın düzeltilmesi Kendini Test Et bölümünde sorulmuştur.

6.7 Harf Dönüştürme: ToUpper ve ToLower

`ToUpper()` bir metnin tüm harflerini büyük, `ToLower()` tüm harflerini küçük harfe çevirir. Her iki metot da **yeni bir metin döndürür**; üzerinde çağrıldığı değişkeni değiştirmez.



Aynı harfin büyük ve küçük biçimleri

```
string ad = "ahmet";
ad.ToUpper();          // ad DEĞİŞMEZ: hâlâ "ahmet"
ad = ad.ToUpper();     // sonuç geri atanınca ad = "AHMET"
```

DİKKAT

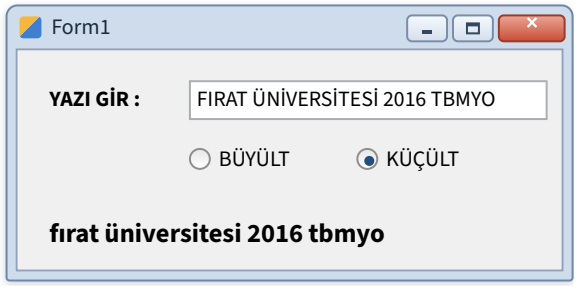
Sunumdaki `ad.toUpper();` örneğinde iki hata vardır. Metodun adı büyük T ile `ToUpper` yazılır. Ayrıca C#'ta string'ler **değiştirilemez (immutable)**: `ad.ToUpper();` satırı büyük harfli yeni bir metin üretir ama bu metin hiçbir yere atanmadığı için kaybolur, `ad` hâlâ *ahmet*'tir. Sonucun kullanılabilmesi için geri atanması gerekir: `ad = ad.ToUpper();`

Uygulama — Yazıyı büyültme ve küçültme

Forma bir TextBox, BÜYÜLT ve KÜÇÜLT radyo butonları ve sonucu gösteren bir Label eklenir. Radyo butonlarından biri seçildiğinde yazı dönüştürülüp Label'a yazılır. Kod, butonun Click olayına değil, radyo butonların **CheckedChanged** olayına yazılmıştır; bu yüzden ayrı bir butona basmaya gerek kalmaz.

```
private void radioButton1_CheckedChanged(object
sender, EventArgs e)
{
    string metin = textBox1.Text;
    if (radioButton1.Checked == true) {
label2.Text = metin.ToUpper(); }
    // ToUpper ile tüm karakterler büyük harfle
yazılır
}

private void radioButton2_CheckedChanged(object
sender, EventArgs e)
{
    string metin = textBox1.Text;
    if (radioButton2.Checked == true) {
label2.Text = metin.ToLower(); }
    // ToLower ile tüm karakterler küçük harfle
yazılır
}
```



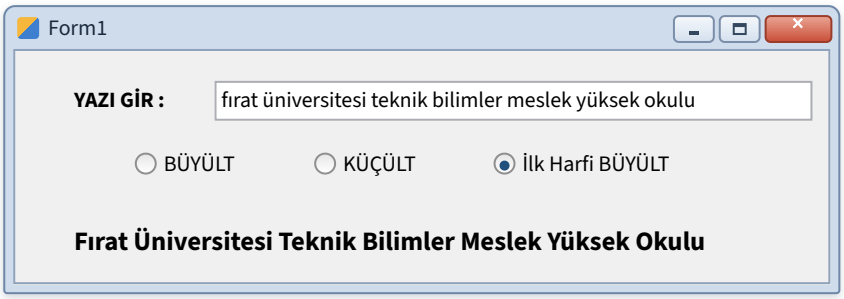
CheckedChanged olayı yalnızca seçim yapıldığında değil, **seçim kaldırıldığında** da tetiklenir. KÜÇÜLT seçildiğinde BÜYÜLT'ün seçimi kalkar ve radioButton1'in olayı da çalışır. Metotlardaki `if (radioButtonX.Checked)` kontrolü, seçimi kalkan butonun Label'ı değiştirmesini engeller.

DİKKAT

Türkçe harf tuzağı. ToUpper ve ToLower bilgisayarın dil ayarına göre çalışır. Türkçe ayarlı bir bilgisayarda *i* harfi **İ**'ye, *I* harfi **ı**'ya dönüşür: `"istanbul".ToUpper()` → *İSTANBUL*, `"ID".ToLower()` → *ıd*. Türkçe metinler için istenen davranış budur. Ancak kullanıcı adı, dosya uzantısı ya da kod gibi dilden bağımsız metinlerde `ToUpperInvariant()` ve `ToLowerInvariant()` kullanılır; bunlar *i*'yi I'ya çevirir.

6.8 Kelimelerin İlk Harfini Büyütme

Önceki forma **İlk Harfi BÜYÜLT** adlı üçüncü bir radyo buton eklenir. Seçildiğinde yazıdaki her kelimenin ilk harfi büyütülerek Label'a yazılır. Bir metnin karakterlerine dizi gibi indisle erişilir: `metin[0]` ilk karakteri, `metin.Length` karakter sayısını verir. Karakterler **char** türündedir ve tek tırnakla yazılır: `' '` bir boşluk karakteridir.



Şekil 6.1 *İlk harfleri büyütülmüş metin*

SUNUMDAKİ ÇÖZÜM

```
private void radioButton3_CheckedChanged(object sender, EventArgs e)
{
    string sonuc = ""; int j = 0;
    string metin = textBox1.Text;   // cümle metin değişkenine atanır
    // cümlenin ilk harfini büyük yaz
    if (metin[0] != ' ') { sonuc = sonuc + "" + metin[0].ToString().ToUpper(); j = 1; }

    for (int i = j; i < metin.Length; i++)
    {   // her boşluğu yakalayıp sonraki harfi büyük yap
        if (metin[i] == ' ') { sonuc = sonuc + " " + metin[i + 1].ToString().ToUpper(); i++; }
        else { sonuc = sonuc + "" + metin[i]; }
    }
    label2.Text = sonuc;
}
```

Algoritma şöyle çalışır: metnin ilk karakteri boşluk değilse büyütülerek sonuca eklenir ve döngü 1. indisten başlar. Döngüde boşluk görüldüğünde boşluk ve **bir sonraki** karakter büyütülerek eklenir, `i++` ile o karakter atlanır. Diğer karakterler olduğu gibi eklenir.

Kutudaki yazı	Sunumdaki çözümün sonucu
fırat üniversitesi teknik…	Fırat Üniversitesi Teknik… (doğru)
ali veli ***(iki boşluk)***	Ali veli — ikinci kelime büyütülmez
FIRAT üni	FIRAT Üni — büyük yazılmış harfler küçültülmez
ali ***(sonda boşluk)***	**IndexOutOfRangeException**: metin[i + 1] metnin dışına taşar
(boş kutu)	**IndexOutOfRangeException**: metin[0] yoktur

Bu tablodaki sonuçlar program çalıştırılarak alınmıştır. Sorunların kaynağı, algoritmanın "boşluktan sonra mutlaka bir harf gelir" varsayımıdır. Aşağıdaki çözüm her karakter için yalnızca şu soruyu sorar: "Bu harf bir kelimenin başında mı?" Bilgi `kelimeBasi` adlı bir bayrakta tutulur; hiçbir zaman `i + 1` indisine erişilmediği için taşma olamaz.

HER DURUMDA ÇALIŞAN ÇÖZÜM

```
private void radioButton3_CheckedChanged(object sender, EventArgs e)
{
    if (!radioButton3.Checked) return;

    string metin = textBox1.Text.ToLower();
    string sonuc = "";
    bool kelimeBasi = true;            // bir sonraki harf kelimenin ilk harfi mi?

    foreach (char harf in metin)
    {
        if (harf == ' ')
        {
            sonuc += harf;
            kelimeBasi = true;
        }
        else if (kelimeBasi)
        {
            sonuc += char.ToUpper(harf);
            kelimeBasi = false;
        }
        else
        {
            sonuc += harf;
        }
    }
    label2.Text = sonuc;
}
```

İPUCU

Aynı iş .NET'in hazır metoduyla da yapılabilir: `new System.Globalization.CultureInfo("tr-TR").TextInfo.ToTitleCase(metin.ToLower())`. Metin önce küçültülmelidir; ToTitleCase tamamı büyük harfle yazılmış kelimeleri kısaltma sayıp olduğu gibi bırakır.

ÖZET

- `Items.Contains(x)` bool döndürür, büyük-küçük harfe **duyarlıdır**. `FindStringExact(x)` indis ya da −1 döndürür, **duyarsızdır**.
- `Items.Count` eleman sayısını (int) verir. Döngüyle arama yapılacaksa döngüde silme yapılmaz; sonuç bir bayrak değişkeninde tutulur.
- `Math.Max` / `Math.Min` iç içe çağrılarak üç sayıya uygulanır. Ortalama için `double` kullanılmalıdır.
- `new Random()` ile nesne oluşturulur. `Next(alt, ust)` alt dahil, ust hariç sayı üretir. Benzersizlik için yeni sayı öncekilerle karşılaştırılır.
- `Checked` CheckBox ve RadioButton'ın seçili olup olmadığını tutar. Aynı türden kontroller bir dizide toplanıp döngüyle işlenebilir.
- `ToUpper` / `ToLower` yeni bir metin döndürür; sonuç geri atanmalıdır. Türkçe ayarda i ↔ İ ve ı ↔ I dönüşümü yapılır.

6.9 Kendini Test Et

Soruları önce kâğıt üzerinde çözün, sonra Visual Studio'da deneyerek kontrol edin. Açıklamalı cevaplar Cevap Anahtarı başlığındadır.

Soru 6.1 ÇIKTI TAHMİNİ Contains mi, FindStringExact mi? ◆◆◇

listBox1'de sırasıyla *mesut*, *hakan*, *ayşe*, *ali* bulunmaktadır. Aşağıdaki her ifadenin sonucunu yazınız.

	İfade
a	`listBox1.Items.Contains("ayşe")`
b	`listBox1.Items.Contains("Ayşe")`
c	`listBox1.Items.Contains("ayşe ")`
d	`listBox1.FindStringExact("AYŞE")`
e	`listBox1.FindStringExact("ay")`
f	`listBox1.FindString("ay")`
g	`listBox1.FindStringExact("veli")`

Ardından Bölüm 6.3'teki EKLE/BUL uygulamasında kullanıcı kutuya **Ayşe** yazıp EKLE'ye basıyor, sonra kutuya **AYŞE** yazıp BUL'a basıyor. Listeye yeni eleman eklenir mi? BUL hangi satırı seçili gösterir? Bu davranış bir kayıt sistemi için neden sorundur ve EKLE nasıl düzeltilmelidir?

Soru 6.2 HATA AVI Uyarı veren ama engellemeyen kod ◆◆◆

Bölüm 6.2'deki sunum çözümü (çözüm 2) aşağıda yeniden verilmiştir.

```
private void button1_Click(object sender, EventArgs e)
{
    string veri = textBox1.Text;
```

```
    for (int i = 0; i < listBox1.Items.Count; i++)
    {
        string data = Convert.ToString(listBox1.Items[i]);
        if (veri == data)
        {
            MessageBox.Show("Ekleyeceğiniz veri daha önce eklenmiştir.");
            listBox1.Items.RemoveAt(i);
        }
    }
    listBox1.Items.Add(veri);
}
```

a. Listede **A, B, C** varken kutuya **B** yazılıp butona basılırsa kaç mesaj çıkar ve liste ne olur?

b. Listede **A, B, B, C** varken (liste başka bir yerden doldurulmuştur) kutuya **B** yazılırsa kaç mesaj çıkar ve liste ne olur? İkinci B'nin neden silinmediğini döngünün turlarını izleyerek açıklayınız.

c. Kodu Contains kullanmadan, yalnızca döngüyle ve **hiçbir elemanı silmeden** doğru çalışacak şekilde yeniden yazınız.

Soru 6.3 HATA AVI Dördüncü sınıf neden birinci? ◆◆◇

```
private void button1_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked == true) { MessageBox.Show("1.sınıf öğrencisiniz."); }
    else if (radioButton2.Checked == true) { MessageBox.Show("2.sınıf öğrencisiniz."); }
    else if (radioButton3.Checked == true) { MessageBox.Show("3.sınıf öğrencisiniz."); }
    else { MessageBox.Show("1.sınıf öğrencisiniz."); }
}
```

a. Kullanıcı 4. *Sınıf*'ı seçip MESAJ'a basarsa ne yazar? Hiçbir seçenek seçmeden basarsa ne yazar?

b. radioButton'ların `Text` özellikleri *1. Sınıf*, *2. Sınıf*, *3. Sınıf*, *4. Sınıf*'tır. Dört RadioButton'ı bir dizide toplayıp seçili olanın `Text`'ini mesajda kullanan, hiçbiri seçili değilse uyaran kodu yazınız. Kodunuzda `MessageBox.Show` içinde sınıf numarası elle yazılmamalıdır.

Soru 6.4 İZLEME TABLOSU Sayısal lotoyu izlemek ◆◆◆

Bölüm 6.5'teki loto kodunda `uret.Next(1, 50)` çağrısının sırasıyla şu değerleri ürettiğini varsayalım:

```
7, 12, 7, 30, 12, 45, 3, 30, 22
```

a. Her `Next` çağrısında `i`'nin değerini, üretilen sayıyı ve sayının kabul edilip edilmediğini (edilmediyse hangi indisteki sayıyla çakıştığını) tabloya yazınız.

b. Labellarda hangi sayılar görünür? Toplam kaç kez `Next` çağrılmıştır?

c. `Next(1, 49)` yazılsaydı 49 sayısı hiç çıkabilir miydi? Neden?

d. Bir öğrenci `break` satırının silinmesi hâlinde aynı sayının iki kez çıkabileceğini iddia ediyor. Bu iddia doğru mu? `i--` yapıldıktan sonra iç döngü devam etseydi `dizi[i]` hangi sayıyı gösterir ve hangi sayılarla karşılaştırılırdı?

Çağrı	i	Üretilen	Sonuç
1			
2			
3			
4			
5			
6			
7			
8			
9			

Soru 6.5 ÇIKTI TAHMİNİ Değişmeyen metin ◆◆◇

Kod Türkçe ayarlı bir bilgisayarda çalışmaktadır. Her satırdan sonra label1'de ne yazacağını belirtiniz.

```
string s = "istanbul";
s.ToUpper();
label1.Text = s;                       // a

s = s.ToUpper();
label1.Text = s;                       // b

label1.Text = "ID".ToLower();          // c
label1.Text = "ID".ToLowerInvariant(); // d
label1.Text = "Fırat".ToUpper() + "Fırat".ToLower(); // e
```

Bir oturum açma ekranında kullanıcı adları karşılaştırılmadan önce küçük harfe çevrilmektedir. Kullanıcı adı *ADMIN* olarak kaydedilmiş, kullanıcı girişte *admin* yazmıştır. `kayitli.ToLower() == giris.ToLower()` karşılaştırması Türkçe ayarlı bir bilgisayarda neden `false` verir? Nasıl düzeltilir?

Soru 6.6 YENİDEN DÜZENLEME İlk harfi büyüt: sınır durumları ◆◆◆

Bölüm 6.8'deki sunum çözümünü düşününüz.

a. Kutuya *mehmet ali* (sonda boşluk yok) yazıldığında döngünün her turunda `i`'nin değerini ve `sonuc`'un aldığı değeri gösteren bir izleme yapınız. `i++` satırı kaldırılsaydı sonuç ne olurdu?

b. Kutuya *ali veli* (iki boşluk) yazıldığında ikinci kelimenin neden büyütülmediğini açıklayınız.

c. Sunum çözümünü değiştirmeden yalnızca başına eklenecek **bir** `if` satırıyla boş kutu hatasını önleyiniz. Sonda boşluk bulunan metin hatası için döngüde hangi koşul değiştirilmelidir?

Soru 6.7 KOD YAZMA Kredi sınırlı ders seçimi ◆◆◆

Bölüm 6.6'daki ders seçimi formunu geliştiriniz. Derslerin kredileri sırasıyla **5, 4, 4, 5, 3, 2**'dir (checkBox1–checkBox6).

- CheckBox'ları ve kredileri iki ayrı dizide tutunuz; aynı indis aynı derse karşılık gelsin.

- Hiç ders seçilmemişse uyarı verilsin. Seçilen derslerin toplam kredisi **15**'i aşarsa uyarı verilsin ve liste değiştirilmesin.
- Geçerli seçimde listeye *ders adı · kredi* satırları eklensin, label1'e *4 ders, 14 kredi* biçiminde özet yazılsın.

6.10 Cevap Anahtarı

Soru 6.1

	Sonuç	Neden?
a	**true**	Birebir aynı eleman vardır.
b	**false**	Contains büyük-küçük harfe duyarlıdır.
c	**false**	Sondaki boşluk da karakterdir; metinler aynı değildir.
d	**2**	FindStringExact duyarsızdır; *ayşe* 2. indistedir.
e	**−1**	Metnin tamamı *ay* olan eleman yoktur.
f	**2**	FindString, *ay* ile başlayan ilk elemanı bulur.
g	**−1**	Eleman yoktur.

Senaryo: *Ayşe* listeye **eklenir**, çünkü Contains duyarlıdır ve listede *ayşe* ile *Ayşe* farklı sayılır. BUL'da AYŞE aranınca FindStringExact ilk eşleşmeyi, yani 2. indisteki *ayşe*'yi seçer; yeni eklenen *Ayşe* hiçbir zaman bulunamaz. Aynı kişi listede iki kez yer alır. EKLE de aynı kurala göre çalışmalıdır: `if (listBox1.FindStringExact(veri) == -1) listBox1.Items.Add(veri);` Böylece ekleme ve arama aynı karşılaştırmayı kullanır. Ayrıca girişten önce `Trim()` ile baştaki ve sondaki boşluklar temizlenmelidir.

Denetim: Sonuçlar gerçek bir ListBox üzerinde çalıştırılarak alınmıştır.

Soru 6.2

a) 1 mesaj. i = 1'de B bulunur, mesaj çıkar ve B silinir; döngü bittikten sonra B sona eklenir. Liste: **A, C, B**.

b) 1 mesaj. Liste: **A, B, C, B**.

Tur	i	Count	Items[i]	İşlem	Liste (tur sonu)
1	0	4	A	—	A B B C
2	1	4	B	mesaj, RemoveAt(1)	A B C
3	2	3	C	—	A B C
—	3	3		şart yanlış	A B C

Silme sonrasında ikinci B 1. indise kayar, ama i bir sonraki turda 2 olur. Böylece kayan eleman atlanır. Döngüden sonra B tekrar eklenir.

c)

```
private void button1_Click(object sender, EventArgs e)
{
    string veri = textBox1.Text;
    bool varMi = false;                    // bayrak (flag) değişkeni
```

```
    for (int i = 0; i < listBox1.Items.Count; i++)
    {
        if (Convert.ToString(listBox1.Items[i]) == veri)
        {
            varMi = true;
            break;                          // bulundu, aramaya gerek kalmadı
        }
    }

    if (varMi)
        MessageBox.Show("Ekleyeceğiniz veri daha önce eklenmiştir.");
    else
        listBox1.Items.Add(veri);
}
```

Döngü yalnızca arar ve `break` ile ilk eşleşmede durur. Karar döngüden sonra tek bir yerde verilir; liste hiç değiştirilmeden uyarı gösterilir.

Denetim: Her iki senaryo sunum kodu çalıştırılarak doğrulanmıştır.

Soru 6.3

a) İki durumda da **1.sınıf öğrencisiniz.** yazar. 4. sınıf seçiliyken ilk üç koşul yanlış olduğu için `else` çalışır; hiçbir şey seçili değilken de aynı şey olur. Son blok aslında "geri kalan her durum" anlamına gelir; 4. sınıf için ayrı bir `else if` yazılmalı ve `else` "seçim yapılmadı" uyarısına ayrılmalıdır.

b)

```
private void button1_Click(object sender, EventArgs e)
{
    RadioButton[] siniflar = { radioButton1, radioButton2, radioButton3, radioButton4 };

    foreach (RadioButton rb in siniflar)
    {
        if (rb.Checked)
        {
            MessageBox.Show(rb.Text + " öğrencisiniz.");
            return;
        }
    }
    MessageBox.Show("Lütfen sınıfınızı seçiniz.");
}
```

4. sınıf seçiliyken mesaj *4. Sınıf öğrencisiniz.* olur. `return` seçili buton bulununca metodu bitirir; döngü hiçbir seçili buton bulamadan biterse uyarı satırına gelinir. Sınıf eklense bile mesaj kodu değişmez.

Soru 6.4

Çağrı	i	Üretilen	Sonuç
1	0	7	kabul → dizi[0]
2	1	12	kabul → dizi[1]
3	2	7	**red**: dizi[0] ile aynı, i tekrar 2
4	2	30	kabul → dizi[2]
5	3	12	**red**: dizi[1] ile aynı, i tekrar 3

Çağrı	i	Üretilen	Sonuç
6	3	45	kabul → dizi[3]
7	4	3	kabul → dizi[4]
8	5	30	**red**: dizi[2] ile aynı, i tekrar 5
9	5	22	kabul → dizi[5]

b) Labellar: **7, 12, 30, 45, 3, 22.** `Next` **9 kez** çağrılmıştır.

c) Hayır. Üst sınır sonuca dahil değildir; `Next(1, 49)` en fazla 48 üretir.

d) İddia **yanlıştır**. `i--` yapıldıktan sonra `dizi[i]` bir önceki, zaten **kabul edilmiş** sayıyı gösterir. Bu sayı daha önce kendinden önceki tüm sayılardan farklı bulunduğu için kalan karşılaştırmaların hiçbiri eşleşmez; iç döngü kendi şartıyla biter ve dış döngünün `i++` adımı çakışan konuma geri döner. Yani `break` olmadan da sonuç doğrudur. `break` yalnızca gereksiz karşılaştırmaları önler ve "çakışma bulundu, bu tur bitti" niyetini kodda açıkça gösterir.

Denetim: Tablo, verilen sayı dizisi koda beslenerek elde edilmiştir. (d) şıkkı, break satırı silinmiş kodun 200 000 kez çalıştırılmasıyla doğrulanmıştır.

Soru 6.5

	label	Neden?
a	**istanbul**	ToUpper'ın sonucu atanmadı; string değiştirilemez.
b	**İSTANBUL**	Sonuç geri atandı; Türkçe ayarda i → İ.
c	**ıd**	Türkçe ayarda I → ı.
d	**id**	Invariant (dilden bağımsız) dönüşümde I → i.
e	**FIRATfırat**	ı → I, I → ı; iki metin birleştirilir.

Oturum açma: ADMIN.ToLower() Türkçe ayarda *admın* olur; *admin* ile eşit değildir. Dilden bağımsız metinlerde `ToLowerInvariant()` kullanılmalı ya da doğrudan büyük-küçük harfe duyarsız karşılaştırma yapılmalıdır: `string.Equals(kayitli, giris, StringComparison.OrdinalIgnoreCase)`.

Denetim: Sonuçlar Türkçe bölge ayarlı .NET ortamında alınmıştır.

Soru 6.6

a) *mehmet ali* (10 karakter). İlk harf boşluk olmadığı için sonuc = M, j = 1.

i (tur başı)	metin[i]	İşlem	sonuc (tur sonu)
1–5	e h m e t	olduğu gibi eklenir	Mehmet
6	boşluk	" " + "A" eklenir, i++ → 7	Mehmet A
8–9	l i	olduğu gibi eklenir	Mehmet Ali

`i++` kaldırılsaydı boşluktan sonraki harf hem büyük hem küçük olarak iki kez eklenirdi: *Mehmet Aali*.

b) İlk boşlukta "boşluk + sonraki karakter" eklenir; sonraki karakter de boşluktur. `i++` ile bu ikinci boşluk atlanır ve *v* sıradan bir karakter gibi küçük eklenir.

c) Başa `if (metin.Length == 0) { label2.Text = ""; return; }` eklenir. Sonda boşluk hatası için boşluk koşulu `if (metin[i] == ' ' && i + 1 < metin.Length)` yapılmalıdır; böylece son karakter boşluksa `metin[i + 1]`'e erişilmez. (Bölüm 6.8'deki bayraklı çözüm bu iki sorunu ve çift boşluk sorununu birlikte çözer.)

Soru 6.7

```
private void button1_Click(object sender, EventArgs e)
{
    CheckBox[] dersler = { checkBox1, checkBox2, checkBox3, checkBox4, checkBox5, checkBox6 };
    int[] krediler = { 5, 4, 4, 5, 3, 2 };

    int adet = 0, toplamKredi = 0;
    for (int i = 0; i < dersler.Length; i++)
    {
        if (dersler[i].Checked)
        {
            adet++;
            toplamKredi += krediler[i];
        }
    }

    if (adet == 0)
    {
        MessageBox.Show("En az bir ders seçmelisiniz.");
        return;
    }
    if (toplamKredi > 15)
    {
        MessageBox.Show("Kredi sınırı aşıldı: " + toplamKredi + " / 15");
        return;
    }

    listBox1.Items.Clear();
    for (int i = 0; i < dersler.Length; i++)
    {
        if (dersler[i].Checked)
            listBox1.Items.Add(dersler[i].Text + "\t" + krediler[i] + " kredi");
    }
    label1.Text = adet + " ders, " + toplamKredi + " kredi";
}
```

Devam edecektir...

Bu kitap, dersler işlendikçe yeni konularla güncellenmektedir.
Elinizdeki sürüm, kitabın yayımlanan son hâlidir.

SON GÜNCELLEME · 27 EKİM 2026